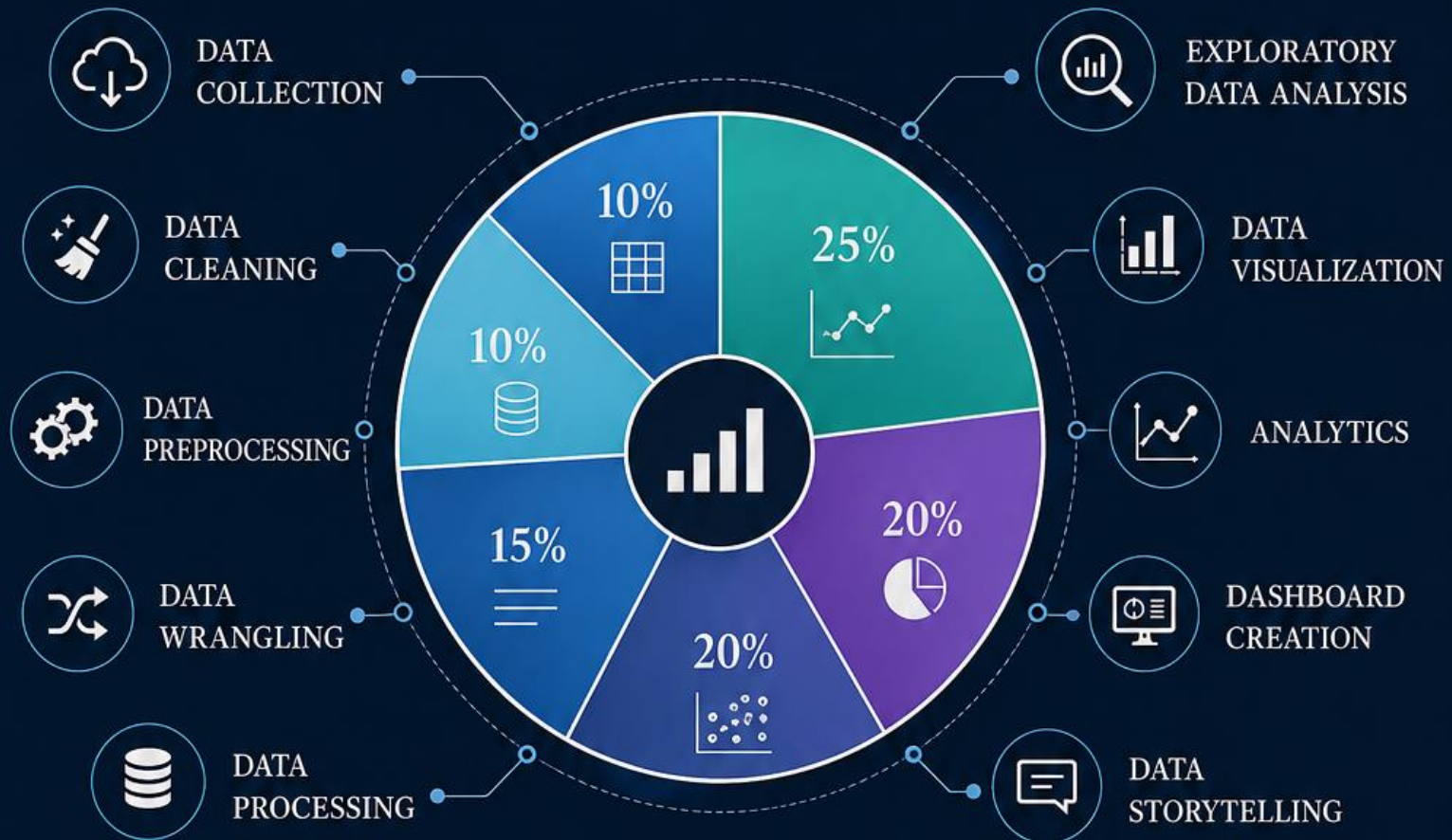


DATA ANALYSIS WITH PYTHON

COMPREHENSIVE LEARNING NOTES

Explore, Clean, Analyze and Visualize Data to
Extract Insights and Drive Impact.



TOOLS I WORKED WITH



Python



Pandas



NumPy



Matplotlib



Seaborn



Plotly



Jupyter
Notebook



PREPARED BY

KASHIF MAQBOOL JOIYA

Data Scientist | AI Engineer



GitHub

github.com/KashifMaqbool

Table of Contents

1	DATA ANALYSIS WITH PYTHON	1
1.1	KEY PYTHON LIBRARIES FOR DATA ANALYSIS	1
1.2	COURSE MODULES	1
1.2.1	MODULE 1: DATA UNDERSTANDING AND IMPORTING	1
1.2.2	MODULE 2: DATA WRANGLING AND PREPROCESSING	1
1.2.3	MODULE 3: EXPLORATORY DATA ANALYSIS (EDA)	1
1.2.4	MODULE 4: REGRESSION AND MODEL EVALUATION	1
1.2.5	MODULE 5: MODEL REFINEMENT AND OPTIMIZATION	1
1.3	HANDS-ON PROJECT	1
1.4	PREREQUISITES	1
2	DATA ANALYSIS WITH PYTHON: USED CAR PRICES DATASET	2
2.1	INTRODUCTION TO THE DATASET	2
2.1.1	DATASET STRUCTURE	2
2.2	UNDERSTANDING THE ATTRIBUTES	2
2.2.1	SYMBOLING	2
2.2.2	NORMALIZED LOSSES	2
2.2.3	ADDITIONAL ATTRIBUTES	2
2.3	TARGET VARIABLE	2
2.4	GOAL OF THE PROJECT	2
3	DATA ANALYSIS IN PYTHON: KEY LIBRARIES	3
3.1	INTRODUCTION TO PYTHON DATA ANALYSIS LIBRARIES	3
3.2	SCIENTIFIC COMPUTING LIBRARIES	3
3.2.1	PANDAS	3
3.2.2	NUMPY	3
3.2.3	SCIPY	3
3.3	DATA VISUALIZATION LIBRARIES	3
3.3.1	MATPLOTLIB	3
3.3.2	SEABORN	3
3.4	MACHINE LEARNING AND STATISTICAL MODELING LIBRARIES	3
3.4.1	SCIKIT-LEARN	3
3.4.2	STATSMODELS	4
4	READING DATA USING PANDAS	4
4.1	INTRODUCTION	4
4.2	DATA ACQUISITION PROCESS	4
4.2.1	DATA FORMAT	4
4.2.2	FILE PATH	4
4.3	EXAMPLE: USED CAR DATASET	4
4.4	READING DATA INTO PANDAS	4

4.4.1	HANDLING MISSING COLUMN HEADERS	4
4.5	EXPLORING THE DATA	5
4.6	ASSIGNING COLUMN NAMES	5
4.7	EXPORTING DATA	5
4.8	SUMMARY	5
5	TYPES OF DATA	5
5.1	INTRODUCTION	5
5.2	QUALITATIVE OR CATEGORICAL DATA	6
5.2.1	EXAMPLES OF QUALITATIVE DATA	6
5.2.2	CLASSIFICATION OF QUALITATIVE DATA	6
5.2.3	DIFFERENCE BETWEEN NOMINAL AND ORDINAL DAT	7
5.3	QUANTITATIVE DATA	7
5.3.1	EXAMPLES OF QUANTITATIVE DATA	7
5.3.2	CLASSIFICATION OF QUANTITATIVE DATA	8
5.3.3	DIFFERENCE BETWEEN DISCRETE AND CONTINUOUS DATA	8
6	EXPLORING DATA USING PANDAS	8
6.1	INTRODUCTION	8
6.2	UNDERSTANDING DATA TYPES	9
6.2.1	COMMON DATA TYPES IN PANDAS	9
6.2.2	WHY CHECK DATA TYPES?	9
6.2.3	VIEWING DATA TYPES	9
6.3	STATISTICAL SUMMARY OF DATA	9
6.3.1	DEFAULT BEHAVIOR	9
6.3.2	INCLUDING ALL COLUMNS	9
6.3.3	SUMMARY PARAMETERS IN THE OUTPUT	9
6.3.4	HANDLING MISSING VALUES	11
6.4	DATASET INFORMATION	11
6.5	SUMMARY	11
7	ACCESSING DATABASES USING PYTHON	12
7.1	INTRODUCTION	12
7.2	CONNECTING TO DATABASES	12
7.2.1	SQL APIs	12
7.2.2	ENDING DATABASE CONNECTIONS	12
7.3	PYTHON DB API	12
7.3.1	KEY ADVANTAGES	12
7.3.2	CORE CONCEPTS	12
7.4	WORKING WITH CURSORS	12
7.4.1	CURSOR METHODS	13
7.5	SAMPLE WORKFLOW IN PYTHON	13
7.6	SUMMARY	14
8	LESSON SUMMARY	14

8.1	KEY CONCEPTS LEARNED	14
8.1.1	DATASET STRUCTURE	14
8.1.2	PYTHON LIBRARIES	14
8.1.3	DATA HANDLING WITH PANDAS	14
8.1.4	STATISTICAL ANALYSIS	15
8.1.5	DATABASE CONNECTIVITY WITH PYTHON	15
9	OVERVIEW: LAPTOP PRICING DATA SET	15
9.1	PARAMETERS	15
9.1.1	MANUFACTURER	15
9.1.2	CATEGORY	15
9.1.3	GPU	16
9.1.4	OS	16
9.1.5	CPU_CORE	16
9.1.6	SCREEN_SIZE_CM	16
9.1.7	CPU_FREQUENCY	16
9.1.8	RAM_GB	16
9.1.9	STORAGE_GB_SSD	16
9.1.10	WEIGHT_KG	16
9.1.11	PRICE	17
10	DATA PRE-PROCESSING TECHNIQUES	17
10.1	INTRODUCTION TO DATA PRE-PROCESSING	17
10.2	TOPICS COVERED IN THIS MODULE	17
10.2.1	HANDLING MISSING VALUES	17
10.2.2	DATA FORMATTING	17
10.2.3	DATA NORMALIZATION	17
10.2.4	DATA BINNING	17
10.2.5	CATEGORICAL VARIABLES	17
10.3	WORKING WITH DATA IN PYTHON	17
10.3.1	MANIPULATING DATA FRAMES	18
11	HANDLING MISSING VALUES IN DATA	18
11.1	INTRODUCTION TO MISSING VALUES	18
11.2	STRATEGIES TO HANDLE MISSING VALUES	18
11.2.1	CHECKING DATA SOURCES	18
11.2.2	DROPPING MISSING DATA	18
11.2.3	REPLACING MISSING DATA	18
11.2.4	ADVANCED REPLACEMENT TECHNIQUES	18
11.3	METHODS IN PYTHON FOR MISSING VALUES	18
11.3.1	DROPPING DATA	18
11.3.2	REPLACING DATA	18
11.4	SUMMARY	19
12	HANDLING DATA WITH DIFFERENT FORMATS AND UNITS	19

12.1	INTRODUCTION TO DATA FORMATTING	19
12.1.1	IMPORTANCE OF DATA FORMATTING	19
12.2	CONVERTING UNITS IN DATA	19
12.2.1	CONVERSION FORMULA	19
12.2.2	RENAMING COLUMNS	19
12.3	HANDLING DATA TYPES	19
12.3.1	IDENTIFYING DATA TYPES	20
12.3.2	CONVERTING DATA TYPES	20
12.3.3	COMMON DATA TYPES IN PANDAS	20
12.4	SUMMARY	20
13	DATA NORMALIZATION	20
13.1	INTRODUCTION TO DATA NORMALIZATION	20
13.1.1	IMPORTANCE OF NORMALIZATION	20
13.2	EXAMPLES OF NORMALIZATION	20
13.3	METHODS OF NORMALIZATION	21
13.3.1	SIMPLE FEATURE SCALING	21
13.3.2	MIN-MAX METHOD	21
13.3.3	Z-SCORE METHOD (STANDARD SCORE)	21
13.4	APPLYING NORMALIZATION IN PYTHON	21
13.5	SUMMARY	21
14	BINNING IN DATA PREPROCESSING	22
14.1	INTRODUCTION TO BINNING	22
14.1.1	IMPORTANCE OF BINNING	22
14.2	EXAMPLE: BINNING CAR PRICE DATA	22
14.2.1	CREATING EQUAL-WIDTH BINS	22
14.2.2	APPLYING BINNING IN PYTHON	22
14.2.3	INSIGHTS FROM THE HISTOGRAM	23
14.3	SUMMARY	23
15	CONVERTING CATEGORICAL VARIABLES TO QUANTITATIVE VARIABLES	24
15.1	INTRODUCTION	24
15.2	EXAMPLE: FUEL TYPE IN CAR DATASET	24
15.2.1	ONE-HOT ENCODING TECHNIQUE	24
15.3	IMPLEMENTING ONE-HOT ENCODING IN PYTHON	24
15.4	SUMMARY	25
16	LESSON SUMMARY	25
16.1	KEY TAKEAWAYS	25
16.1.1	DATA FORMATTING	25
16.1.2	DATA TYPES AND NORMALIZATION	25
16.1.3	BINNING AND VISUALIZATION	25
16.1.4	ENCODING CATEGORICAL VARIABLES	25

17	OUTLIER DETECTION	26
17.1	DEFINITION	26
17.2	PURPOSE	26
17.3	METHODS	26
17.4	INTERQUARTILE RANGE (IQR)	26
17.5	FINDING OUTLIERS USING IQR	26
17.6	EXAMPLE	27
18	INTRODUCTION TO EXPLORATORY DATA ANALYSIS (EDA)	27
18.1	KEY LEARNING AREAS	27
18.1.1	DESCRIPTIVE STATISTICS	27
18.1.2	GROUPING DATA USING GROUPBY	28
18.1.3	ANOVA (ANALYSIS OF VARIANCE)	28
18.1.4	CORRELATION ANALYSIS	28
18.1.5	ADVANCED CORRELATION TECHNIQUES	28
19	DESCRIPTIVE STATISTICS	28
19.1	USING THE DESCRIBE() FUNCTION IN PANDAS	28
19.1.1	PURPOSE	28
19.2	WORKING WITH CATEGORICAL VARIABLES	29
19.2.1	SUMMARIZING CATEGORICAL DATA	29
19.3	VISUALIZING DATA WITH BOX PLOTS	29
19.3.1	KEY FEATURES OF A BOX PLOT	29
19.3.2	EXAMPLE	30
19.4	ANALYZING CONTINUOUS VARIABLES WITH SCATTER PLOTS	30
19.4.1	EXPLORING RELATIONSHIPS WITH SCATTER PLOTS	30
19.4.2	BEST PRACTICES FOR SCATTER PLOTS	31
19.4.3	INSIGHTS FROM THE SCATTER PLOT	31
20	GROUPING AND TRANSFORMING DATA IN PANDAS	31
20.1	UNDERSTANDING THE GROUPBY METHOD	32
20.1.1	PURPOSE OF GROUPING	32
20.2	GROUPING BY MULTIPLE VARIABLES	32
20.3	USING PIVOT TABLES FOR EASIER VISUALIZATION	32
20.3.1	CONCEPT OF PIVOT TABLES	32
20.3.2	EXAMPLE IN PANDAS	33
20.4	VISUALIZING PIVOT TABLES WITH HEAT MAPS	33
20.4.1	CREATING A HEAT MAP	33
20.4.2	INSIGHTS FROM THE HEAT MAP	33
21	DATA VISUALIZATION COMMANDS IN PYTHON	34
21.1	IMPORTING LIBRARIES	34
21.1.1	IMPORTING MATPLOTLIB	34

21.1.2	IMPORTING SEABORN	34
21.2	MATPLOTLIB FUNCTIONS	34
21.2.1	STANDARD LINE PLOT	34
21.2.2	SCATTER PLOT	35
21.2.3	HISTOGRAM	36
21.2.4	BAR PLOT	36
21.2.5	PSEUDO COLOR PLOT	37
21.3	SEABORN FUNCTIONS	37
21.3.1	REGRESSION PLOT	37
21.3.2	BOX AND WHISKER PLOT	38
21.3.3	RESIDUAL PLOT	39
21.3.4	KDE PLOT	40
21.3.5	DISTRIBUTION PLOT	41
21.4	CONCLUSION	41
22	CORRELATION BETWEEN VARIABLES	42
22.1	UNDERSTANDING CORRELATION	42
22.1.1	EXAMPLES OF CORRELATION	42
22.2	CORRELATION VS. CAUSATION	42
22.3	VISUALIZING CORRELATION	42
22.3.1	EXAMPLE 1: ENGINE SIZE VS. PRICE	42
22.3.2	EXAMPLE 2: HIGHWAY MILES PER GALLON VS. PRICE	42
22.3.3	EXAMPLE 3: PEAK RPM VS. PRICE	43
22.4	SUMMARY	43
23	CORRELATION BETWEEN VARIABLES	43
23.1	INTRODUCTION TO CORRELATION METHODS	43
23.2	PEARSON CORRELATION	43
23.2.1	INTERPRETING THE CORRELATION COEFFICIENT	43
23.2.2	INTERPRETING THE P-VALUE	43
23.2.3	STRONG CORRELATION EXAMPLE	44
23.3	CORRELATION HEAT MAP	44
23.3.1	UNDERSTANDING THE HEAT MAP	44
23.4	SUMMARY	45
24	CHI-SQUARE TEST FOR CATEGORICAL VARIABLES	45
24.1	INTRODUCTION	45
24.2	CONCEPT	45
24.3	HYPOTHESES	45
24.3.1	NULL HYPOTHESIS (H_0)	45
24.3.2	ALTERNATIVE HYPOTHESIS (H_1)	45
24.4	FORMULA	45
24.5	CHI-SQUARE DISTRIBUTION TABLE EXAMPLE	46
24.6	PYTHON IMPLEMENTATION EXAMPLE	46
24.7	APPLICATIONS	47
24.8	PRACTICAL EXAMPLE – WEAK CORRELATION	47

24.8.1	SCENARIO	47
24.8.2	STEP 1: CALCULATE EXPECTED FREQUENCIES	47
24.8.3	STEP 2: COMPUTE CHI-SQUARE STATISTIC	47
24.8.4	STEP 3: DEGREES OF FREEDOM	47
24.8.5	STEP 4: INTERPRETATION	47
24.9	PRACTICAL EXAMPLE – STRONG ASSOCIATION	48
24.9.1	SCENARIO	48
24.9.2	STEP 1: CALCULATE EXPECTED FREQUENCIES	48
24.9.3	STEP 2: COMPUTE CHI-SQUARE STATISTIC	48
24.9.4	STEP 3: DEGREES OF FREEDOM	48
24.9.5	STEP 4: INTERPRETATION	48
24.10	CONCLUSION	48
25	LESSON SUMMARY	48
25.1	KEY STATISTICAL TOOLS	49
25.2	DATA VISUALIZATION TECHNIQUES	49
25.3	UNDERSTANDING CORRELATION	49
25.4	PEARSON CORRELATION	49
25.5	HEATMAPS FOR CORRELATION	49
26	MODEL DEVELOPMENT	50
26.1	OVERVIEW	50
26.2	UNDERSTANDING MODELS AND ESTIMATORS	50
26.2.1	EXAMPLE	50
26.3	IMPORTANCE OF MORE DATA	50
26.4	EXPLORING DIFFERENT TYPES OF MODELS	50
27	LINEAR REGRESSION	51
27.1	INTRODUCTION	51
27.2	SIMPLE LINEAR REGRESSION (SLR)	51
27.2.1	UNDERSTANDING THE CONCEPT	51
27.2.2	EXAMPLE	51
27.2.3	MODEL TRAINING	52
27.2.4	NOISE IN DATA	53
27.2.5	PREDICTION PROCESS	53
27.3	IMPLEMENTING SIMPLE LINEAR REGRESSION IN PYTHON	54
27.3.1	STEPS	54
27.3.2	MODEL PARAMETERS	54
27.4	MULTIPLE LINEAR REGRESSION (MLR)	54
27.4.1	CONCEPT	54
27.4.2	EXAMPLE	54
27.4.3	VISUALIZATION	55
27.4.4	IMPLEMENTATION IN PYTHON	55
27.4.5	MODEL ATTRIBUTES	55
27.5	SUMMARY	56

28	MODEL EVALUATION USING VISUALIZATION	56
28.1	INTRODUCTION	56
28.2	REGRESSION PLOTS	56
28.2.1	CREATING A REGRESSION PLOT	56
28.3	RESIDUAL PLOTS	56
28.3.1	INTERPRETATION	57
28.3.2	IDENTIFYING MODEL ISSUES	57
28.3.3	CREATING A RESIDUAL PLOT	58
28.4	DISTRIBUTION PLOTS	58
28.4.1	HOW IT WORKS	59
28.4.2	EXAMPLE: DISTRIBUTION PLOT FOR PRICE PREDICTION	59
28.4.3	CODE EXAMPLE FOR DISTRIBUTION PLOT	60
29	POLYNOMIAL REGRESSION AND PIPELINES	61
29.1	INTRODUCTION	61
29.2	POLYNOMIAL REGRESSION	61
29.2.1	PURPOSE	61
29.2.2	CURVILINEAR RELATIONSHIP	61
29.2.3	EXAMPLE IN PYTHON	61
29.3	FEATURE SCALING AND NORMALIZATION	62
29.4	PIPELINES	63
29.4.1	PURPOSE	63
29.4.2	CREATING A PIPELINE	63
29.4.3	EXAMPLE WORKFLOW	64
30	FEATURE SCALING / NORMALIZATION	65
30.1	UNDERSTANDING FEATURE SCALING	65
30.2	EXAMPLE FEATURE RANGES	65
30.3	WHAT NORMALIZATION DOES	65
30.4	WHEN FEATURE SCALING IS NEEDED	65
30.5	STEPS FOR FEATURE SCALING	65
30.5.1	STEP 1: IMPORT STANDARDSCALER	65
30.5.2	STEP 2: CREATE SCALER AND FIT IT TO DATA	65
30.5.3	STEP 3: TRANSFORM THE DATA	66
30.6	BEFORE AND AFTER SCALING	66
31	POLYNOMIAL TRANSFORMATION	66
31.1	UNDERSTANDING POLYNOMIAL TRANSFORMATION	66
31.2	EXAMPLE	66
31.3	SHAPE CHANGE EXAMPLE	66
31.4	REASON FOR 15 FEATURES	67
31.5	CODE FOR POLYNOMIAL TRANSFORMATION	67
31.6	WHY USE POLYNOMIAL TRANSFORMATION?	67
31.7	COMPARISON	67
31.8	SUMMARY	67

32	KERNEL DENSITY ESTIMATION (KDE) PLOTS FOR MODEL EVALUATION	67
32.1	INTRODUCTION	67
32.2	WHY USE KDE PLOTS?	67
32.3	IMPLEMENTING KDE PLOTS IN PYTHON	68
32.4	EXAMPLE: EVALUATING A REGRESSION MODEL	68
32.5	INTERPRETATION OF THE KDE PLOT	69
32.6	CONCLUSION	69
33	MODEL EVALUATION	69
33.1	OVERVIEW	69
33.2	MEAN SQUARED ERROR (MSE)	69
33.2.1	STEPS TO CALCULATE MSE	70
33.2.2	EXAMPLE	70
33.2.3	PYTHON IMPLEMENTATION	70
33.3	R-SQUARED (COEFFICIENT OF DETERMINATION)	70
33.3.1	CONCEPT	70
33.3.2	FORMULA	70
33.3.3	INTERPRETATION	70
33.3.4	EXAMPLE EVALUATION	71
33.3.5	PYTHON IMPLEMENTATION	73
33.4	ADDITIONAL NOTE	73
34	PREDICTION AND DECISION MAKING	73
34.1	ENSURING MODEL VALIDITY	73
34.2	EXAMPLE: PREDICTING CAR PRICE	73
34.3	INTERPRETING COEFFICIENTS	74
34.4	UNREALISTIC PREDICTIONS AND MODEL ASSUMPTIONS	74
34.5	GENERATING SEQUENCES WITH NUMPY	74
34.6	VISUAL EVALUATION	75
34.7	MEAN SQUARED ERROR (MSE)	76
34.8	R-SQUARED (COEFFICIENT OF DETERMINATION)	78
34.9	COMPARING MODELS	80
34.10	CONCLUSION	80
35	LESSON SUMMARY	80
35.1	OVERVIEW	80
35.2	KEY CONCEPTS	80
35.2.1	LINEAR REGRESSION	80
35.2.2	SIMPLE LINEAR REGRESSION (SLR)	81
35.2.3	VISUALIZATION TECHNIQUES	81
35.2.4	POLYNOMIAL REGRESSION	81
35.2.5	FEATURE TRANSFORMATION	81
35.2.6	PIPELINES	81
35.3	MODEL EVALUATION	81
35.3.1	MEAN SQUARED ERROR (MSE)	81

35.3.2	R-SQUARED (COEFFICIENT OF DETERMINATION)	81
35.3.3	CHOOSING A GOOD MODEL	82
35.3.4	USING RESIDUAL PLOTS FOR EVALUATION	82
35.4	SUMMARY	82
36	MODEL EVALUATION AND CROSS VALIDATION	82
36.1	MODEL EVALUATION	82
36.1.1	TRAINING DATA	82
36.1.2	TESTING DATA	82
36.2	TRAINING AND TESTING SETS	83
36.3	TRAIN TEST SPLIT FUNCTION	83
36.3.1	PARAMETERS	83
36.3.2	OUTPUT	83
36.4	GENERALIZATION ERROR	83
36.5	ACCURACY AND PRECISION IN DATA SPLITTING	84
36.6	CROSS VALIDATION	84
36.7	CROSS VALIDATION SCORE	85
36.7.1	PARAMETERS	85
36.8	CROSS-VALIDATION PROCESS	85
36.9	CROSS-VALIDATION PREDICTION	86
36.10	CROSS-VALIDATION PREDICTION PROCESS	87
37	POLYNOMIAL ORDER SELECTION, UNDERFITTING, AND OVERFITTING	87
37.1	DATA ANALYSIS WITH PYTHON	87
37.2	POLYNOMIAL REGRESSION AND MODEL SELECTION	88
37.3	UNDERFITTING	88
37.4	GOOD FIT	88
37.5	OVERFITTING	89
37.6	MEAN SQUARED ERROR AND POLYNOMIAL ORDER	89
37.6.1	ERROR BEHAVIOR	90
37.7	IRREDUCIBLE ERROR	90
37.8	MODEL ASSUMPTION ERROR	90
37.9	REAL DATA EXAMPLE: HORSEPOWER VS PRICE	91
37.9.1	MEAN MODEL	91
37.9.2	LINEAR MODEL	91
37.9.3	SECOND AND THIRD ORDER	91
37.9.4	FOURTH ORDER	91
37.10	USING R^2 FOR MODEL SELECTION	91
37.11	CODE FOR FINDING BEST POLYNOMIAL ORDER	91
37.12	EXPLANATION OF THE CODE	92
37.13	KEY SUMMARY	92
38	INTRODUCTION TO RIDGE REGRESSION	92
38.1	WHAT IS RIDGE REGRESSION?	93
39	RIDGE REGRESSION	93

39.1	INTRODUCTION	93
39.2	OVERFITTING IN POLYNOMIAL REGRESSION	93
39.3	HOW RIDGE REGRESSION SOLVES OVERFITTING	94
39.4	EFFECT OF ALPHA VALUES	94
39.4.1	IF ALPHA IS TOO SMALL	94
39.4.2	IF ALPHA IS TOO LARGE	95
39.5	RIDGE REGRESSION IN SCIKIT-LEARN	95
39.6	SELECTING THE BEST ALPHA	95
39.7	ALPHA SELECTION USING R^2	96
39.8	BEHAVIOR OF R^2 WITH ALPHA	96
39.9	TRAINING VS VALIDATION PERFORMANCE	96
40	GRID SEARCH AND HYPERPARAMETER TUNING	97
40.1	DATA ANALYSIS WITH PYTHON	97
40.2	INTRODUCTION TO GRID SEARCH	97
40.3	WHAT ARE HYPERPARAMETERS?	97
40.4	HOW GRID SEARCH WORKS	97
40.5	DATASET SPLIT FOR GRID SEARCH	98
40.5.1	TRAINING SET	98
40.5.2	VALIDATION SET	98
40.5.3	TEST SET	98
40.6	PARAMETER GRID	99
40.7	GRIDSEARCHCV IN SCIKIT-LEARN	99
40.8	FIT THE GRID SEARCH MODEL	99
40.9	BEST HYPERPARAMETER VALUES	99
40.10	ACCESS VALIDATION RESULTS	100
40.11	MULTIPLE HYPERPARAMETERS	100
40.12	SCORE RESULTS	101
40.13	KEY SUMMARY	101
41	LESSON SUMMARY	102
41.1	TRAIN-TEST SPLIT	102
41.2	GENERALIZATION ERROR	102
41.3	CROSS VALIDATION	102
41.4	POLYNOMIAL ORDER SELECTION	102
41.5	RIDGE REGRESSION	103
41.6	SELECTING THE BEST ALPHA VALUE	103
41.7	GRID SEARCH	103
41.8	GRIDSEARCHCV()	103

Module 1

1 Data Analysis with Python

Python is one of the top programming languages for data science, and now that you know how to program with Python, you will learn to use it for analyzing data.

1.1 Key Python Libraries for Data Analysis

In this course, you will work with essential Python libraries widely used in industries like healthcare, finance, insurance, and technology:

- **NumPy**
- **Pandas**
- **scikit-learn (sklearn)**

These libraries are fundamental for data analysis, machine learning, and building data pipelines.

1.2 Course Modules

The course is divided into multiple modules, each focusing on different aspects of data analysis and machine learning.

1.2.1 Module 1: Data Understanding and Importing

- Understanding dataset characteristics
- Overview of Python packages for data analysis
- Importing and exploring data

1.2.2 Module 2: Data Wrangling and Preprocessing

- Handling missing values
- Data formatting
- Data normalization

1.2.3 Module 3: Exploratory Data Analysis (EDA)

- Descriptive statistics
- GroupBy operations
- Correlation analysis
- Key statistical methods for data insights

1.2.4 Module 4: Regression and Model Evaluation

- Linear regression
- Polynomial regression
- Pipeline creation
- In-sample evaluation measures
- Prediction and decision making

1.2.5 Module 5: Model Refinement and Optimization

- Model evaluation
- Overfitting and underfitting
- Model selection techniques
- Ridge regression
- Grid search for hyperparameter tuning

1.3 Hands-On Project

At the end of the course, you will apply your skills to a **real-world dataset** through a hands-on project, giving you practical experience in data analysis and machine learning.

1.4 Prerequisites

To get the most out of this course, you should have:

- Basic Python programming knowledge
- High school level mathematics

2 Data Analysis with Python: Used Car Prices Dataset

2.1 Introduction to the Dataset

The dataset on **used car prices**, created by **Jeffrey C. Schlimmer**, is provided in **CSV format**, where each value is separated by commas for easy import into most tools and applications. Each line represents a row in the dataset.

2.1.1 Dataset Structure

- Typically, the **first row** contains column headers, but in this dataset, it is simply another row of data.
- The dataset has **26 columns**, each describing a specific attribute of cars.

2.2 Understanding the Attributes

Key attributes include:

2.2.1 Symboling

- Represents the **insurance risk level** of a car.
- Cars receive an initial risk factor based on price, which can be adjusted:
 - **+3**: High risk
 - **-3**: Low risk / Safe

2.2.2 Normalized Losses

- Measures the **relative average loss payment** per insured vehicle year.
- Normalized across cars of specific size categories (e.g., two-door small cars, station wagons, sports cars).
- Values range from **65 to 256**.

2.2.3 Additional Attributes

- Includes variables such as **make, fuel type, engine size**, and many others.
- Full details are available in the course documentation.

2.3 Target Variable

The **26th attribute** is **price**, which serves as the **target variable** or label. All other attributes, such as symboling, normalized losses, and make, act as **predictors**.

2.4 Goal of the Project

The objective is to **predict car prices** using the dataset's attributes. Although the data is from **1985** and prices may seem outdated, the primary aim is to practice and learn **data analysis techniques**.

3 Data Analysis in Python: Key Libraries

3.1 Introduction to Python Data Analysis Libraries

To perform data analysis in Python, it is essential to understand the main packages relevant to analysis. A **Python library** is a collection of functions and methods that allow you to perform tasks without writing code from scratch. These libraries come with built-in modules providing different functionalities that can be used directly.

Python data analysis libraries can be grouped into three major categories:

3.2 Scientific Computing Libraries

3.2.1 Pandas

- Provides data structures and tools for effective data manipulation and analysis.
- Primary structure: **DataFrame**, a two-dimensional table with labeled rows and columns.
- Offers fast access to structured data and easy indexing functionality.

3.2.2 NumPy

- Uses **arrays** for input and output.
- Can be extended to **matrices** with minimal changes to the code.
- Enables **fast array processing** for numerical computations.

3.2.3 SciPy

- Includes functions for advanced **mathematical problems**.
- Supports **data visualization** methods to effectively communicate analytical results.

3.3 Data Visualization Libraries

3.3.1 Matplotlib

- Most well-known library for creating **graphs and plots**.
- Highly **customizable** for various visualization needs.

3.3.2 Seaborn

- Built on top of Matplotlib.
- Simplifies creation of **heat maps**, **time series plots**, **violin plots**, and more.

3.4 Machine Learning and Statistical Modeling Libraries

3.4.1 Scikit-learn

- Provides tools for **statistical modeling** such as regression, classification, and clustering.
- Built on top of **NumPy**, **SciPy**, and **Matplotlib**.

3.4.2 Statsmodels

- Used for **exploring data**, estimating **statistical models**, and performing **statistical tests**.

4 Reading Data Using Pandas

4.1 Introduction

In this section, you will learn how to read data into Python using the **Pandas** package. Once the data is loaded, you can perform various data analysis tasks efficiently.

4.2 Data Acquisition Process

Data acquisition involves loading and reading data into a Python notebook from different sources. Two important factors influence this process:

4.2.1 Data Format

- Format specifies how data is encoded.
- Common formats include **CSV**, **JSON**, **XLSX**, and **HDF**.
- File format is often determined by the file extension.

4.2.2 File Path

- Indicates the location of the data.
- Data can be stored **locally** on your computer or **online**.

4.3 Example: Used Car Dataset

- A dataset of used cars is available online.
- Data points are separated by **commas**, indicating a **CSV (Comma-Separated Values)** format.

4.4 Reading Data into Pandas

You can read data in Pandas in just three steps:

1. Import the **Pandas** library.
2. Define a **file path** variable.
3. Use the **read_csv()** method to load the data into a DataFrame.

4.4.1 Handling Missing Column Headers

- If the data has **no headers**, set `header=None` in the `read_csv()` method.
- Pandas will assign default **integer column names**.

4.5 Exploring the Data

- Use `dataframe.head()` to display the first n rows.
- Use `dataframe.tail()` to display the last n rows.
- These methods help confirm that the data has been loaded correctly.

4.6 Assigning Column Names

- Column names can be loaded from a separate file or list.
- Example:
`headers = ["col1", "col2", "col3"]`
`df.columns = headers`
- After assigning headers, use `head()` to verify the update.

4.7 Exporting Data

- Use `to_csv()` to save a DataFrame to a CSV file.
- Example:
`df.to_csv("automobile.csv")`
- Pandas supports many other file formats with similar syntax.



Data Format	Read	Save
csv	<code>pd.read_csv()</code>	<code>df.to_csv()</code>
json	<code>pd.read_json()</code>	<code>df.to_json()</code>
Excel	<code>pd.read_excel()</code>	<code>df.to_excel()</code>
sql	<code>pd.read_sql()</code>	<code>df.to_sql()</code>

4.8 Summary

- **Pandas** makes it easy to load, explore, and export data.
- Key methods: `read_csv()`, `head()`, `tail()`, `to_csv()`.
- You can work with various formats, but CSV is the most commonly used in this course.

5 Types of Data

5.1 Introduction

Data can be categorized into two main types:

- **Qualitative Data**
- **Quantitative Data**

5.2 Qualitative or Categorical Data

Qualitative or Categorical Data refers to information that cannot be measured or counted in numerical form. Instead, it is classified by categories or characteristics such as gender, color, or opinions.

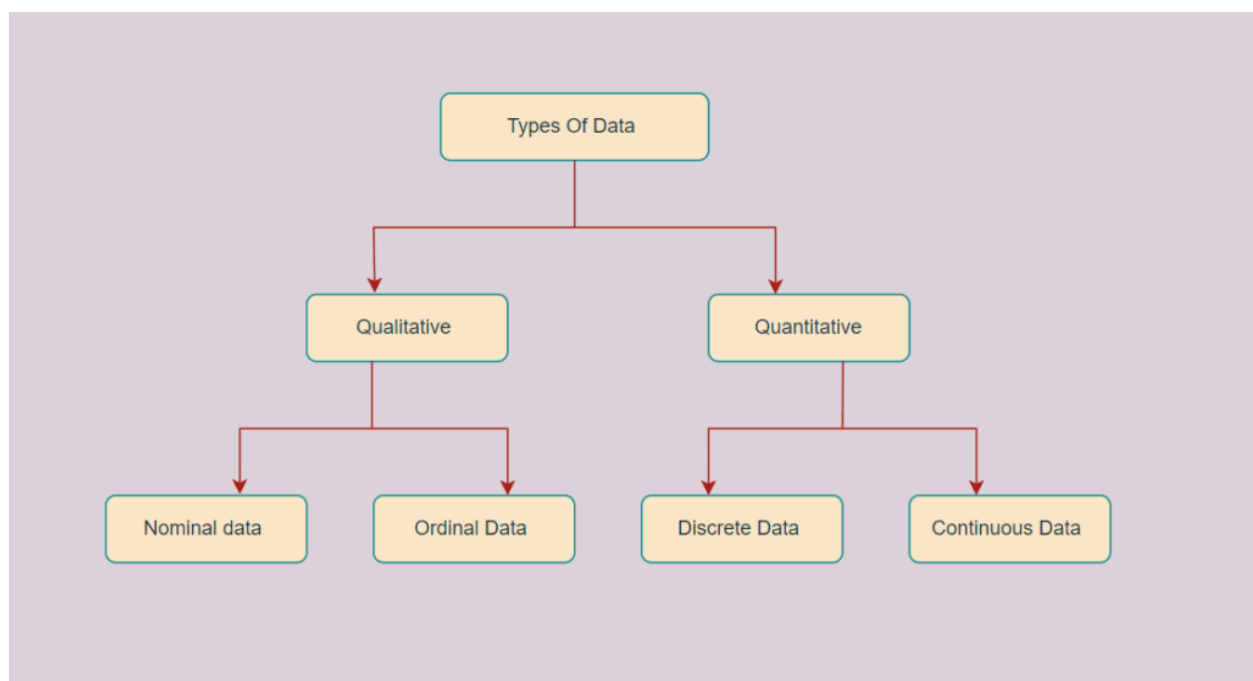
These data types consist of text, symbols, audio, or images and are useful in understanding perceptions or preferences. For example, the gender of a person (male, female, or others) is qualitative data. Market researchers use this type of data to understand customers' tastes and design their strategies accordingly.

5.2.1 Examples of Qualitative Data

- Language spoken
- Favorite holiday destination
- Opinion (agree, disagree, or neutral)
- Colors

5.2.2 Classification of Qualitative Data

Qualitative data is further divided into two types:



5.2.2.1 A. Nominal Data

Nominal data is used to label variables without assigning any order or quantitative value. It is purely categorical and cannot be compared or ranked. The word “nominal” comes from the Latin term *nomen*, meaning “name.”

5.2.2.1.1 Examples of Nominal Data

- Hair color (Blonde, Red, Brown, Black)

- Marital status (Single, Married, Widowed)
- Nationality (Indian, German, American)
- Gender (Male, Female, Others)
- Eye color (Black, Brown)

5.2.2.2 B. Ordinal Data

Ordinal data has a meaningful order or ranking but lacks precise numerical differences between ranks. It is qualitative data with relative positioning.

Ordinal data is often used in surveys or observations like customer satisfaction or educational level. While ordinal data can be ordered, arithmetic operations cannot be performed on it.

5.2.2.2.1 Examples of Ordinal Data

- Feedback or satisfaction scale (1 to 10)
- Letter grades (A, B, C, D)
- Ranking in a competition (First, Second, Third)
- Economic status (High, Medium, Low)
- Education level (Primary, Secondary, Higher)

5.2.3 Difference between Nominal and Ordinal Dat

Nominal Data	Ordinal Data
Cannot be quantified or ordered	Have a meaningful order or sequence
Purely qualitative	Between qualitative and quantitative
No arithmetic operations possible	Numbers can be assigned but no arithmetic operations allowed
Cannot compare categories	Allows comparison by ranking
Examples: Eye color, gender, hair color	Examples: Education level, satisfaction rating

5.3 Quantitative Data

Quantitative data is expressed numerically and can be measured, counted, and statistically analyzed. It answers questions like *how much*, *how many*, and *how often*. Examples include height, weight, time, or income.

Quantitative data is represented using visual formats like bar graphs, histograms, scatter plots, or line charts.

5.3.1 Examples of Quantitative Data

- The height or weight of a person
- Room temperature
- Scores or marks (e.g., 59, 80, 60)
- Time

5.3.2 Classification of Quantitative Data

Quantitative data is further classified into two types:

5.3.2.1 A. Discrete Data/Numerica Data

Discrete data includes countable and distinct values, usually whole numbers or integers. These data cannot take decimal values. For example, the number of students in a class is discrete.

5.3.2.1.1 Examples of Discrete Data

- Number of students in a class
- Cost of a mobile phone
- Number of employees in a company
- Number of players in a competition
- Days in a week

5.3.2.2 B. Continuous Data

Continuous data can take any value within a range, including fractions and decimals. It represents measurements that can be infinitely subdivided, such as temperature or height.

5.3.2.2.1 Examples of Continuous Data

- Height of a person
- Speed of a vehicle
- Time taken to complete a task
- Wi-Fi frequency
- Market share price

5.3.3 Difference between Discrete and Continuous Data

Discrete Data	Continuous Data
Countable and finite; whole numbers	Measurable; includes fractions or decimals
Represented by bar graphs	Represented by histograms
Cannot be subdivided	Can be subdivided into smaller units
Has gaps between values	Continuous and without gaps
Examples: Number of students, days in a week	Examples: Temperature, height, weight

6 Exploring Data Using Pandas

6.1 Introduction

In this section, we introduce essential **Pandas** methods that data scientists and analysts should know when exploring a dataset in Python. We assume the **data has already been loaded**, and now we focus on exploring its **structure and summary**.

6.2 Understanding Data Types

Pandas provides built-in methods to check data types and understand the distribution of data. This helps identify potential issues, such as columns assigned the wrong data type.

6.2.1 Common Data Types in Pandas

- **objects**: Similar to strings in Python.
- **float**: For continuous numeric values.
- **int**: For integer numeric values.
- **datetime**: For time series data.

Pandas automatically assigns data types based on the source data, but these assignments may sometimes be incorrect.

6.2.2 Why Check Data Types?

1. **Detect Incorrect Assignments**: For example, a column like *car price* should be numeric, but it might be assigned as an object type.
2. **Function Compatibility**: Certain Python functions, especially mathematical ones, work only with numeric data.

6.2.3 Viewing Data Types

- Use the `dataframe.dtypes` method to view each column's data type.
- This helps spot mismatches early in the analysis process.

6.3 Statistical Summary of Data

To understand data distribution, Pandas provides the `describe()` method.

6.3.1 Default Behavior

- Returns statistics like **count**, **mean**, **standard deviation (STD)**, **min**, **max**, and **quartiles**.
- By default, only numeric columns are summarized.

6.3.2 Including All Columns

- Use `dataframe.describe(include="all")` to summarize all columns, including objects.
- For object columns, metrics include:
 - **unique**: Number of distinct objects
 - **top**: Most frequent object
 - **freq**: Frequency of the top object

6.3.3 Summary Parameters in the Output

1. **count**
 - Number of **non-missing** (non-NaN) values in that column.

- Example: 201 means there are 201 valid entries.
- 2. **unique** (*for categorical data only*)
 - Number of **unique/distinct values** in the column.
 - Example: 186 means there are 186 unique price values.
- 3. **top** (*for categorical data only*)
 - The **most frequent value** (mode) in that column.
 - Example: 16500 means this value appears the most times.
- 4. **freq** (*for categorical data only*)
 - Frequency (count) of the **most frequent value**.
 - Example: 2 means 16500 appeared twice.
- 5. **mean** (*for numeric data only*)
 - The **average value** of the column.
 - Example: 0.840796 is the mean for symboling.
- 6. **std** (*for numeric data only*)
 - **Standard deviation** → shows how spread out the data is.
 - Example: 1.254802 → larger std = more spread out.
- 7. **min** (*for numeric data only*)
 - The **minimum** value in the column.
 - Example: -2.000000 is the lowest symboling value.
- 8. **25%, 50%, 75%** (*for numeric data only*)
 - These are **percentiles**:
 - 25% → value below which 25% of data falls
 - 50% → median (middle value)
 - 75% → value below which 75% of data falls
- 9. **max** (*for numeric data only*)
 - The **maximum** value in the column.
 - Example: 3.000000 is the highest symboling value.

Example

Consider this dataset of exam scores:

[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

1. **Sort the data:** Already sorted.
2. **Find Q2 (Median)** → Middle value(s):
 - Median = $(50 + 60) / 2 = 55$
3. **Find Q1 (25th percentile)** → Median of the lower half [10, 20, 30, 40, 50]
 - Q1 = 30
4. **Find Q3 (75th percentile)** → Median of the upper half [60, 70, 80, 90, 100]
 - Q3 = 80

Quartile Summary

Quartile	Meaning	Value
Q1	25% of data below this	30
Q2 (Median)	50% of data below this	55
Q3	75% of data below this	80

6.3.4 Handling Missing Values

- Some statistics may return **NaN** (Not a Number) if the metric does not apply to a column's data type.

6.4 Dataset Information

The `dataframe.info()` method provides a concise summary, including:

- Column names
- Data types
- Non-null counts
- Memory usage

6.5 Summary

- Use `dtypes` to check data types.
- Use `describe()` for statistical summaries.
- Use `info()` for a quick overview of the dataset structure.

7 Accessing Databases Using Python

7.1 Introduction

In this section, we learn how to access databases using Python, a critical skill for data scientists. By the end, you will understand how Python communicates with database systems using APIs and how to perform basic operations.

7.2 Connecting to Databases

Python programs communicate with Database Management Systems (DBMS) using **API calls**. A common environment for writing this code is **Jupyter Notebook**, a web-based editor.

7.2.1 SQL APIs

- **Definition:** An Application Programming Interface (API) is a set of functions used to access services.
- **SQL API:** Provides functions to send SQL statements to a DBMS and retrieve results.
- **Typical Flow:**
 1. Connect to the DBMS using API calls.
 2. Build SQL statements as text strings.
 3. Pass SQL commands to the DBMS via API functions.
 4. Retrieve query results and status information.
 5. Handle errors if any occur.

7.2.2 Ending Database Connections

- After completing operations, an API call disconnects the application from the database, ensuring proper resource management.

7.3 Python DB API

Python provides a standard API, known as **DB API**, for accessing relational databases.

7.3.1 Key Advantages

- Write one program that works with multiple relational databases.
- Learn DB API functions once, then apply to any database system.

7.3.2 Core Concepts

- **Connection Objects:** Used to connect to the database and manage transactions.
- **Cursor Objects:** Used to execute queries and retrieve results.

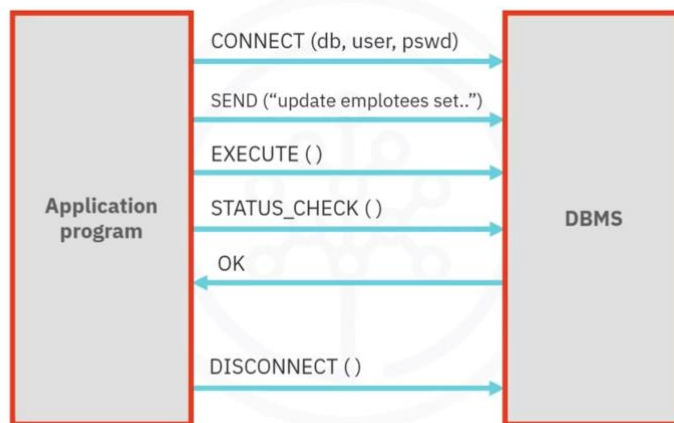
7.4 Working with Cursors

A cursor works like a text cursor, scanning through the query results.

7.4.1 Cursor Methods

- **cursor()**: Creates a new cursor object from a connection.
- **execute()**: Runs SQL queries.
- **fetch()**: Retrieves query results.
- **commit()**: Saves any pending transaction.
- **rollback()**: Reverts the database to the state before the transaction.
- **close()**: Closes the cursor or connection.

What is a SQL API?



7.5 Sample Workflow in Python

1. **Import Database Module:** Import the appropriate database library.
2. **Establish Connection:** Use the `connect()` function with parameters like database name, username, and password.
3. **Create Cursor:** Generate a cursor object from the connection.
4. **Execute Queries:** Run SQL statements using the cursor.
5. **Fetch Results:** Retrieve and process query results.
6. **Close Connection:** Release resources by closing the connection after completion.

Writing code using DB-API

```
from dmodule import connect

#Create connection object
connection = connect( 'databasename' , 'username' , 'pswd' )

#Create a cursor object
cursor = connection.cursor ( )

#Run queries
cursor.execute( 'select * from mytable' )
results = cursor.fetchall ( )

#Free resources
cursor.close ( )
connection.close ( )
```

7.6 Summary

- Python connects to databases using API calls, primarily through the **DB API**.
- **Connection objects** handle transactions; **cursor objects** manage queries.
- Properly closing connections prevents resource wastage.

8 Lesson Summary

8.1 Key Concepts Learned

At this point in the course, you should understand the following concepts:

8.1.1 Dataset Structure

- Each line in a dataset is a **row**, and commas separate the **values**.
- Analyze the **attributes** for each column to understand the data better.

8.1.2 Python Libraries

- Libraries are **collections of functions and methods** that simplify programming tasks.
- Categories include:
 - Scientific Computing
 - Data Visualization
 - Machine Learning Algorithms
- Many libraries are interconnected; for example, **Scikit-learn** is built on top of **NumPy**, **SciPy**, and **Matplotlib**.

8.1.3 Data Handling with Pandas

- **Data format** and **file path** are key factors for reading data.

- The **read_csv()** method loads CSV files into a Pandas DataFrame.
- Pandas has unique data types: **object**, **float**, **int**, and **datetime**.
- Use **dtype** to check each column's data type; misclassified types may need correction.
- Correct data types allow appropriate Python functions to be applied to columns.

8.1.4 Statistical Analysis

- The **describe()** method provides count, mean, standard deviation, min, max, and quartile ranges for numerical columns.
- Use **include='all'** to get summaries for object-type columns.
- Statistical summaries help detect potential issues such as **outliers**¹.
- The **info()** method gives an overview of the DataFrame, including top and bottom rows.
- **NaN** indicates missing values, preventing some statistics from being calculated.

8.1.5 Database Connectivity with Python

- Python connects to databases via **SQL APIs** or **DB APIs**.
- **SQL APIs**: Build SQL queries as text strings, send them to the DBMS, and retrieve results.
- **DB API**: Python's standard for interacting with relational databases using:
 - **Connection objects**: Establish/manage connections and transactions.
 - **Cursor objects**: Run queries and navigate results.
- Connection methods include **cursor()**, **commit()**, **rollback()**, and **close()**.
- Always close the database connection after use to free resources.

9 Overview: Laptop Pricing Data Set

The dataset used for this practice exercise is a **modified subset** that maps the **price of laptops** with different attributes.

The dataset is a filtered and modified version of the **Laptop Price Prediction using Specifications** dataset on the Kaggle website.

9.1 Parameters

9.1.1 Manufacturer

The company that manufactured the laptop.

9.1.2 Category

The category to which the laptop belongs. This parameter is mapped to numerical values as follows:

Category	Assigned Value
Gaming	1
Netbook	2
Notebook	3

¹ In **Statistical Analysis**, **outliers** are data points that differ significantly from the rest of the dataset. They are unusually high or low values compared to the overall pattern of the data.

Ultrabook	4
Workstation	5

9.1.3 GPU

The manufacturer of the GPU. This parameter is mapped to numerical values as follows:

GPU	Assigned Value
AMD	1
Intel	2
NVidia	3

9.1.4 OS

The operating system type (Windows or Linux). This parameter is mapped to numerical values as follows:

OS	Assigned Value
Windows	1
Linux	2

9.1.5 CPU_core

The type of processor used in the laptop. This parameter is mapped to numerical values as follows:

CPU core	Assigned Value
Intel Pentium i3	3
Intel Pentium i5	5
Intel Pentium i7	7

9.1.6 Screen_Size_cm

The size of the laptop screen, measured in **centimeters (cm)**.

9.1.7 CPU_frequency

The frequency at which the CPU operates, measured in **gigahertz (GHz)**.

9.1.8 RAM_GB

The size of the laptop's **RAM**, measured in **gigabytes (GB)**.

9.1.9 Storage_GB_SSD

The size of the **SSD storage**, measured in **gigabytes (GB)**.

9.1.10 Weight_kg

The weight of the laptop, measured in **kilograms (kg)**.

9.1.11 Price

The **price of the laptop**, measured in **US Dollars (USD)**.

Module 2

10 Data Pre-Processing Techniques

10.1 Introduction to Data Pre-Processing

Data pre-processing is an essential step in data analysis. It involves converting or mapping raw data into a format suitable for further analysis. This step is also known as **data cleaning** or **data wrangling**.

10.2 Topics Covered in This Module

10.2.1 Handling Missing Values

- Missing values occur when data entries are left empty.
- We'll explore techniques to identify and handle missing data effectively.

10.2.2 Data Formatting

- Data from different sources may come in various formats, units, or conventions.
- We'll introduce Python Pandas methods to standardize these values into consistent formats for better analysis.

10.2.3 Data Normalization

- Numerical data columns may have very different ranges, making direct comparisons challenging.
- **Normalization** brings all data into a similar range for meaningful comparisons.
- Techniques covered include **centering** and **scaling**.

10.2.4 Data Binning

- Binning converts a set of numerical values into larger categories.
- It is particularly useful for comparisons between groups of data.

10.2.5 Categorical Variables

- Categorical values often need conversion into numeric variables for statistical modeling.
- We'll show how to perform this conversion in Python.

10.3 Working with Data in Python

- Operations are often performed along columns, with each row representing a sample.
- Access columns using their names, e.g., `symboling` or `body style`.

10.3.1 Manipulating Data Frames

- Each column in a DataFrame is a **Pandas Series**.
- Example: Adding a value to each entry in a column.
 - To add 1 to each symboling entry:
 - `df["symboling"] = df["symboling"] + 1`
- This command updates each value in the column by adding one to the current value.

11 Handling Missing Values in Data

11.1 Introduction to Missing Values

Missing values occur when no data value is stored for a feature in a particular observation. These can appear as question marks, N/A, zero, blank cells, or NaN. Missing data is a common issue across all tools and platforms, including Python and R.

11.2 Strategies to Handle Missing Values

11.2.1 Checking Data Sources

- Confirm whether the missing values can be retrieved by contacting the data collector.

11.2.2 Dropping Missing Data

- **Drop Rows:** Remove rows containing missing values if they are few and non-critical.
- **Drop Columns:** Remove entire variables if too many values are missing.
- Use Pandas `dropna()` method:
- `df.dropna(subset=["price"], axis=0, inplace=True)`

11.2.3 Replacing Missing Data

- **Numerical Data:** Replace missing values with the mean of the column.
- `mean_value = df["normalized-losses"].mean()`
- `df["normalized-losses"].replace(np.nan, mean_value, inplace=True)`
- **Categorical Data:** Use the most frequent value (mode) for replacement.

11.2.4 Advanced Replacement Techniques

- Replace values using group means or additional known information.
- Keep data missing if it holds analytical value.

11.3 Methods in Python for Missing Values

11.3.1 Dropping Data

- Use `dropna()` with `axis=0` to drop rows or `axis=1` to drop columns.
- Set `inplace=True` to update the original DataFrame.

11.3.2 Replacing Data

- Use `replace()` to fill missing values with calculated or known values.

- Commonly replaced with mean, median, or mode depending on data type.

11.4 Summary

- Identify missing values and decide whether to drop or replace them.
- Use `dropna()` for removal and `replace()` for substitution in Python.
- Consider data quality, analysis goals, and data context before choosing the method.

12 Handling Data with Different Formats and Units

12.1 Introduction to Data Formatting

Data collected from multiple sources often comes in different formats, units, and conventions. Data formatting involves bringing this data into a common standard to make meaningful comparisons and ensure consistency for analysis.

12.1.1 Importance of Data Formatting

- Ensures consistency and clarity in datasets.
- Helps in accurate statistical analysis.
- Reduces errors caused by varied data representations.

For example, the city name "New York" might appear as N.Y., Ny, NY, or New York. Depending on the analysis goal, this variation might be useful (e.g., for detecting fraud) or might need to be standardized.

12.2 Converting Units in Data

In some cases, datasets use units that may not be convenient for all users. For example:

- The feature **city-miles per gallon** represents fuel consumption in miles per gallon.
- Some users may want to convert this to **liters per 100 kilometers**, the metric unit.

12.2.1 Conversion Formula

To convert miles per gallon to liters per 100 kilometers:

$$\text{liters_per_100km} = 235 / \text{miles_per_gallon}$$

This can be done in Python with one line of code.

12.2.2 Renaming Columns

After conversion, rename the column using the DataFrame `rename` method for clarity.

12.3 Handling Data Types

Sometimes datasets import with incorrect data types:

- Example: The **price** column may import as an object instead of int or float.

12.3.1 Identifying Data Types

- Use `dataframe.dtypes` to check each feature's data type.

12.3.2 Converting Data Types

- Use `dataframe.astype()` to convert columns:

```
df["price"] = df["price"].astype("int")
```

12.3.3 Common Data Types in Pandas

- **Object:** Letters or words.
- **Int64:** Integer numbers.
- **Float:** Real numbers with decimals.

12.4 Summary

- Standardize data formats for consistency.
- Convert units when necessary for analysis convenience.
- Check and correct data types to ensure accurate results in further analysis.

13 Data Normalization

13.1 Introduction to Data Normalization

Data normalization is an important technique in data preprocessing. It ensures that different features have consistent ranges, allowing fair comparisons and improving statistical analysis accuracy.

13.1.1 Importance of Normalization

- Makes variable ranges consistent.
- Enables fair comparisons between features.
- Prevents certain attributes from dominating models due to larger value ranges.
- Improves computational efficiency.

13.2 Examples of Normalization

Consider the following examples:

- **Used Car Dataset:** The feature *length* ranges from 150 to 250, while the *width* and *height* range from 50 to 100. Without normalization, features with larger ranges could dominate analyses.
- **Age and Income Dataset:** *Age* ranges from 0 to 100, while *income* ranges from 20,000 to 500,000. Income would heavily influence models like linear regression due to its larger numerical values, even if it is not more important.

13.3 Methods of Normalization

There are three commonly used normalization methods:

13.3.1 Simple Feature Scaling

- Formula:

$$x_{\text{new}} = x_{\text{old}} / \max(x)$$

- Divides each feature value by the maximum value, resulting in values between 0 and 1.

13.3.2 Min-Max Method

- Formula:

$$x_{\text{new}} = (x_{\text{old}} - \min(x)) / (\max(x) - \min(x))$$

- Subtracts the minimum value and divides by the range, scaling values between 0 and 1.

13.3.3 Z-Score Method (Standard Score)

- Formula:

$$z = (x_{\text{old}} - \mu) / \sigma$$

- Subtracts the mean (μ) and divides by the standard deviation (σ).
- Produces values typically between -3 and 3.

13.4 Applying Normalization in Python

Using the **length** feature as an example:

- **Simple Feature Scaling:** Divide each value by the maximum using `df['length'].max()`.
- **Min-Max Method:** Subtract the minimum and divide by the range (`max - min`).
- **Z-Score Method:** Subtract the mean and divide by the standard deviation using `mean()` and `std()` methods.

13.5 Summary

- Normalization ensures fair comparisons between features.
- It prevents numerical dominance of certain variables.
- Three main methods: Simple Feature Scaling, Min-Max, and Z-Score.
- Normalization is easily implemented in Python using Pandas methods.

14 Binning in Data Preprocessing

14.1 Introduction to Binning

Binning is a data preprocessing technique where numerical values are grouped into discrete intervals or "bins." This process helps simplify data, improve model accuracy, and provide a clearer understanding of data distribution.

14.1.1 Importance of Binning

- Group numerical values into fewer categories.
- Simplifies data interpretation.
- Can improve the accuracy of predictive models.
- Useful for visualizing data distribution.

14.2 Example: Binning Car Price Data

Consider a dataset with the attribute *price* ranging from 5,188 to 45,400, containing 201 unique values. We can categorize the prices into three bins:

- **Low Price**
- **Medium Price**
- **High Price**

14.2.1 Creating Equal-Width Bins

To create three bins with equal width, we need four numbers equally spaced across the price range. Using Python:

- **NumPy's linspace** function generates equally spaced dividers.
- **Group Names:** A list like ["Low", "Medium", "High"] labels each bin.

14.2.2 Applying Binning in Python

- Use **Pandas' cut** function to segment data values into bins.
- Visualize the data using **histograms** to show how many values fall into each bin.

price		price	price-binned
13495		13495	Low
16500		16500	Low
18920		18920	Medium
41315		41315	High
5151		5151	Low
6295		6295	Low
...		...	...

```
bins = np.linspace(min(df["price"]), max(df["price"]), 4)
```

```
group_names = ["Low", "Medium", "High"]
```

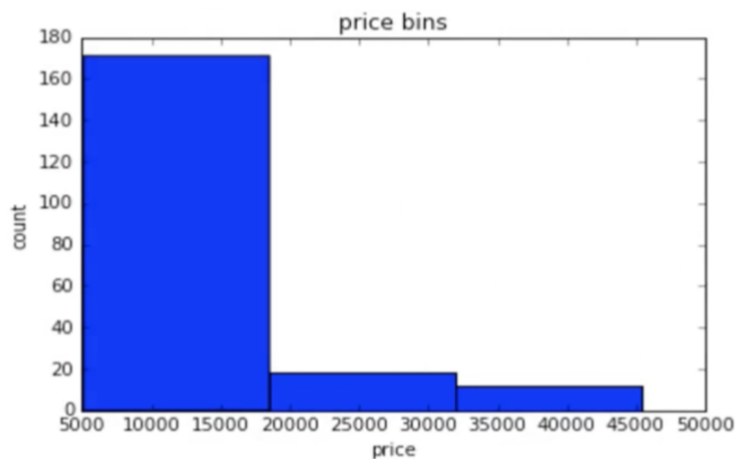
```
df["price-binned"] = pd.cut(df["price"], bins, labels=group_names, include_lowest=True )
```

14.2.3 Insights from the Histogram

From the resulting plot:

- Most cars fall into the *low price* category.
- Only a few cars have *high prices*.

• Histograms



14.3 Summary

- Binning converts numerical data into categorical intervals.
- It simplifies analysis and visualization.
- In Python, **NumPy** and **Pandas** provide straightforward methods for implementing binning.
- Histograms help interpret the distribution after binning.

15 Converting Categorical Variables to Quantitative Variables

15.1 Introduction

Most statistical models cannot process objects or strings as inputs and require numerical values for model training. In Python, categorical variables must be converted into quantitative (numeric) variables before further analysis.

15.2 Example: Fuel Type in Car Dataset

Consider the *fuel type* feature in a car dataset, which has two string values: *gas* and *diesel*. For analysis, these categorical values need to be encoded into a numeric format.

15.2.1 One-Hot Encoding Technique

- **Process:**
 - Create new features corresponding to each unique value in the original categorical feature.
 - For each observation, assign 1 to the feature matching the category and 0 to all others.
- **Example:**
 - For *Car B* with fuel type *diesel*:
 - diesel = 1, gas = 0
 - For *Car D* with fuel type *gas*:
 - gas = 1, diesel = 0

This approach is called **one-hot encoding**.

Categorical → Numeric

Solution:

- Add dummy variables for each unique category
- Assign 0 or 1 in each category

Car	Fuel	...	gas	diesel
A	gas	...	1	0
B	diesel	...	0	1
C	gas	...	1	0
D	gas	...	1	0

“One-hot encoding”

15.3 Implementing One-Hot Encoding in Python

- **Using Pandas:**

- The `get_dummies` method in pandas can convert categorical variables into dummy variables automatically.
- Example:
- `dummy_variable_one = pd.get_dummies(df['fuel'])`
- The method generates a new DataFrame with binary indicators for each category.

15.4 Summary

- Categorical variables must be encoded for statistical models.
- **One-hot encoding** creates binary variables for each category.
- **Pandas' `get_dummies`** simplifies this process efficiently.

16 Lesson Summary

16.1 Key Takeaways

At this point in the course, you know:

16.1.1 Data Formatting

- Data formatting is critical for making data from various sources consistent and comparable.
- You can master techniques in Python to convert units of measurement, such as transforming *city miles per gallon* to *city-liters per 100 kilometers* for easier comparison and analysis.

16.1.2 Data Types and Normalization

- Acquire skills to identify and correct data types in Python to ensure accurate representation for statistical analyses.
- Data normalization makes variables comparable and removes inherent biases in statistical models.
- Techniques include **Feature Scaling**, **Min-Max**, and **Z-Score**, all of which can be applied using pandas methods in Python.

16.1.3 Binning and Visualization

- Binning improves model accuracy and data visualization by categorizing numerical variables into intervals.
- You can run binning techniques in Python using **numpy's `linspace`** and **pandas' `cut`** methods.
- Use **histograms** to visualize the distribution of binned data and gain insights into feature distributions.

16.1.4 Encoding Categorical Variables

- Statistical models require numerical inputs, so categorical variables like *fuel type* must be converted into numeric formats.
- Implement **one-hot encoding** in Python using pandas' `get_dummies` method to transform categorical variables for machine learning models.

Module 3

17 Outlier Detection

17.1 Definition

Outlier detection is the process of finding and possibly removing data points that are very different from the majority of the data.

17.2 Purpose

Outliers can distort statistical results and reduce model performance. Common examples include:

- Fraud detection
- Anomaly detection
- Data cleaning

17.3 Methods

- Z-Score Method
- IQR Method
- Isolation Forest
- DBSCAN
- Mahalanobis Distance

17.4 Interquartile Range (IQR)

The Interquartile Range (IQR) measures the spread of the central portion of the data:

- **Q1 (25th percentile):** 25% of the data lies below this value.
- **Q3 (75th percentile):** 75% of the data lies below this value.
- **IQR:** $Q3 - Q1$ → The range between Q1 and Q3.

17.5 Finding Outliers Using IQR

We assume points far away from the middle 50% are potential outliers.

- **Lower Bound:** $Q1 - 1.5 \times IQR$
- **Upper Bound:** $Q3 + 1.5 \times IQR$

Any data point:

- Less than the Lower Bound → Potential outlier (too small)
- Greater than the Upper Bound → Potential outlier (too large)

17.6 Example

Given the dataset:

Data: 5, 6, 7, 8, 10, 12, 15, 20, 50

- $Q1 = 7$
- $Q3 = 15$
- $IQR = 15 - 7 = 8$

Bounds:

- Lower Bound = $7 - 1.5(8) = -5$
- Upper Bound = $15 + 1.5(8) = 27$

Conclusion:

Values outside the range -5 to 27 are outliers. Here, **50** is an outlier.

18 Introduction to Exploratory Data Analysis (EDA)

In this module, we are going to cover the **basics of exploratory data analysis (EDA)** using Python.

Exploratory Data Analysis (EDA) is an approach to **analyze data** in order to:

- Summarize the main characteristics of the dataset.
- Gain a better understanding of the data.
- Uncover relationships between different variables.
- Identify important variables that influence the problem we are trying to solve.

The **main question** we aim to answer in this module is:

What are the characteristics that have the most impact on the car price?

To address this question, we will explore several **useful EDA techniques**.

18.1 Key Learning Areas

In this module, we will learn about the following key areas:

18.1.1 Descriptive Statistics

Descriptive statistics describe the **basic features of a dataset** and provide a **summary** of the sample and its measures. They help in understanding the central tendency, dispersion, and distribution of the data.

18.1.2 Grouping Data using GroupBy

We will learn how to **group data using the GroupBy function** and how it helps to **transform and summarize datasets** effectively.

18.1.3 ANOVA (Analysis of Variance)

ANOVA is a **statistical method** used to analyze the **differences among group means** in a dataset by dividing the variation into distinct components.

18.1.4 Correlation Analysis

We will examine the **correlation between various variables** to understand how they interact with one another.

18.1.5 Advanced Correlation Techniques

In this section, we will introduce **advanced correlation methods**, including:

- **Pearson Correlation** — measures linear relationships between variables.
- **Correlation Heatmaps** — visualize the strength and direction of relationships across multiple variables.

19 Descriptive Statistics

When beginning data analysis, it is essential to first **explore the dataset** before building complex models. One effective approach to examine and summarize data is through **descriptive statistics**.

Descriptive statistical analysis helps to describe the **basic features of a dataset** and provides a concise **summary of the sample and its measures**.

19.1 Using the describe() Function in Pandas

One easy way to generate descriptive statistics is by using the `describe()` function in the **Pandas** library.

19.1.1 Purpose

The `describe()` function automatically computes basic statistics for all numerical variables, including:

- Mean
- Total number of data points
- Standard deviation
- Quartiles (25%, 50%, 75%)
- Minimum and maximum values

Note: Any NaN (missing) values are automatically ignored in these calculations.

This function provides a clear overview of the **distribution and variability** of your dataset's numerical variables.

19.2 Working with Categorical Variables

Datasets often include **categorical variables**, which represent discrete groups or categories. For example, the **drive system** variable in a car dataset can include the following categories:

- Front wheel-drive
- Rear wheel-drive
- Four wheel-drive

19.2.1 Summarizing Categorical Data

A simple way to summarize categorical data is by using the `value_counts()` function.

Example:

- 118 cars belong to the *front-wheel-drive* category.
- 75 cars belong to the *rear-wheel-drive* category.
- 8 cars belong to the *four-wheel-drive* category.

You can also rename the column for better readability when presenting the results.

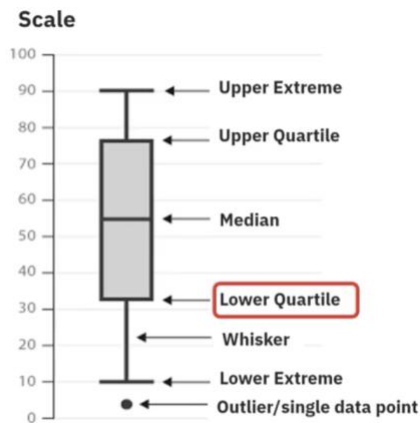
19.3 Visualizing Data with Box Plots

Box plots are an excellent way to visualize the distribution of numerical data.

19.3.1 Key Features of a Box Plot

- **Median:** The midpoint of the data.
- **Upper Quartile (Q3):** Represents the 75th percentile.
- **Lower Quartile (Q1):** Represents the 25th percentile.
- **Interquartile Range (IQR):** The range between Q1 and Q3.
- **Extremes:** Calculated as $1.5 \times \text{IQR}$ above Q3 and below Q1.
- **Outliers:** Data points outside the upper and lower extremes, shown as individual dots.

Box plots help identify **outliers**, **distribution shape**, and **skewness** in the data. They are particularly useful for comparing distributions across different groups.

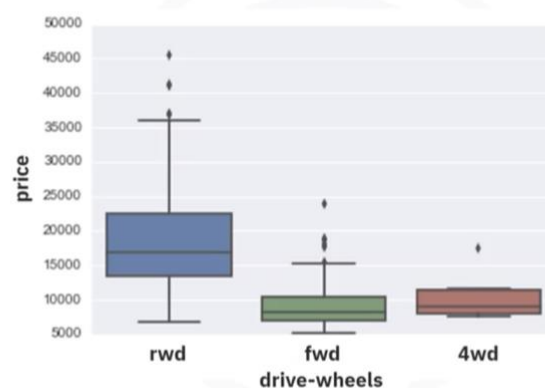


19.3.2 Example

Using box plots, we can visualize the **distribution of car prices** across different **drive-wheel categories**. The box plot reveals that:

- Prices for rear-wheel-drive cars differ significantly from those of other categories.
- Prices for front-wheel-drive and four-wheel-drive cars are almost indistinguishable.

```
sns.boxplot(x= "drive-wheels", y= "price", data=df)
```



19.4 Analyzing Continuous Variables with Scatter Plots

Many datasets contain **continuous variables**—numerical values that exist within a specific range. For instance, in a car dataset:

- **Price** and **engine size** are continuous variables.

19.4.1 Exploring Relationships with Scatter Plots

A **scatter plot** is useful for visualizing the relationship between two continuous variables.

Each observation in the dataset is represented as a single point on the plot.

- **Predictor Variable (X-axis):** The variable used to predict an outcome (e.g., engine size).
- **Target Variable (Y-axis):** The variable being predicted (e.g., price).

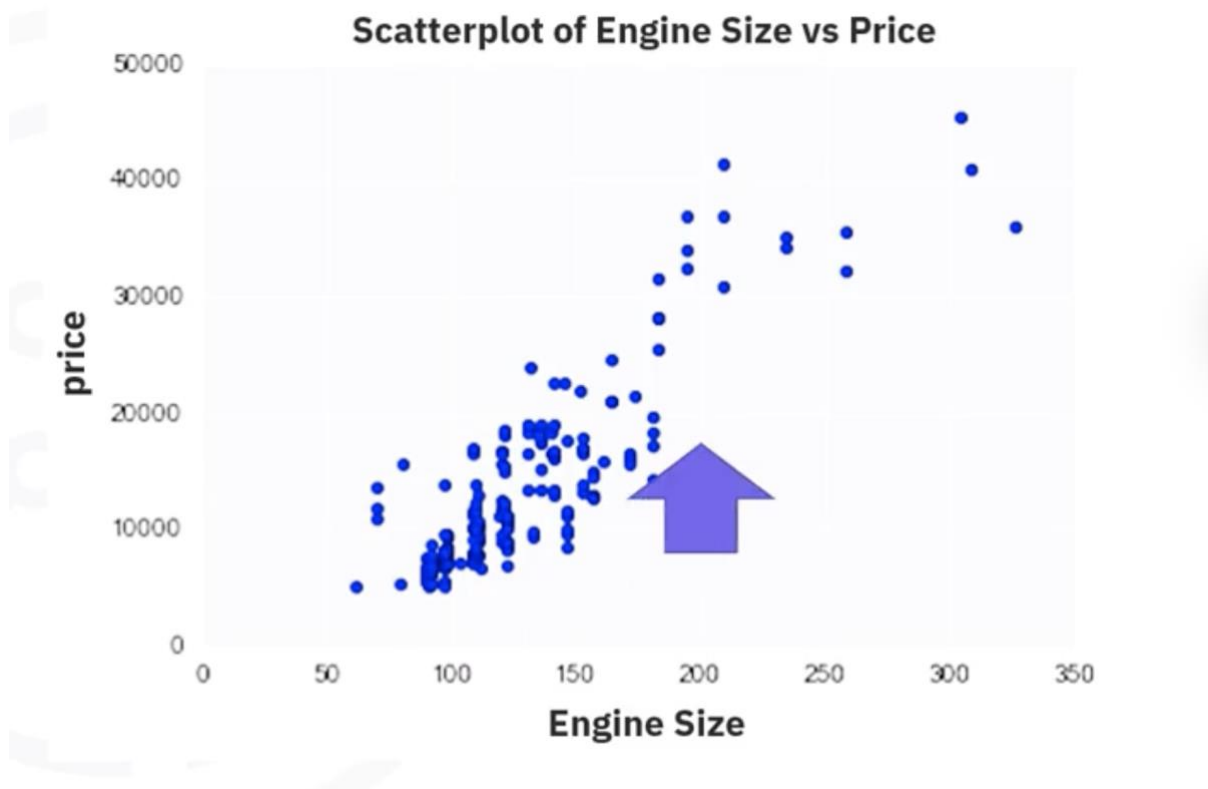
In this case, we plot **engine size** on the x-axis and **price** on the y-axis using the Matplotlib function `scatter()`.

19.4.2 Best Practices for Scatter Plots

- Always **label axes** clearly.
- Add a **descriptive title** to explain what the plot represents.

19.4.3 Insights from the Scatter Plot

From the scatter plot, we can observe that as **engine size increases**, the **price of the car** also tends to increase. This suggests a **positive linear relationship** between the two variables.



20 Grouping and Transforming Data in Pandas

Grouping data allows us to analyze relationships between different categories and numerical values within a dataset. In this section, we explore how to group data in **Pandas** and transform it into more meaningful visual formats.

20.1 Understanding the GroupBy Method

The **groupby()** method in Pandas is a powerful function used to group data based on **categorical variables**. It divides the dataset into subsets according to the categories of a selected variable or multiple variables.

20.1.1 Purpose of Grouping

Grouping helps to:

- Identify relationships between categories.
- Compare statistical summaries (e.g., mean, median) across categories.
- Simplify complex datasets for better analysis.

For example, suppose we want to explore whether there is a relationship between **drive system type** (front-wheel, rear-wheel, four-wheel drive) and **vehicle price**. We can use the **groupby()** method to compare average prices across these categories.

20.2 Grouping by Multiple Variables

We can group data by one or more categorical variables. For instance, if we are interested in the **average price of vehicles** by both **drive wheels** and **body style**, we can perform the following steps:

1. **Select Relevant Columns:** Extract only the necessary data columns such as drive-wheels, body-style, and price.
2. **Apply Grouping:** Use the **groupby()** function to group the data by drive-wheels and body-style.
3. **Compute Mean Values:** Use the **mean()** function to find the average price within each group.

The resulting grouped data shows how **average prices** vary across combinations of drive wheels and body styles. For example, according to the data:

- **Rear-wheel drive convertibles** and **rear-wheel drive hardtops** have the **highest average prices**.
- **Four-wheel drive hatchbacks** have the **lowest average prices**.

20.3 Using Pivot Tables for Easier Visualization

A **pivot table** provides a clearer, more structured way to view grouped data. In Pandas, you can create a pivot table using the **pivot()** method.

20.3.1 Concept of Pivot Tables

A pivot table reorganizes data so that:

- One variable is displayed along the **rows**.
- Another variable is displayed along the **columns**.

This layout creates a rectangular grid of data that is easier to interpret—similar to how data is represented in **Excel spreadsheets**.

20.3.2 Example in Pandas

Using one line of code with the `pivot()` method, we can pivot the body-style variable to appear as columns and the drive-wheels variable as rows. The resulting grid shows **average prices** for each combination of these variables.

20.4 Visualizing Pivot Tables with Heat Maps

A **heat map** is an excellent tool for visualizing pivot table data. It assigns color intensities based on numerical values, providing visual cues about data patterns.

20.4.1 Creating a Heat Map

In this example, we use the `pcolor()` function from **Matplotlib's Pyplot** module to create a heat map of average vehicle prices.

- The **x-axis** represents body styles.
- The **y-axis** represents drive wheels.
- **Color intensity** represents the average price, according to the color bar.

20.4.2 Insights from the Heat Map

The heat map provides a quick and intuitive view of how vehicle prices vary across drive-wheel and body-style combinations:

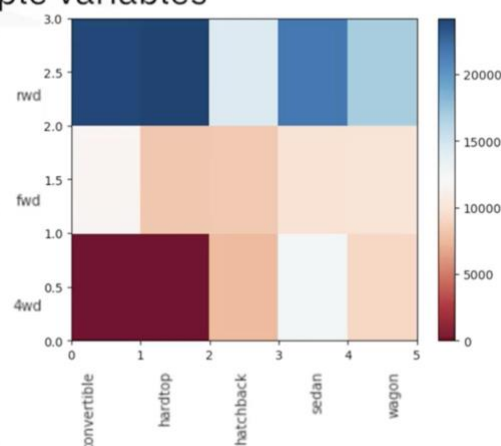
- The **upper section** (e.g., rear-wheel drive convertibles) shows **higher prices**.
- The **lower section** (e.g., four-wheel drive hatchbacks) shows **lower prices**.

This visual representation helps to quickly identify which categories add the most value to vehicles.

Heatmap

- Plot target variable against multiple variables

```
plt.pcolor(df_pivot, cmap='RdBu')
plt.colorbar()
plt.show()
```



21 Data Visualization Commands in Python

Visualizations play a key role in data analysis. In this section, we explore various types of graphs and plots that can be created in Python to better understand and analyze data. The two major libraries used for data visualization are **Matplotlib** and **Seaborn**.

21.1 Importing Libraries

To begin visualizing data, we first need to import the necessary libraries.

21.1.1 Importing Matplotlib

```
from matplotlib import pyplot as plt
```

Alternatively:

```
import matplotlib.pyplot as plt
```

Most plotting functions of interest are found in the **pyplot** submodule. When working in Jupyter Notebooks, it's recommended to use the following magic command to ensure plots display inline:

```
%matplotlib inline
```

21.1.2 Importing Seaborn

Seaborn is typically imported using the following command:

```
import seaborn as sns
```

21.2 Matplotlib Functions

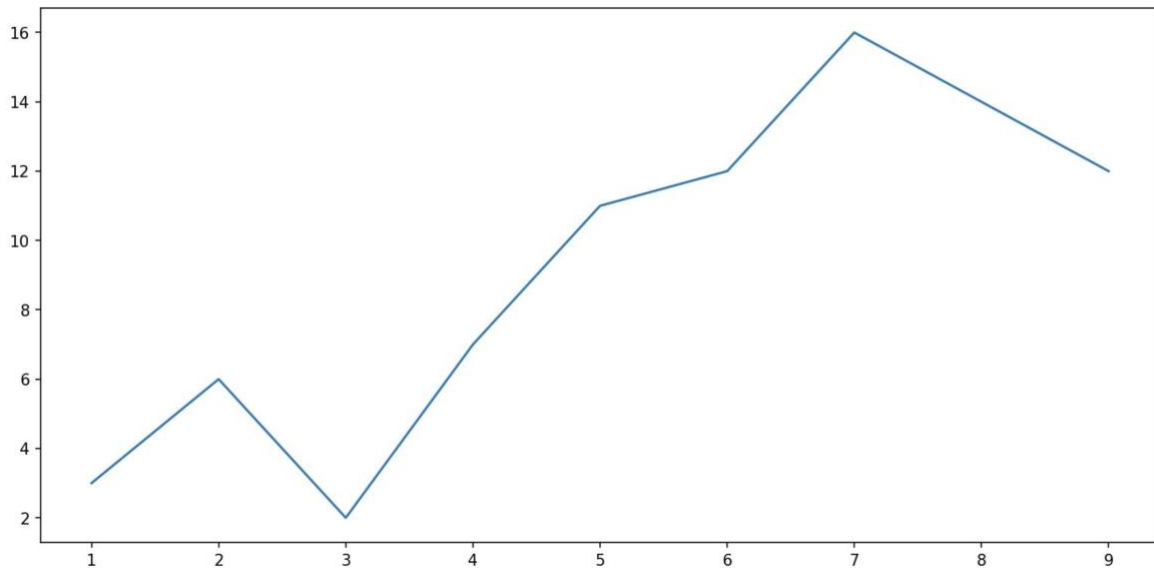
21.2.1 Standard Line Plot

A **line plot** is the simplest and most fundamental type of visualization. It represents data points connected by straight lines.

Syntax:

```
plt.plot(x, y)
```

Here, *x* and *y* are arrays of equal length representing independent and dependent variables, respectively.



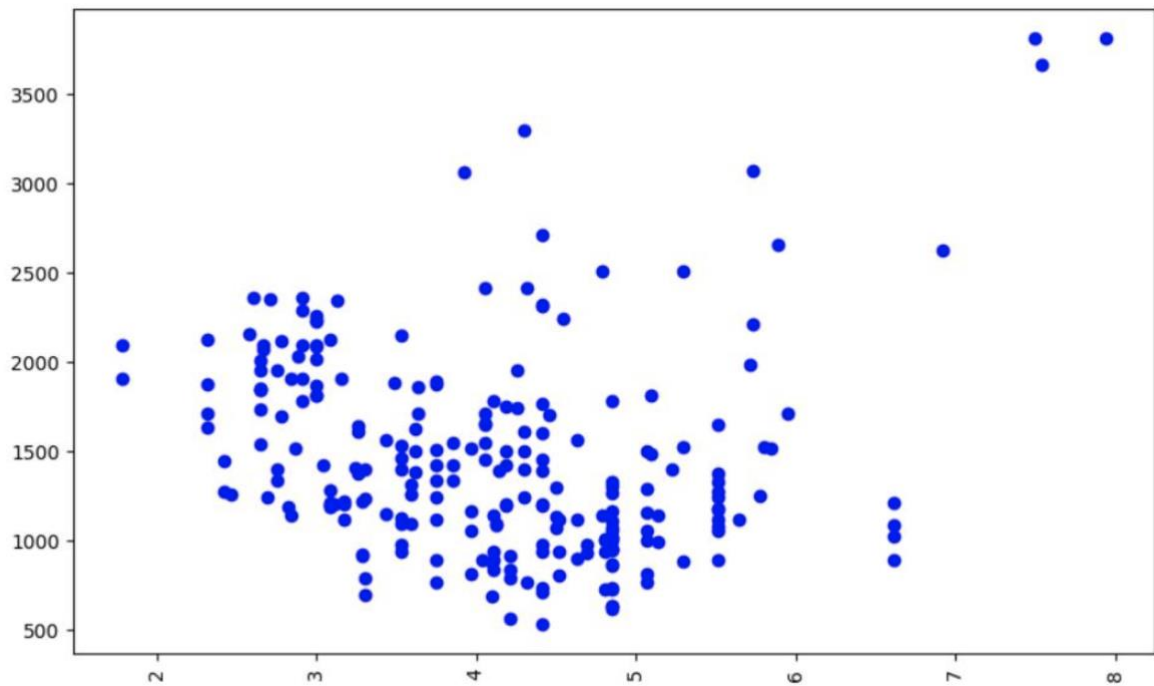
21.2.2 Scatter Plot

A **scatter plot** displays the relationship between two numerical variables using points on a two-dimensional plane.

Syntax:

```
plt.scatter(x, y)
```

You can modify marker size, color, and shape using additional arguments.



21.2.3 Histogram

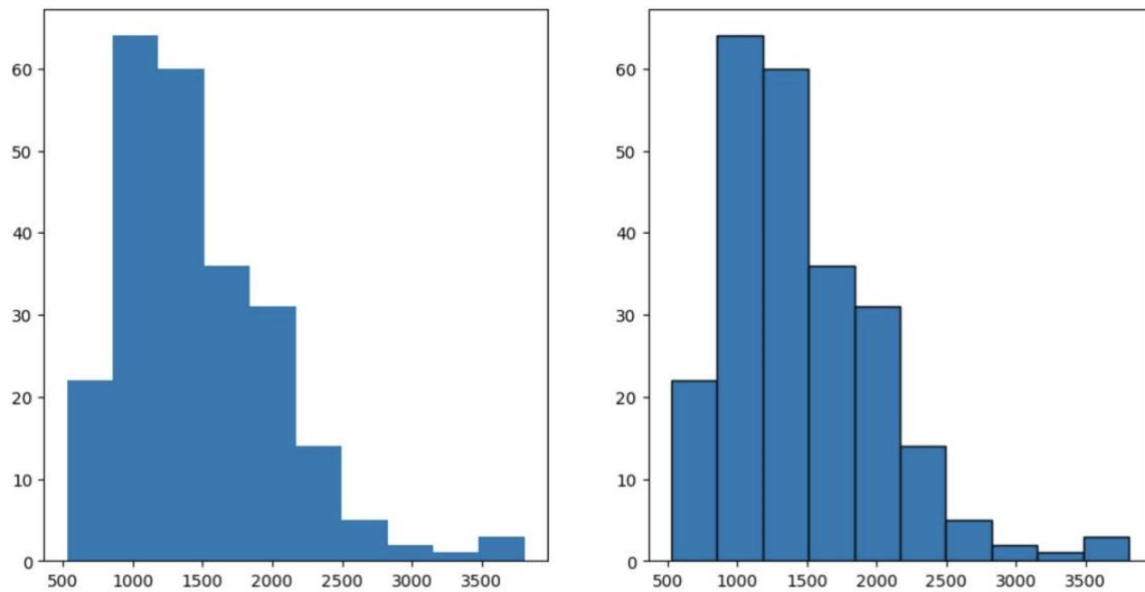
A **histogram** represents data distribution using bins. The x-axis shows data intervals (bins), while the y-axis shows frequency counts.

Syntax:

```
plt.hist(x, bins)
```

You can enhance clarity by adding an edge color:

```
plt.hist(x, bins, edgecolor='black')
```



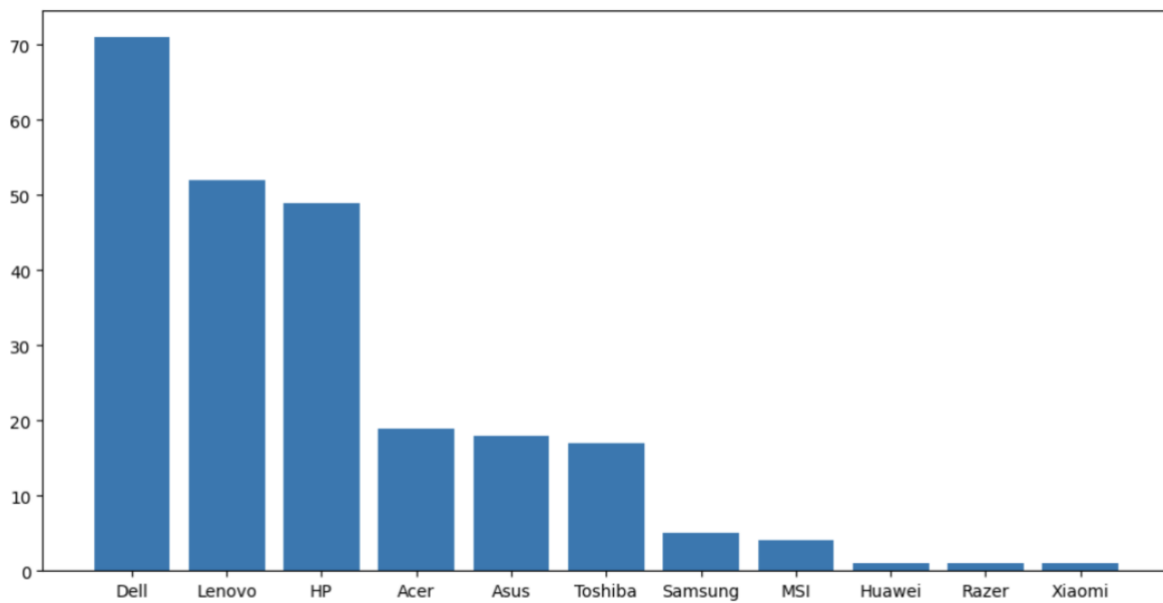
21.2.4 Bar Plot

A **bar plot** visualizes categorical data, where each bar represents a category's value.

Syntax:

```
plt.bar(x, height)
```

Here, `x` represents categories and `height` represents their corresponding values.



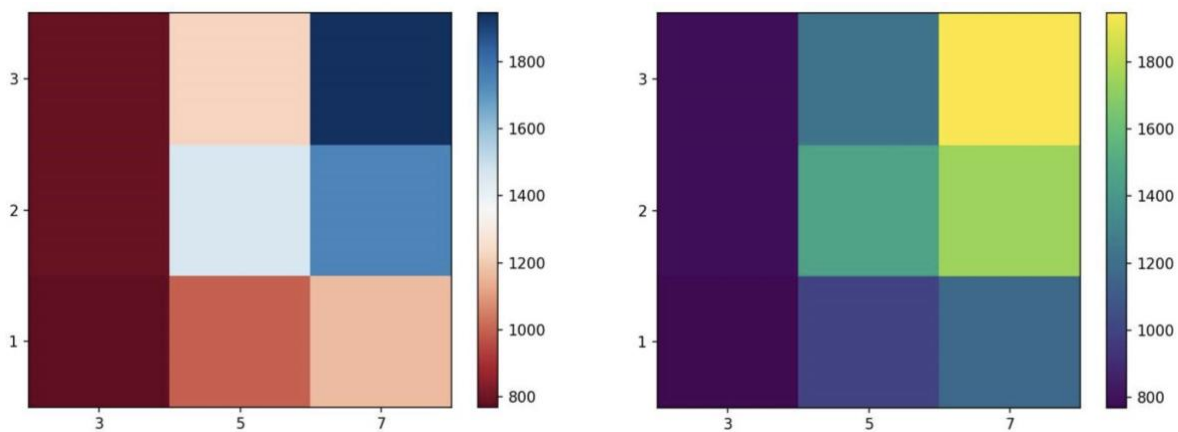
21.2.5 Pseudo Color Plot

A **pcolor plot** displays matrix data as colored cells, ideal for visualizing pivot tables.

Syntax:

```
plt.pcolor(C, cmap='viridis')
```

Where C is a matrix and cmap defines the color scheme.



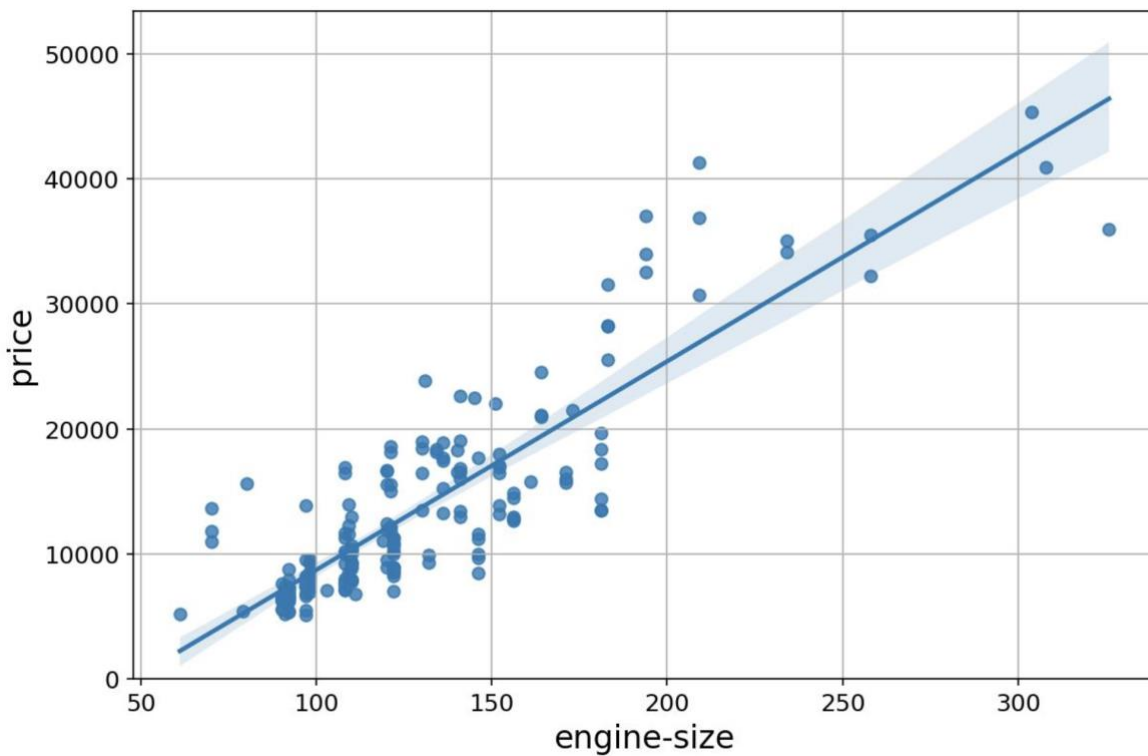
21.3 Seaborn Functions

21.3.1 Regression Plot

A **regression plot** draws a scatter plot along with a fitted regression line and confidence interval.

Syntax:

```
sns.regplot(x='header_1', y='header_2', data=df)
```



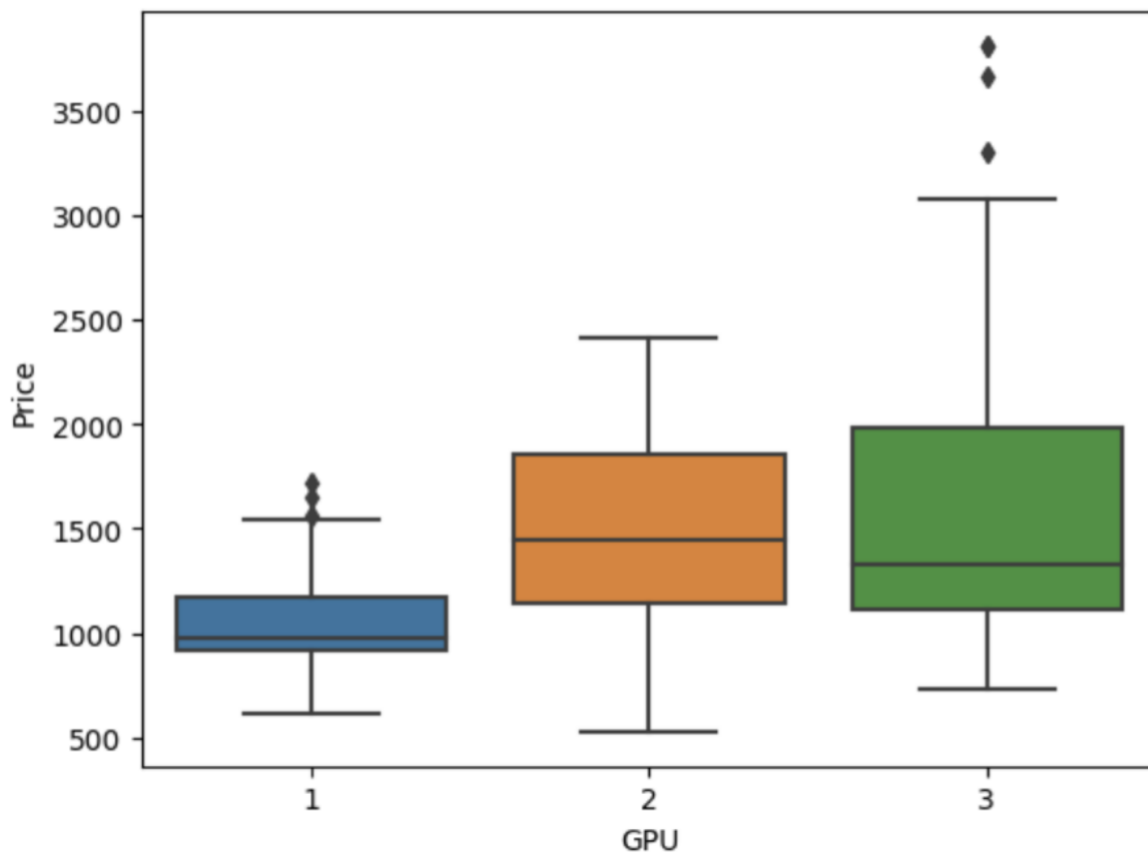
21.3.2 Box and Whisker Plot

A **box plot** visualizes the distribution of numerical data, showing quartiles and outliers.

Syntax:

```
sns.boxplot(x='category', y='value', data=df)
```

The box represents the interquartile range (IQR), and whiskers extend to show the rest of the distribution.



21.3.3 Residual Plot

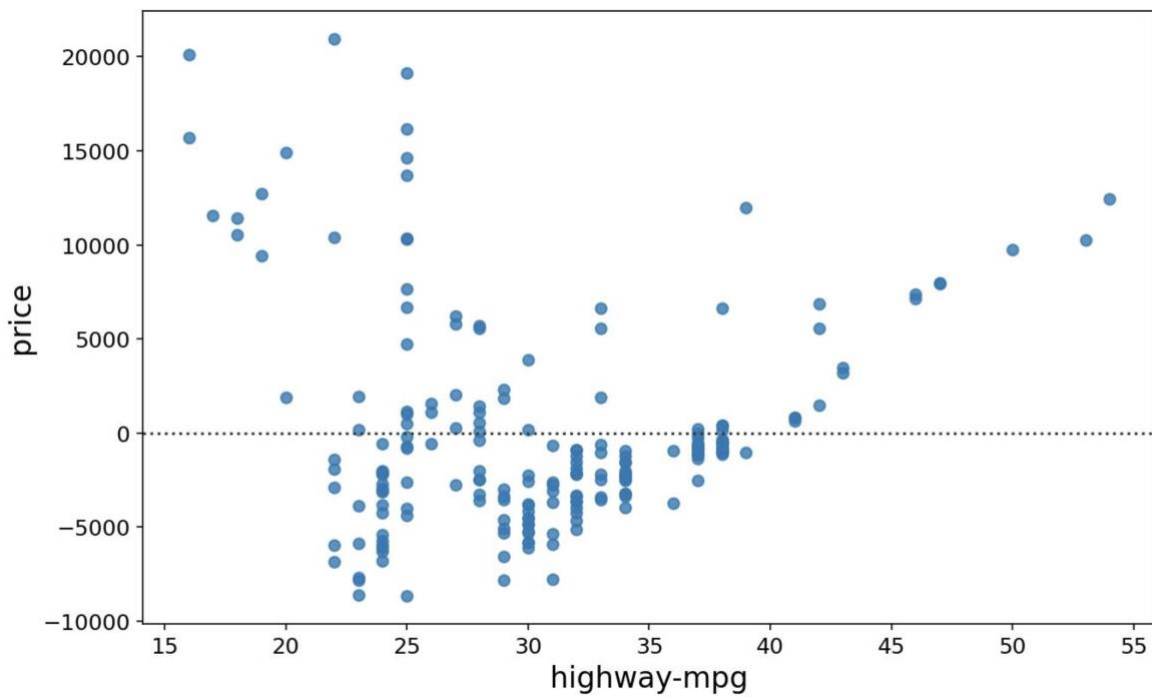
A **residual plot** is used to evaluate regression model performance by showing the difference between observed and predicted values.

Syntax:

```
sns.residplot(x='header_1', y='header_2', data=df)
```

Alternatively:

```
sns.residplot(x=df['header_1'], y=df['header_2'])
```



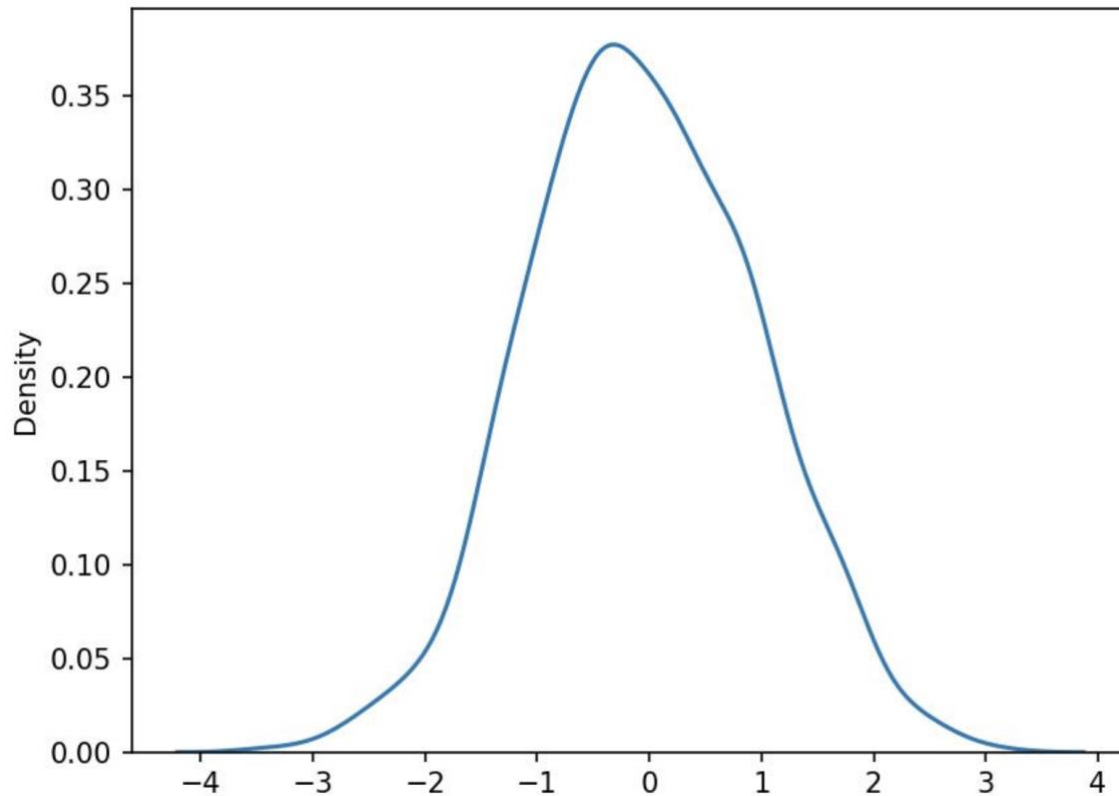
21.3.4 KDE Plot

A **Kernel Density Estimate (KDE) plot** visualizes the probability distribution of a dataset.

Syntax:

```
sns.kdeplot(X)
```

This plot shows the likelihood of data occurrences along the value range.



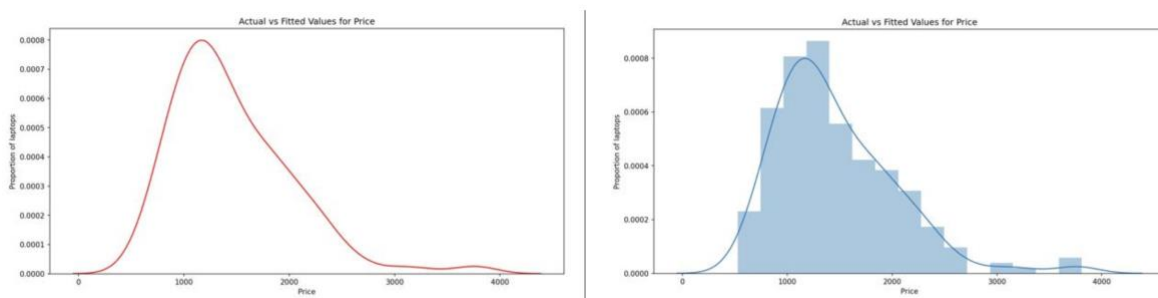
21.3.5 Distribution Plot

A **distribution plot** combines a histogram and a KDE plot, allowing comparison of both in a single visualization.

Syntax:

```
sns.distplot(X, hist=False)
```

Setting `hist=True` includes both the histogram and the KDE curve.



21.4 Conclusion

This section summarizes key visualization techniques in Python using **Matplotlib** and **Seaborn**. These plots—ranging from simple line and scatter plots to complex regression and KDE plots—enable better insights into data patterns and relationships.

22 Correlation Between Variables

22.1 Understanding Correlation

Correlation is a statistical metric that measures the degree to which two variables are interdependent. In other words, it shows how the change in one variable affects the change in another variable over time.

22.1.1 Examples of Correlation

- **Smoking and Lung Cancer:** There is a positive correlation because individuals who smoke have a higher chance of developing lung cancer.
- **Umbrella and Rain:** When precipitation increases, more people carry umbrellas. If it doesn't rain, fewer people use umbrellas. Therefore, these two variables are interdependent and correlated.

22.2 Correlation vs. Causation

It is important to remember that **correlation does not imply causation**. Although umbrellas and rain are correlated, we cannot conclude that umbrellas cause rain or vice versa. In data science, we usually focus on understanding correlation rather than establishing causation.

22.3 Visualizing Correlation

To understand relationships between variables, we can visualize them using **scatter plots** and **regression lines**.

22.3.1 Example 1: Engine Size vs. Price

When comparing engine size and car price:

- A scatter plot with a regression line shows a **positive linear relationship**.
- As engine size increases, the car price also increases.
- The steep positive slope of the line indicates a **strong positive correlation**.

You can use **Seaborn's regplot()** function to create this type of plot:

```
sns.regplot(x='engine-size', y='price', data=df)
```

22.3.2 Example 2: Highway Miles per Gallon vs. Price

When comparing highway miles per gallon (mpg) and car price:

- The scatter plot shows a **negative linear relationship**.
- As highway mpg increases, car price decreases.
- Despite being negative, the steep slope means highway mpg is still a **good predictor of price**.

```
sns.regplot(x='highway-mpg', y='price', data=df)
```

22.3.3 Example 3: Peak RPM vs. Price

In this case, both low and high RPM values correspond to low and high prices, showing **no clear pattern**. Therefore, RPM is **weakly correlated** with price and is not a reliable predictor.

22.4 Summary

- Correlation measures the interdependence between variables.
- Correlation does not imply causation.
- Visualizations like scatter plots and regression lines help identify the type and strength of correlation.
- Relationships can be positive, negative, or weak depending on how one variable affects another.

23 Correlation Between Variables

23.1 Introduction to Correlation Methods

In this section, we examine various statistical methods for measuring the strength and direction of relationships between continuous numerical variables. One of the most common and powerful methods is the **Pearson Correlation**.

23.2 Pearson Correlation

The **Pearson Correlation** method measures the linear relationship between two continuous variables. It provides two key outputs:

- **Correlation Coefficient (r)**
- **p-value**

23.2.1 Interpreting the Correlation Coefficient

- A value **close to 1** indicates a **strong positive correlation**.
- A value **close to -1** indicates a **strong negative correlation**.
- A value **close to 0** indicates **no correlation** between the variables.

23.2.2 Interpreting the p-value

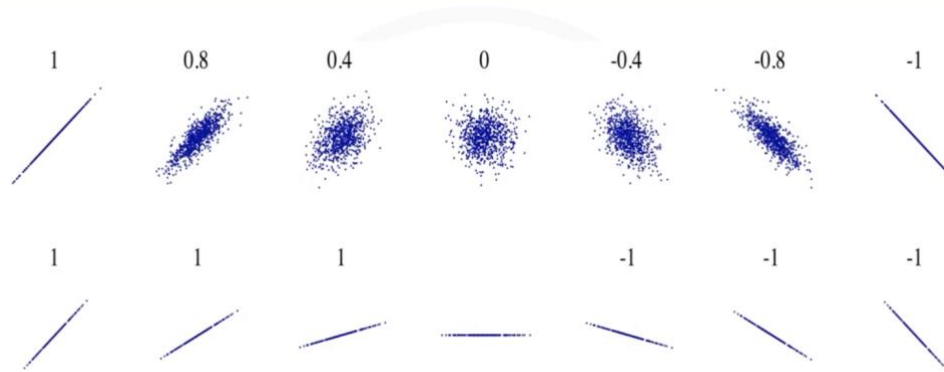
The **p-value** indicates the level of confidence we have in the correlation coefficient.

- $p < 0.001 \rightarrow$ Strong certainty about the correlation.
- $0.001 \leq p < 0.05 \rightarrow$ Moderate certainty.
- $0.05 \leq p < 0.1 \rightarrow$ Weak certainty.
- $p \geq 0.1 \rightarrow$ No certainty of correlation.

23.2.3 Strong Correlation Example

A **strong correlation** occurs when the correlation coefficient is close to **1 or -1** and the **p-value < 0.001**.

Pearson Correlation



https://en.wikipedia.org/wiki/Correlation_and_dependence

For example, when analyzing the relationship between **horsepower** and **car price**:

- The correlation coefficient is approximately **0.8**, indicating a **strong positive correlation**.
- The p-value is **much smaller than 0.001**, meaning we are highly certain about this relationship.

You can calculate the Pearson Correlation easily using the **SciPy stats package**:

```
from scipy import stats
stats.pearsonr(df['horsepower'], df['price'])
```

23.3 Correlation Heat Map

To visualize multiple correlations between several variables, we can create a **correlation heat map**.

23.3.1 Understanding the Heat Map

- The color scheme represents the **Pearson correlation coefficient** between pairs of variables.
- A **dark red diagonal line** shows where each variable is correlated with itself (always equal to 1).
- This provides a clear visual overview of how all variables relate to one another, especially how they relate to **price**.

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, cmap='RdBu_r')
plt.show()
```

23.4 Summary

- The Pearson Correlation is used to measure linear relationships between continuous variables.
- The correlation coefficient and p-value together help interpret strength and reliability.
- Heat maps give a comprehensive view of how all variables relate to each other.
- Understanding these relationships is key in data analysis, especially in predicting values like price.

24 Chi-Square Test for Categorical Variables

24.1 Introduction

The **Chi-Square Test** is a statistical method used to determine whether there is a significant association between two categorical variables. It is widely applied in social sciences, marketing, healthcare, and other fields to analyze survey data, experiments, and observational studies.

24.2 Concept

The Chi-Square Test is a **non-parametric** statistical method that evaluates whether the observed frequencies in each category differ significantly from the expected frequencies, assuming no association between the variables.

The test relies on the **Chi-Square distribution**, which varies according to the **degrees of freedom (df)**. A Chi-Square distribution table provides **critical values** for given degrees of freedom and significance levels (α), which help determine if the test statistic is extreme enough to reject the null hypothesis.

24.3 Hypotheses

24.3.1 Null Hypothesis (H_0)

There is **no association** between the categorical variables. Any observed difference is due to random chance.

24.3.2 Alternative Hypothesis (H_1)

There is a **significant association** between the categorical variables, meaning the differences are not due to chance alone.

24.4 Formula

The Chi-Square statistic is calculated as:

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

Where:

- O_i = Observed frequency for category i
- E_i = Expected frequency for category i , calculated as:

$$[E_i = \{(\{\text{Row Total}\} * \{\text{Column Total}\}) / \{\{\text{Grand Total}\}\}]$$

The degrees of freedom (df) are calculated using:

$$[df = (r - 1) * (c - 1)]$$

Where:

- **r** = Number of rows
- **c** = Number of columns

24.5 Chi-Square Distribution Table Example

df	$\alpha = 0.05$	Critical Value
1	0.05	3.841

Decision Rule:

- If $\chi^2 > 3.841 \rightarrow$ Reject H_0
- If $\chi^2 \leq 3.841 \rightarrow$ Fail to reject H_0

The higher the χ^2 value, the stronger the evidence against the null hypothesis.

24.6 Python Implementation Example

```
import pandas as pd
from scipy.stats import chi2_contingency

# Create contingency table
data = [[20, 30], # Male: [Like, Dislike]
        [25, 25]] # Female: [Like, Dislike]

# Create DataFrame
df = pd.DataFrame(data, columns=["Like", "Dislike"], index=["Male", "Female"])

# Perform Chi-Square Test
chi2, p, dof, expected = chi2_contingency(df)

# Display results
print("Chi-square Statistic:", chi2)
print("Degrees of Freedom:", dof)
print("P-value:", p)
print("Expected Frequencies:\n", expected)
```

Output:

```
Chi-square Statistic: 1.008
Degrees of Freedom: 1
P-value: 0.3156
Expected Frequencies:
[[22.5 27.5]
 [22.5 27.5]]
```

Interpretation: Since the p-value (0.3156) > 0.05, we fail to reject the null hypothesis, indicating no significant association.

24.7 Applications

- **Market Research:** Relationship between customer demographics and product preferences.
- **Healthcare:** Association between patient characteristics and disease occurrence.
- **Social Sciences:** Link between education level and voting behavior.
- **Education:** Relationship between teaching methods and performance.
- **Quality Control:** The Association between Production Conditions and Defects.

24.8 Practical Example – Weak Correlation

24.8.1 Scenario

A researcher examines the association between gender (male, female) and product preference (like, dislike). The data collected is:

Category	Like	Dislike	Total
Male	20	30	50
Female	25	25	50
Total	45	55	100

24.8.2 Step 1: Calculate Expected Frequencies

$$\begin{aligned}
 [E_{\{\text{Male,Like}\}} &= \{50 * 45\} / \{100\} = 22.5] \\
 [E_{\{\text{Male,Dislike}\}} &= \{50 * 55\} / \{100\} = 27.5] \\
 [E_{\{\text{Female,Like}\}} &= 22.5]
 \end{aligned}$$

$$[E_{\{\text{Female,Dislike}\}} = 27.5]$$

24.8.3 Step 2: Compute Chi-Square Statistic

$$[\chi^2 = \{(20-22.5)^2\} / \{22.5\} + \{(30-27.5)^2\} / \{27.5\} + \{(25-22.5)^2\} / \{22.5\} + \{(25-27.5)^2\} / \{27.5\} = 1.008]$$

24.8.4 Step 3: Degrees of Freedom

$$[df = (2-1) * (2-1) = 1]$$

24.8.5 Step 4: Interpretation

Since $1.008 < 3.841$, we **fail to reject H_0** . There is **no significant association** between gender and product preference.

24.9 Practical Example – Strong Association

24.9.1 Scenario

A study investigates the relationship between smoking status (smoker, non-smoker) and lung disease (disease, no disease). The data collected is:

Category	Disease	No Disease	Total
Smoker	50	30	80
Non-Smoker	20	100	120
Total	70	130	200

24.9.2 Step 1: Calculate Expected Frequencies

$$\begin{aligned} [E_{\text{Smoker,Disease}} &= 80 * \frac{70}{200} = 28] \\ [E_{\text{Smoker,NoDisease}} &= 80 * \frac{130}{200} = 52] \\ [E_{\text{NonSmoker,Disease}} &= 42] \end{aligned}$$

$$[E_{\text{NonSmoker,NoDisease}} = 78]$$

24.9.3 Step 2: Compute Chi-Square Statistic

$$[\chi^2 = \{(50-28)^2\}/28 + \{(30-52)^2\}/52 + \{(20-42)^2\}/42 + \{(100-78)^2\}/78 = 44.33]$$

24.9.4 Step 3: Degrees of Freedom

$$[df = (2-1) * (2-1) = 1]$$

24.9.5 Step 4: Interpretation

Since $44.33 > 3.841$, we **reject H_0** , indicating a **significant association** between smoking status and lung disease.

24.10 Conclusion

The Chi-Square Test is a powerful tool for analyzing relationships between categorical variables. By comparing observed and expected frequencies, researchers can determine whether an observed relationship is statistically significant, providing valuable insights across various disciplines.

25 Lesson Summary

At this point in the course, we know:

25.1 Key Statistical Tools

- The **describe()** function in Pandas quickly calculates key statistical measures such as mean, standard deviation, and quartiles for all numerical variables in a DataFrame.
- The **value_counts()** function summarizes categorical data by showing the count of each category.

25.2 Data Visualization Techniques

- **Box plots** provide a visual representation of data distribution for numerical variables, highlighting the median, quartiles, and outliers.
- **Scatter plots** are useful for exploring relationships between continuous variables, such as engine size and price in a car dataset.
- Use **groupby()** in Pandas to explore relationships between categorical variables.
- **Pivot tables** and **heatmaps** help visualize summarized data effectively.

25.3 Understanding Correlation

- **Correlation** measures how changes in one variable relate to changes in another variable.
- **Scatter plots** with regression lines help visualize correlations between variables.
- The **regplot()** function from the Seaborn library is particularly useful for examining correlations.

25.4 Pearson Correlation

- The **Pearson correlation** provides two important values:
 - **Correlation Coefficient (r)**: Indicates the strength and direction of the correlation.
 - **P-value**: Represents the certainty of the correlation.
- A correlation coefficient close to **1** or **-1** shows a strong positive or negative correlation, while a value near **0** suggests no correlation.
- For **P-values**, values less than **0.001** indicate strong certainty in the correlation, while larger values show weaker certainty.

25.5 Heatmaps for Correlation

- **Heatmaps** offer a comprehensive visual overview of the strength and direction of correlations among multiple variables.

Module 4

26 Model Development

26.1 Overview

In this section, we examine model development by predicting the price of a car using our dataset. This module covers:

- Simple and Multiple Linear Regression
- Model Evaluation using Visualization
- Polynomial Regression and Pipelines
- R-squared and Mean Squared Error (MSE) for in-sample evaluation
- Prediction and Decision Making
- Determining a fair value for a used car

26.2 Understanding Models and Estimators

A **model** or **estimator** is a mathematical equation used to predict a value based on one or more input values. It establishes a relationship between:

- **Independent Variables (Features):** Input variables used for prediction.
- **Dependent Variable (Target):** The output variable we aim to predict.

26.2.1 Example

If we input a car model's highway miles per gallon (MPG) as an independent variable, the dependent variable or output will be the **car price**. The more relevant data and features you include, the more accurate your model becomes.

26.3 Importance of More Data

Consider the following situation:

- You have two nearly identical cars — one **pink** and one **red**.
- Pink cars tend to sell for significantly less.
- If your model does not include **color** as an independent variable, it will predict the same price for both cars, even though one may sell for less.

This example shows why including more relevant data and features is crucial for building accurate models.

26.4 Exploring Different Types of Models

In this Module, we explore various regression models, including:

- **Simple Linear Regression** – Predicts the target variable using a single feature.
- **Multiple Linear Regression** – Uses multiple features for better prediction accuracy.

- **Polynomial Regression** – Models non-linear relationships between features and target variables.

These models provide a foundation for making predictions and informed decisions based on data.

27 Linear Regression

27.1 Introduction

In this section, we explore **Simple Linear Regression (SLR)** and **Multiple Linear Regression (MLR)** — two fundamental methods used to model relationships between variables.

27.2 Simple Linear Regression (SLR)

27.2.1 Understanding the Concept

Simple Linear Regression is used to understand the relationship between two variables:

- **Predictor (Independent Variable, x)**
- **Target (Dependent Variable, y)**

The goal is to find a linear relationship between these variables, represented as:

$$[y = b_0 + b_1 x]$$

Where:

- **b_0** = Intercept (the value of y when x = 0)
- **b_1** = Slope

When we fit or train the model, we determine these parameters (**b_0** and **b_1**).

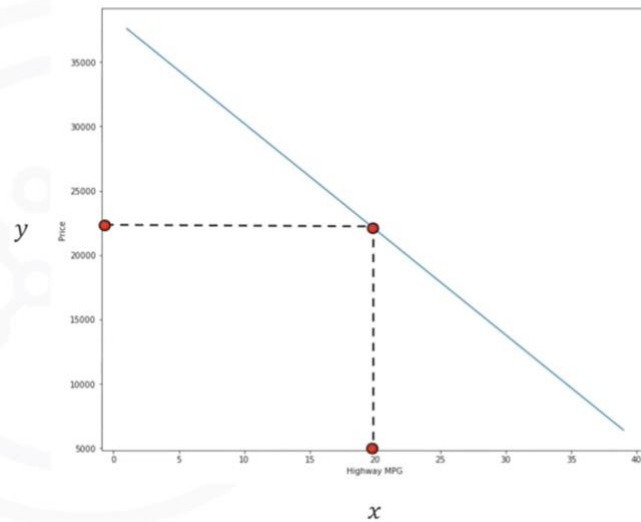
27.2.2 Example

Imagine predicting a car's **price** based on its **highway miles per gallon (MPG)**. Although it's hard to determine the car price directly, the MPG value is available in the owner's manual. If we assume a linear relationship, we can use it to estimate the car's price.

For example, if the highway MPG is **20**, substituting it into the model might predict a price of **\$22,000**.

Simple linear regression: Prediction

$$\begin{aligned} y &= 38423 - 821x \\ &= 38423 - 821(20) \\ &= 22\,003 \end{aligned}$$



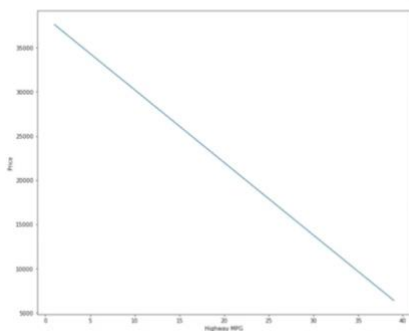
27.2.3 Model Training

To determine the regression line, we use **training data points**. These points are stored as:

- **x**: Independent variable(s) — in this case, MPG
- **y**: Dependent variable — price

Each data sample corresponds to a row in the dataset. During model training, the algorithm fits the best possible line through these data points.

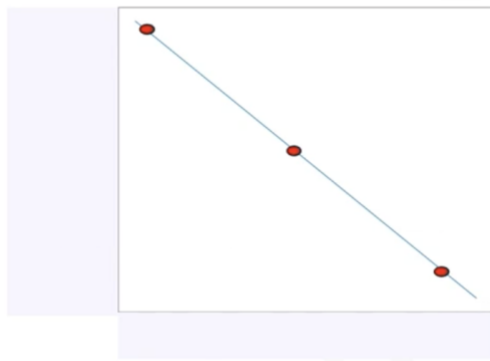
Simple linear regression: Fit



Fit

(b_0, b_1)

Simple linear regression: Fit



$$X = \begin{bmatrix} 0 \\ 20 \\ 40 \end{bmatrix}$$

$$Y = \begin{bmatrix} 38423 \\ 22003 \\ 5583 \end{bmatrix}$$

27.2.4 Noise in Data

Real-world data often includes uncertainty or random variation, commonly referred to as **noise**. This means not all data points will fall exactly on the regression line.

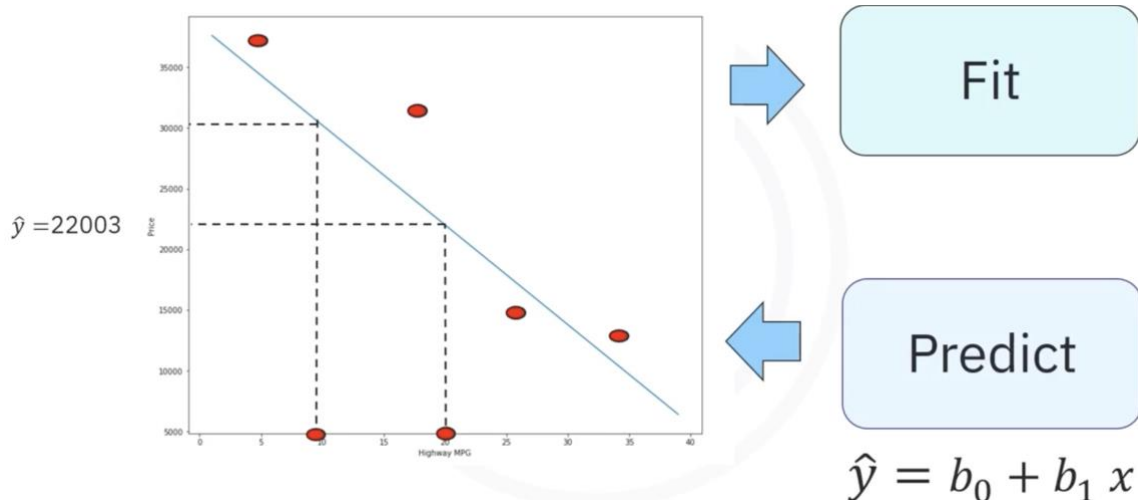
The noise distribution is usually centered around zero — most values are small, but occasionally large deviations occur. This accounts for other unknown factors affecting the target variable.

27.2.5 Prediction Process

Once trained, we can use the model to make predictions for unseen data:

- Example: Predict the price of a car with **20 MPG**.
- The model output is denoted as $\hat{y}$ (**y-hat**), indicating it's an **estimate**, not the true value.

By comparing $\hat{y}$ (predicted value) with y (actual value), we can assess how accurate our model is. If the linear assumption holds, the difference between predicted and actual values mainly reflects the noise.



27.3 Implementing Simple Linear Regression in Python

To fit and use a simple linear regression model in Python:

27.3.1 Steps

1. **Import the model:**
2. `from sklearn.linear_model import LinearRegression`
3. **Create the model object:**
4. `lm = LinearRegression()`
5. **Define the predictor and target variables:**
6. `X = df[['highway-mpg']]`
7. `Y = df['price']`
8. **Fit the model:**
9. `lm.fit(X, Y)`
10. **Make predictions:**
11. `Yhat = lm.predict(X)`

27.3.2 Model Parameters

After fitting the model:

- **Intercept (b_0):** `lm.intercept_`
- **Slope (b_1):** `lm.coef_`

The relationship between price and highway MPG can be written as:

[Price = 38,423.31 - 821.73 * (Highway MPG)]

27.4 Multiple Linear Regression (MLR)

27.4.1 Concept

Multiple Linear Regression extends SLR by using **two or more independent variables** to predict the target variable.

The general equation is:

[$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$]

Where:

- y = Target variable
- $x_1, x_2, \dots, x_n$ = Predictor variables
- $b_0, b_1, b_2, \dots, b_n$ = Model coefficients

27.4.2 Example

Suppose we want to predict car price using four predictors:

- Engine size

- Horsepower
- Curb weight
- Highway MPG

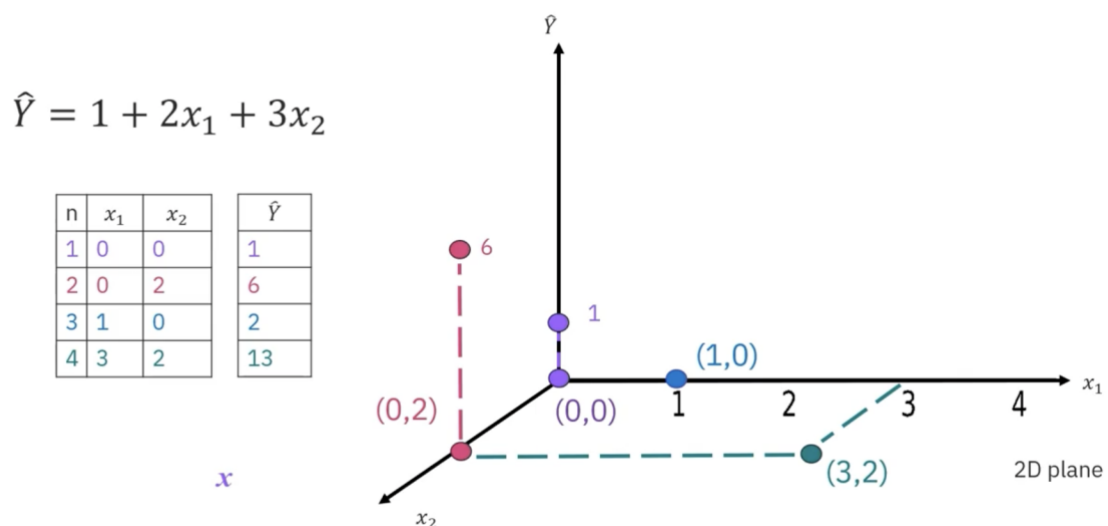
Each coefficient represents how much the car price changes with a one-unit change in that predictor, keeping others constant.

27.4.3 Visualization

If we have only two predictor variables (x_1 and x_2), we can visualize them on a **2D plane**:

- Each point on the plane represents values of x_1 and x_2 .
- The corresponding predicted value $\hat{y}$ is plotted in the vertical direction, showing the model's output.

- This is shown below where



27.4.4 Implementation in Python

To fit a Multiple Linear Regression model:

1. **Extract predictor variables:**
2. `Z = df[['engine-size', 'horsepower', 'curb-weight', 'highway-mpg']]`
3. **Train the model:**
4. `lm.fit(Z, Y)`
5. **Make predictions:**
6. `Yhat = lm.predict(Z)`

27.4.5 Model Attributes

- **Intercept (b_0):** `lm.intercept_`
- **Coefficients ($b_1, b_2, \dots$):** `lm.coef_`

This allows us to express the model equation in the form:

[$\text{Price} = b_0 + b_1(\text{engine-size}) + b_2(\text{horsepower}) + b_3(\text{curb-weight}) + b_4(\text{highway-mpg})$]

27.5 Summary

- **Simple Linear Regression** uses one predictor variable to estimate a target.
- **Multiple Linear Regression** uses multiple predictors for greater accuracy.
- Both models are foundational for understanding more complex regression methods.
- The process includes **training**, **estimating parameters**, **predicting**, and **evaluating** model performance.

28 Model Evaluation Using Visualization

28.1 Introduction

In this section, we explore **model evaluation using visualization**. Visualization helps us understand the relationship between variables, assess model performance, and identify potential issues in predictions.

28.2 Regression Plots

Regression plots estimate the relationship between two variables, indicating both the **strength** and **direction** of the correlation.

- **Horizontal Axis:** Independent variable
- **Vertical Axis:** Dependent variable
- **Each Point:** Represents a data sample
- **Fitted Line:** Represents the predicted value

28.2.1 Creating a Regression Plot

A simple way to create a regression plot is by using the **Seaborn** library's `regplot()` function.

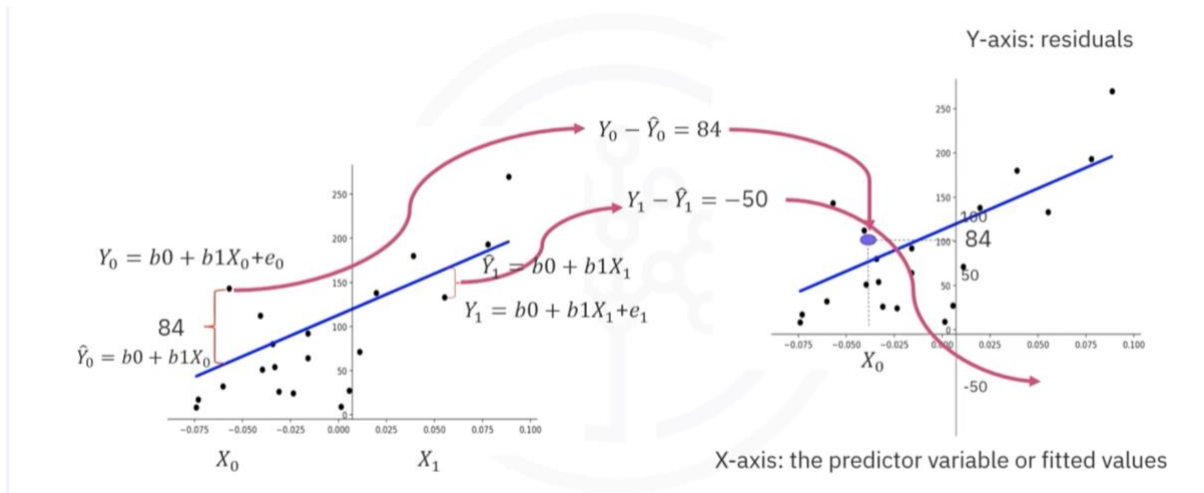
Steps:

1. Import the Seaborn library.
2. Use the `regplot()` function with the following parameters:
 - `x`: Name of the column containing the independent variable (feature).
 - `y`: Name of the column containing the dependent variable (target).
 - `data`: The name of the DataFrame.

The resulting plot visually displays the relationship between the variables and the fitted regression line.

28.3 Residual Plots

Residual plots show the **error between actual and predicted values**. This is obtained by subtracting the predicted value from the actual target value.



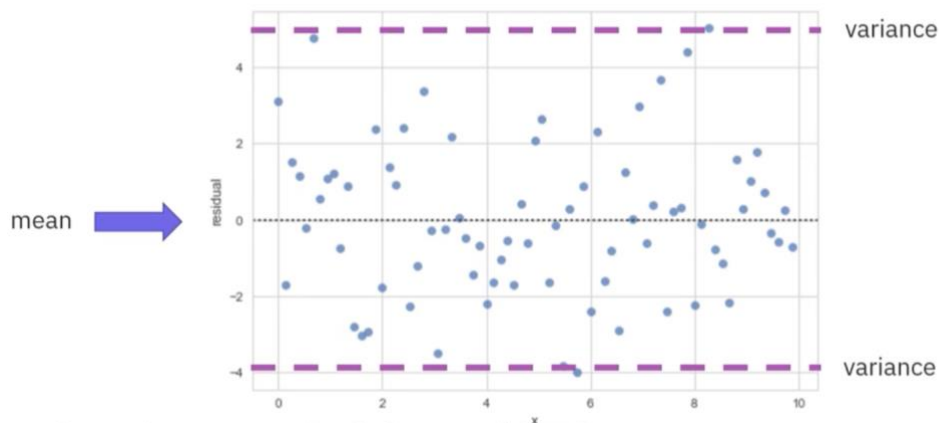
28.3.1 Interpretation

- **Vertical Axis:** Residual error (difference between actual and predicted values)
- **Horizontal Axis:** Dependent variable

We expect residuals to have:

- A mean close to zero
- Even distribution around the x-axis
- Similar variance (no curvature)

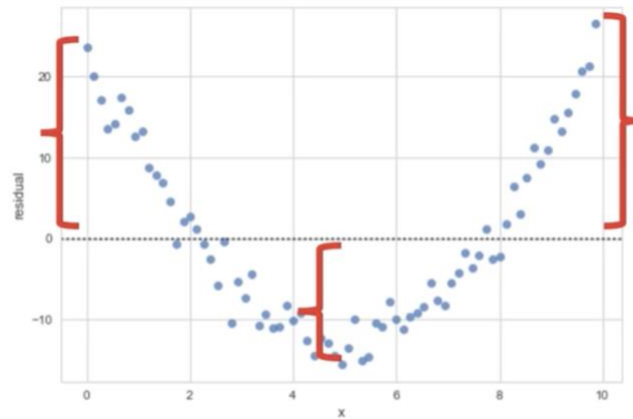
A residual plot with evenly distributed points suggests that a **linear model** is appropriate.



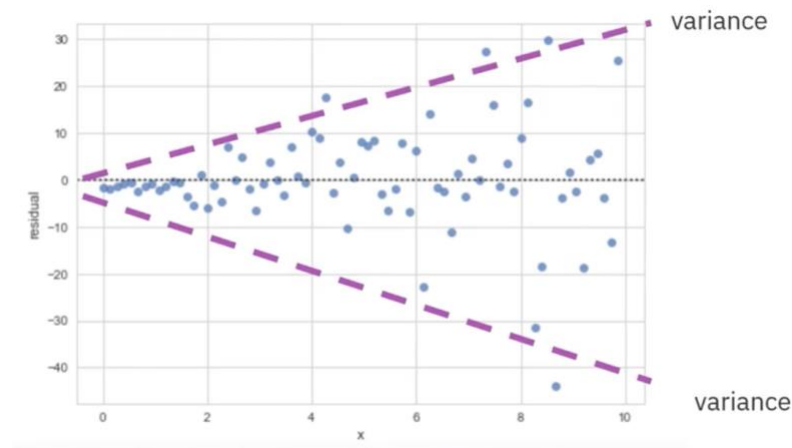
28.3.2 Identifying Model Issues

If the residuals show curvature or patterns:

- **Curvature in residuals:** Indicates that a **nonlinear model** may be more suitable.



- **Variance increasing with x:** Suggests the model is incorrect or violates assumptions of linear regression.



28.3.3 Creating a Residual Plot

You can create a residual plot using Seaborn's `residplot()` function.

Steps:

1. Import the Seaborn library.
2. Use the `residplot()` function with:
 - First parameter: Series of the independent variable (feature)
 - Second parameter: Series of the dependent variable (target)

If the residuals exhibit curvature, it indicates potential issues with linearity.

28.4 Distribution Plots

Distribution plots compare **predicted values** with **actual values** and are useful for visualizing models that involve multiple independent variables.

28.4.1 How It Works

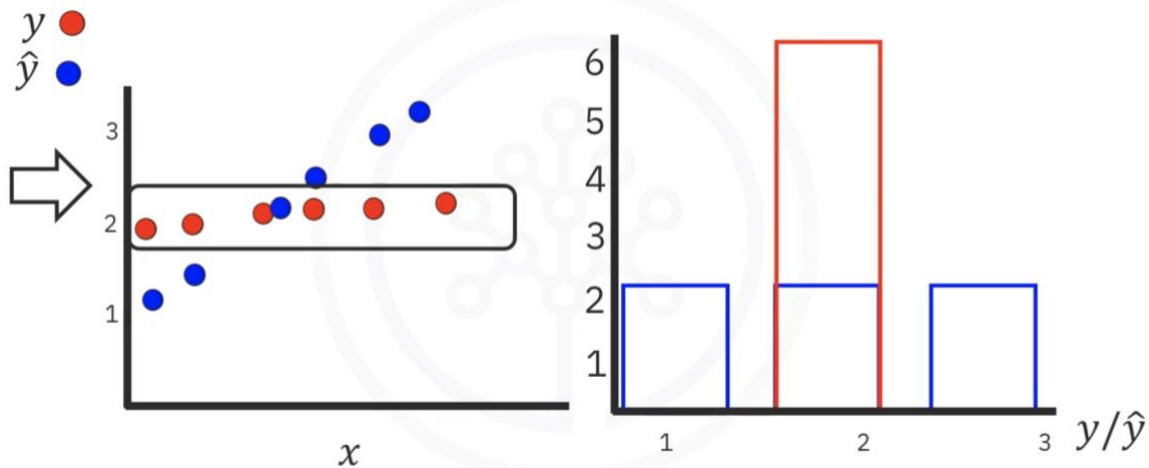
1. Count and plot the number of predicted points approximately equal to each target value.
2. Repeat the process for actual target values.
3. The vertical axis is scaled so that the **area under the distribution equals one**.

28.4.2 Example: Distribution Plot for Price Prediction

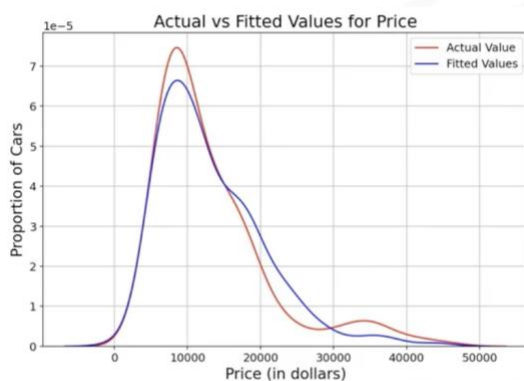
- **Dependent Variable (Feature):** Price
- **Fitted Values (Predicted):** Blue
- **Actual Values:** Red

Observations:

- Predicted prices between **40,000 to 50,000** are less accurate.
- Predictions between **10,000 to 20,000** are much closer to the actual target values.



When using multiple features, the predicted values often align more closely with actual values compared to single-feature models.



28.4.3 Code Example for Distribution Plot

To create a distribution plot in Seaborn:

```
import seaborn as sns
sns.distplot(actual_values, hist=False, color='red', label='Actual Values')
sns.distplot(predicted_values, hist=False, color='blue', label='Predicted Values')
```

This code plots both the actual and predicted distributions, allowing visual comparison of model accuracy.

29 Polynomial Regression and Pipelines

29.1 Introduction

In this section, we explore **polynomial regression** and **pipelines**. These concepts help us handle non-linear relationships in data and streamline complex workflows.

29.2 Polynomial Regression

When a linear model does not provide the best fit for our data, we can use **polynomial regression**. In this method, we transform the original data into a polynomial form and then apply **linear regression** to fit the parameters.

29.2.1 Purpose

Polynomial regression is beneficial for describing **curvilinear relationships**, where the relationship between variables is not strictly linear.

29.2.2 Curvilinear Relationship

A **curvilinear relationship** occurs when the predictor variables include higher-order terms, such as squared or cubed values. For instance:

- A **quadratic model** (2nd order polynomial) squares the predictor variable.
- A **cubic model** (3rd order polynomial) cubes the predictor variable.

By increasing the polynomial order, we can capture more complex relationships in the data. The higher the degree, the more variation the model can represent. However, choosing the correct degree is essential to avoid overfitting.

- Quadratic – 2nd order

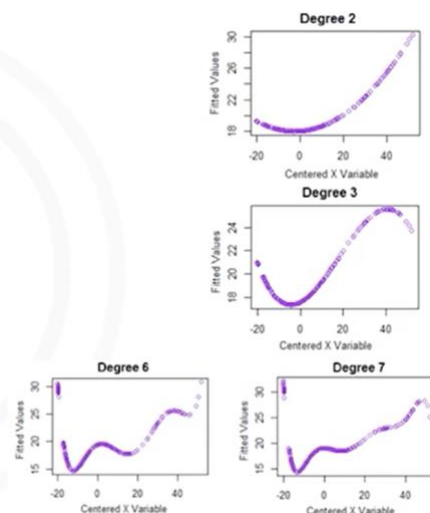
$$\hat{Y} = b_0 + b_1 x_1 + b_2 (x_1)^2$$

- Cubic – 3rd order

$$\hat{Y} = b_0 + b_1 x_1 + b_2 (x_1)^2 + b_3 (x_1)^3$$

- Higher order

$$\hat{Y} = b_0 + b_1 x_1 + b_2 (x_1)^2 + b_3 (x_1)^3 + \dots$$



29.2.3 Example in Python

We can create polynomial regression models in Python using the **NumPy** and **scikit-learn** libraries.

29.2.3.1 Using NumPy

The `polyfit()` function in NumPy can fit a polynomial model. For example, a third-order polynomial regression might look like:

$$-1.557x^3 + 204.8x^2 + 8965x + 1.37 \times 10^5$$

29.2.3.2 Multidimensional Polynomial Regression

For multidimensional data, polynomial regression becomes more complex. NumPy's `polyfit()` cannot handle this directly. Instead, we use **scikit-learn's preprocessing library** to create polynomial features.

Steps:

1. Import `PolynomialFeatures` from `sklearn.preprocessing`.
2. Specify the degree of the polynomial in the constructor.
3. Transform the original features using the `fit_transform()` method.

This transformation creates new polynomial features, expanding the original data for model fitting.

- The "preprocessing" library in scikit-learn:

```
from sklearn.preprocessing import PolynomialFeatures
pr=PolynomialFeatures(degree=2, include_bias=False)
x_polly=pr.fit_transform(x[['horsepower', 'curb-weight']])
```

29.3 Feature Scaling and Normalization

As the dimension of data increases, it becomes important to normalize features. Using **scikit-learn's preprocessing module**, we can standardize all features simultaneously.

Steps:

1. Import `StandardScaler` from `sklearn.preprocessing`.
2. Fit the scaler object to the data using `fit()`.
3. Transform the data using `transform()` or `fit_transform()` to obtain scaled features.

Other normalization methods are also available in the preprocessing library to improve model performance and consistency.

- For example we can Normalize the each feature simultaneously:

```
from sklearn.preprocessing import StandardScaler  
  
SCALE=StandardScaler()  
SCALE.fit(x_data[['horsepower', 'highway-mpg']])  
  
x_scale=SCALE.transform(x_data[['horsepower', 'highway-mpg']])
```

29.4 Pipelines

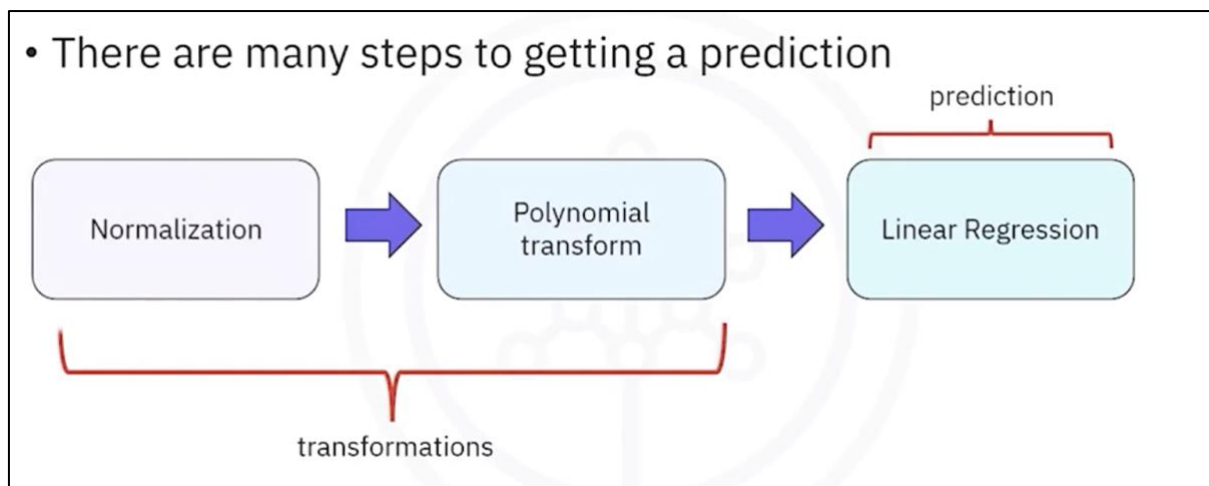
Pipelines in **scikit-learn** simplify machine learning workflows by chaining multiple data transformation and modeling steps together.

29.4.1 Purpose

Pipelines allow sequential execution of multiple steps, such as:

1. Normalization
2. Polynomial transformation
3. Model fitting or prediction

The final step always performs a **prediction**.



29.4.2 Creating a Pipeline

Steps:

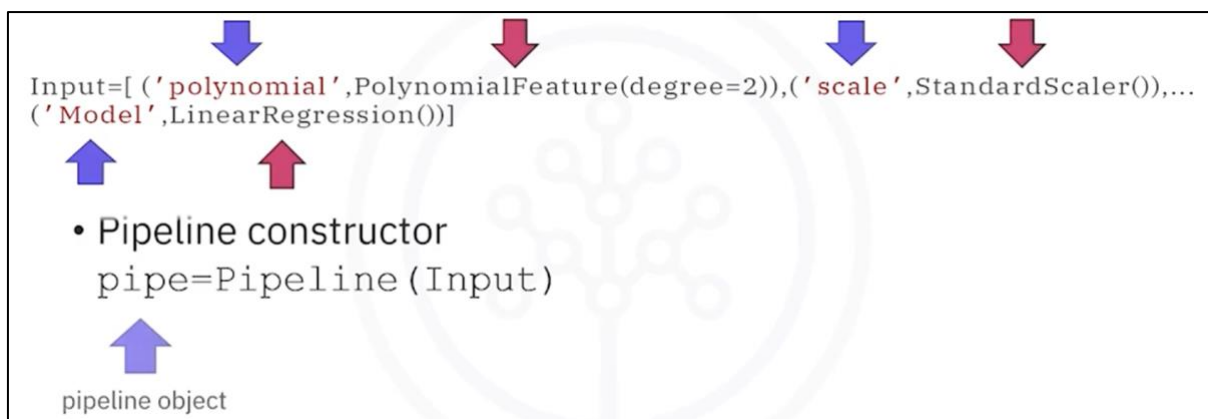
1. Import the necessary modules.
2. Import the Pipeline class from sklearn.pipeline.

```

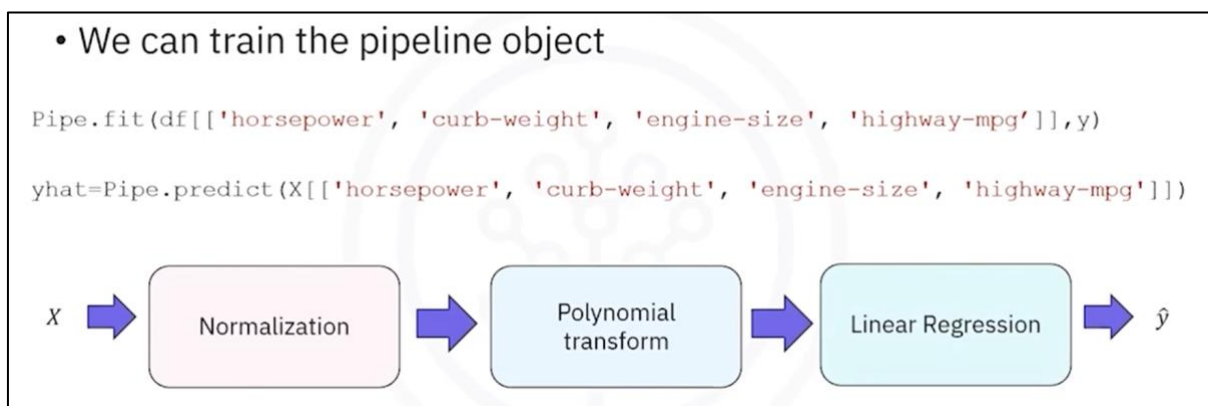
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

```

3. Create a list of tuples, where each tuple contains:
 - The name of the estimator.
 - The estimator or transformer object.
4. Pass the list into the Pipeline constructor.



Once created, the pipeline object can be **trained** using the fit() method and used to **predict** using the predict() method.



29.4.3 Example Workflow

When a pipeline is trained and used for prediction, it:

- Normalizes the input data.
- Applies a polynomial transformation.
- Performs linear regression to generate predictions.

This approach ensures cleaner, more organized code and reduces the likelihood of errors during the modeling process.

30 Feature Scaling / Normalization

30.1 Understanding Feature Scaling

Feature scaling ensures that features with large numerical ranges do not dominate features with smaller ranges. We normalize values so that each feature contributes equally to the model.

30.2 Example Feature Ranges

- Engine Size: 1.0 to 6.5
- Price: 5,000 to 45,000
- Horsepower: 40 to 280
- Mileage: 10 to 60

Without scaling, models may become biased toward features with larger values.

30.3 What Normalization Does

Normalization transforms values to a similar scale. After scaling:

- Mean becomes **0**
- Standard deviation becomes **1**
- No single feature dominates others

30.4 When Feature Scaling Is Needed

We apply scaling when working with models that rely on magnitude or distance:

- Linear Regression
- Logistic Regression
- K-Nearest Neighbors (KNN)
- K-Means Clustering
- Neural Networks

We typically do *not* apply scaling for tree-based models:

- Decision Tree
- Random Forest
- XGBoost

30.5 Steps for Feature Scaling

30.5.1 Step 1: Import StandardScaler

```
from sklearn.preprocessing import StandardScaler
```

30.5.2 Step 2: Create Scaler and Fit It to Data

```
scaler = StandardScaler()
```

```
scaler.fit(df[['horsepower', 'engine-size']])
```

30.5.3 Step 3: Transform the Data

```
scaled_data = scaler.transform(df[['horsepower', 'engine-size']])
```

Or perform both in a single step:

```
scaled_data = scaler.fit_transform(df[['horsepower', 'engine-size']])
```

30.6 Before and After Scaling

Feature	Original Value	Scaled Value
Horsepower	150	0.45
Horsepower	80	-1.25
Engine Size	3.5	1.10
Engine Size	1.6	-0.85

After scaling, the values typically fall within approximately -3 to +3. This helps models train more efficiently and accurately.

31 Polynomial Transformation

31.1 Understanding Polynomial Transformation

Polynomial transformation creates additional features by including powers and interaction terms of existing features. This allows a model to capture **non-linear** relationships.

31.2 Example

Original feature:

- x

Polynomial transformation of degree 3 produces:

- $1, x, x^2, x^3$

31.3 Shape Change Example

Original data:

- $Z.shape \rightarrow (201, 4)$ meaning 201 samples and 4 features

After polynomial transformation (degree = 2):

- $Z_{pr}.shape \rightarrow (201, 15)$

31.4 Reason for 15 Features

- Bias term $\rightarrow 1$
- Each feature $\rightarrow a, b, c, d$
- Squared terms $\rightarrow a^2, b^2, c^2, d^2$
- Interaction terms $\rightarrow a*b, a*c, a*d, b*c, b*d, c*d$

Feature Type	Count
Bias (constant 1)	1
Original features	4
Squared terms	4
Interaction terms	6
Total	15

31.5 Code for Polynomial Transformation

```
from sklearn.preprocessing import PolynomialFeatures
pr = PolynomialFeatures(degree=2)
Z_pr = pr.fit_transform(Z)
```

31.6 Why Use Polynomial Transformation?

If the relationship between variables is not a straight line, polynomial features allow the model to learn curves. This often leads to better model performance when dealing with non-linear patterns.

31.7 Comparison

- **Linear Regression:** $\text{price} = a * x + b$
- **Polynomial Regression:** $\text{price} = a * x^2 + b * x + c$

Polynomial models create more flexible trend lines.

31.8 Summary

- **Feature Scaling** ensures all features contribute equally.
- **Polynomial Transformation** expands the feature space to capture non-linear relationships.
- Both techniques help improve model accuracy and model stability.

32 Kernel Density Estimation (KDE) Plots for Model Evaluation

32.1 Introduction

Kernel Density Estimation (KDE) plots help visualize data distributions by estimating their probability density function (PDF). In regression analysis, they are useful for comparing the distribution of actual and predicted values.

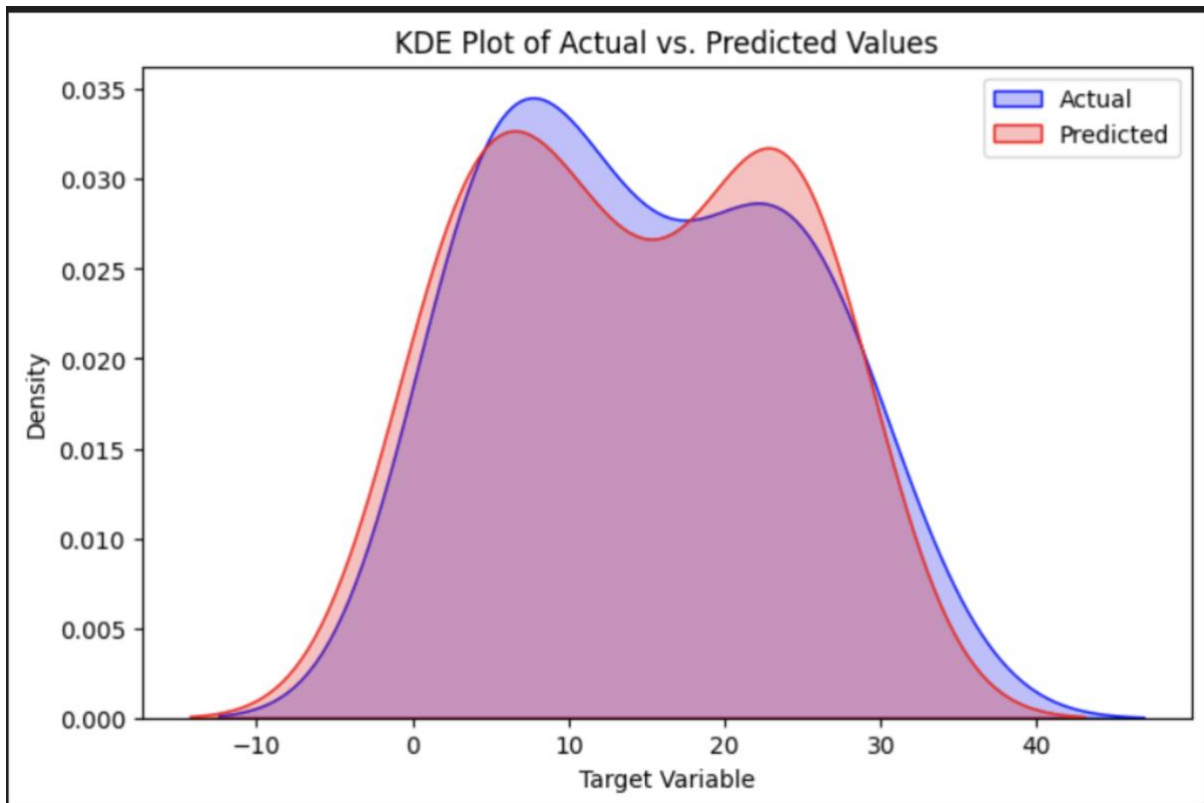
32.2 Why Use KDE Plots?

KDE plots are valuable for model evaluation because they:

- Provide a smooth approximation of data distribution.
- Allow effective comparison between true and predicted values.
- Do not rely on bin size selection, unlike histograms.
- Highlight deviations between observed and predicted outputs.

32.3 Implementing KDE Plots in Python

The `kdeplot()` function from Seaborn can be used to compare the distributions of actual and predicted values. It produces a smooth and clear visualization.



32.4 Example: Evaluating a Regression Model

Below is an example where a simple linear regression model is trained on synthetic data to demonstrate KDE-based evaluation.

```
import numpy as npy
import pandas as pds
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

# Generating Sample Data
npy.random.seed(42)
x = npy.random.rand(100) * 10
y = 3 * x + npy.random.normal(0, 3, 100)
data = pds.DataFrame({'X': x, 'Y': y})
```

```
# Splitting Data
X_train, X_test, y_train, y_test = train_test_split(data[['X']], data['Y'], test_size=0.2, random_state=42)

# Training a Model
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Plotting KDE for Observed vs. Predicted Values
plt.figure(figsize=(8, 5))
sns.kdeplot(y_test, label='Actual', fill=True)
sns.kdeplot(y_pred, label='Predicted', fill=True)
plt.xlabel('Target Variable')
plt.ylabel('Density')
plt.title('KDE Plot of Actual vs. Predicted Values')
plt.legend()
plt.show()
```

32.5 Interpretation of the KDE Plot

- **Overlap Between Distributions:** A strong overlap suggests the model is capturing the general trend accurately.
- **Peak Differences:** Differences in peak height or location indicate where the model estimation may diverge from true values.
- **Distribution Spread:** A narrower predicted distribution may indicate underestimation of variance.
- **Tails Behavior:** Similar tails show the model is handling extreme values reasonably well.

32.6 Conclusion

KDE plots provide a clear and effective way to compare predicted and actual values in regression analysis. They are a modern replacement for deprecated plotting methods and help provide deeper insight into how well a model captures data distribution.

33 Model Evaluation

33.1 Overview

We have evaluated our model visually. Now, we want to numerically evaluate how well our model fits the data. To do this, we use in-sample evaluation measures. These measures allow us to verify how well the model explains the variation in the target variable.

Two commonly used evaluation metrics are:

- **Mean Squared Error (MSE)**
- **R-squared (Coefficient of Determination)**

33.2 Mean Squared Error (MSE)

MSE measures how close the predicted values are to the actual values.

33.2.1 Steps to Calculate MSE

1. Subtract the predicted value ($\hat{y}$) from the actual value (y).
2. Square the result.
3. Repeat for all samples.
4. Compute the average of all squared errors.

33.2.2 Example

- Actual value (y): 150
- Predicted value ($\hat{y}$): 50
- Difference: $150 - 50 = 100$
- Squared difference: $100^2 = 10,000$

33.2.3 Python Implementation

```
from sklearn.metrics import mean_squared_error  
mse = mean_squared_error(y_actual, y_predicted)
```

A **lower MSE** indicates that the model fits the data better.

33.3 R-squared (Coefficient of Determination)

R-squared measures how close the data is to the fitted regression line. It tells us how much of the variation in the target variable is explained by our model.

33.3.1 Concept

We compare the performance of our regression model to a simple baseline model that uses the mean of the target variable.

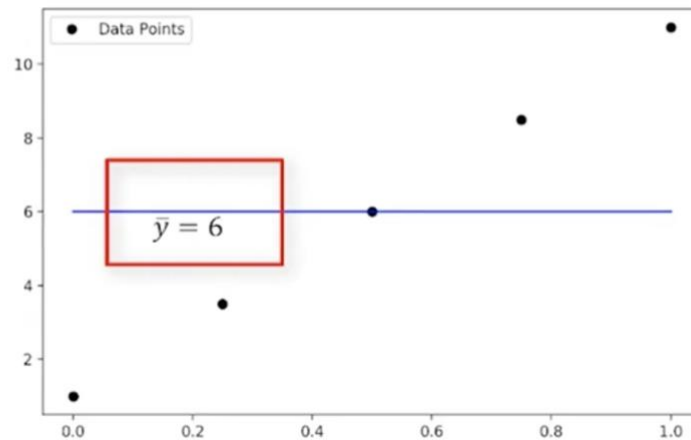
33.3.2 Formula

$$R^2 = 1 - (\text{MSE}_{\text{regression}} / \text{MSE}_{\text{average}})$$

33.3.3 Interpretation

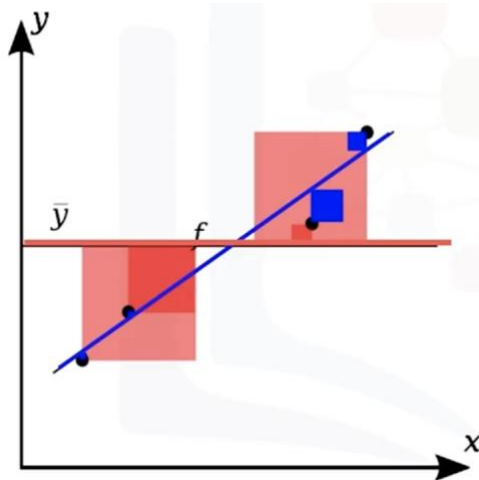
- **R^2 close to 1** → The model fits the data well.
- **R^2 close to 0** → The model performs similarly to just using the mean.
- **Negative R^2** → The model performs worse than using the mean (may indicate overfitting).

- In this example the average of the data points $\bar{y}$ is 6



33.3.4 Example Evaluation

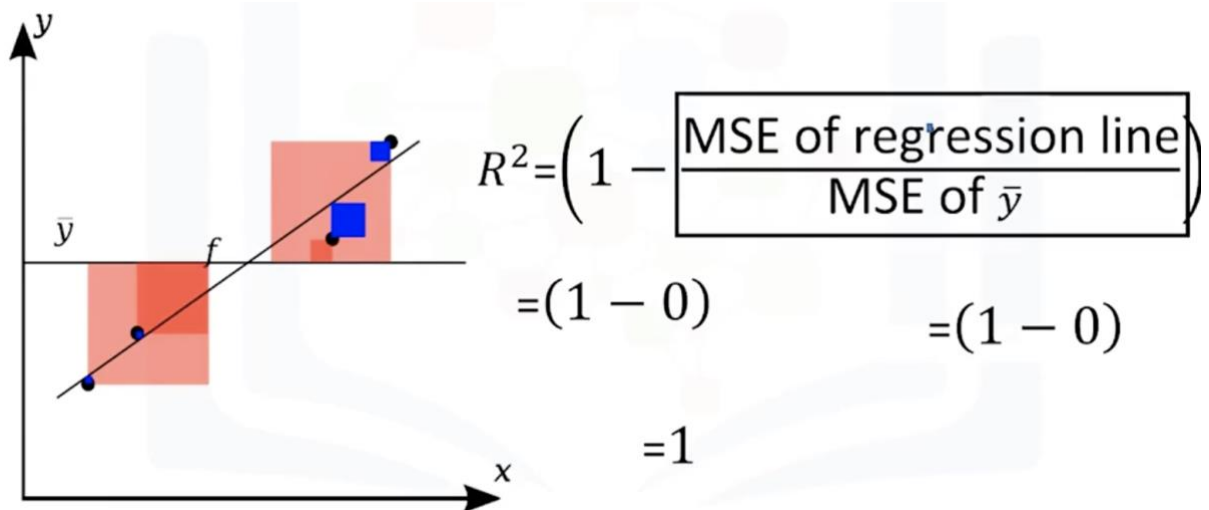
- A good regression line has a **small mean squared error**, resulting in a high R^2 value close to 1.



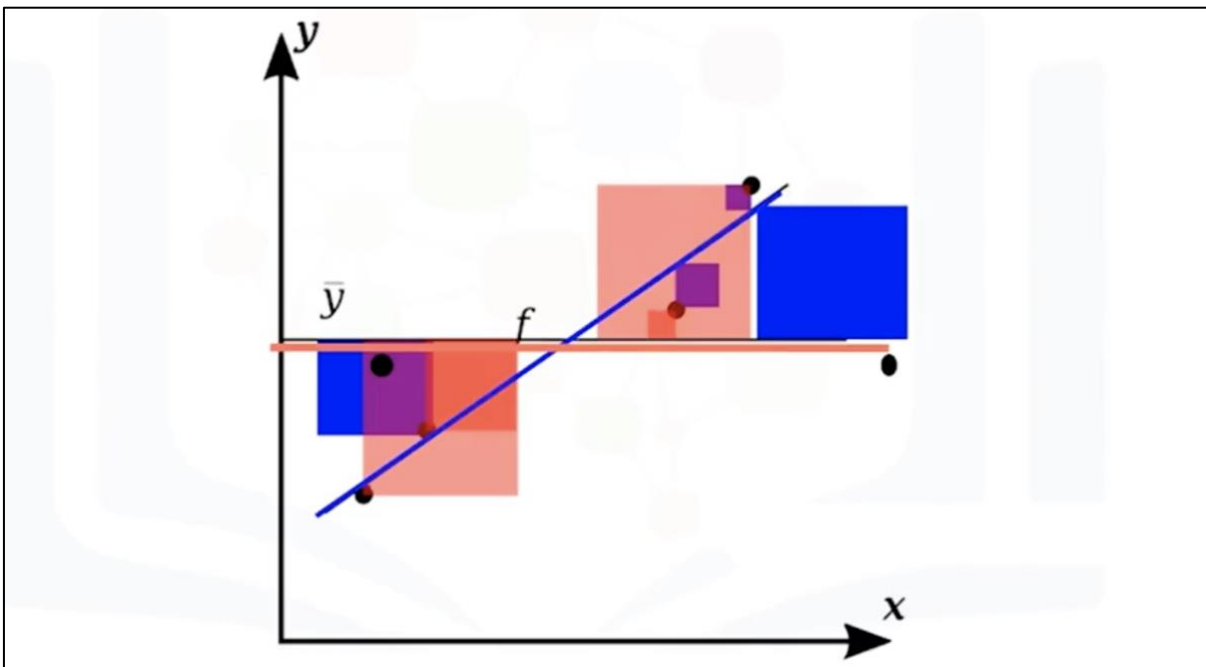
- The blue line represents the regression line
- The blue squares represents the MSE of the regression line
- The red line represents the average value of the data points
- The red squares represent the MSE of the red line
- We see the area of the blue squares is much smaller than the area of the red squares

- In this case ratio of the areas of MSE is close to zero

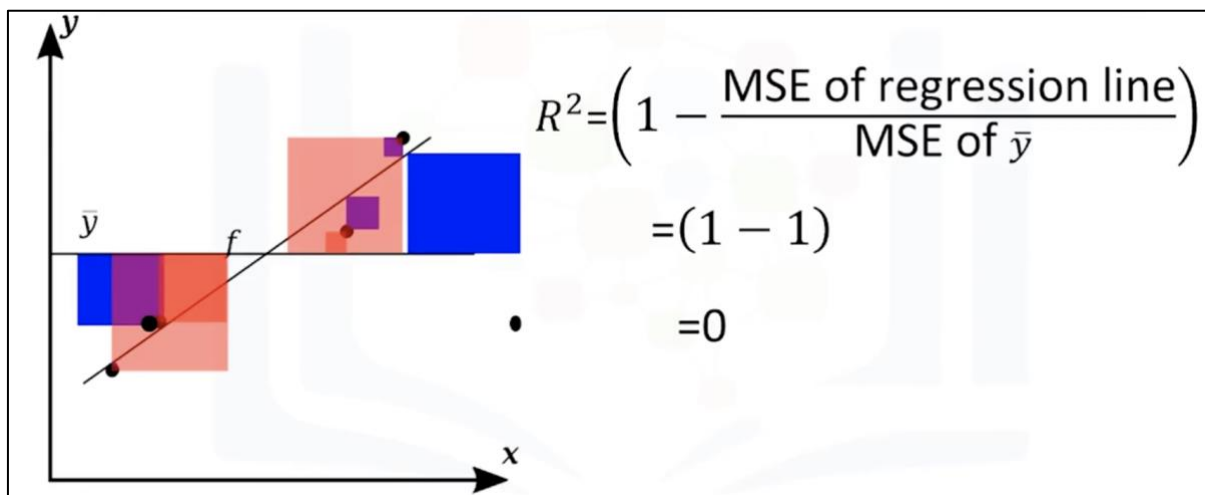
$$\frac{\text{MSE of regression line}}{\text{MSE of } \bar{y}} = \frac{\text{[Small Blue Squares]}}{\text{[Large Red Squares]}} = 0$$



- A poor regression line has an MSE similar to the baseline model, resulting in an R^2 near 0.



$$\frac{\text{MSE of regression line}}{\text{MSE of } \bar{y}} = \frac{\text{Blue squares} + \text{Blue squares}}{\text{Red squares} + \text{Red squares}} = 1$$



33.3.5 Python Implementation

We can compute R-squared using the score method from the regression model:

```
r_squared = model.score(X, y)
```

If we obtain an R^2 value of approximately **0.49659**, we interpret this as:

- **About 49.659%** of the variation in the target variable is explained by our simple regression model.

33.4 Additional Note

If R^2 is negative, it may suggest **overfitting**, which we will discuss in the next module.

34 Prediction and Decision Making

34.1 Ensuring Model Validity

We want to evaluate whether our model makes sense. To do this, we:

- Use visualization
- Use numerical evaluation measures
- Compare different models

34.2 Example: Predicting Car Price

We previously trained a model using the fit method. Now, we want to predict the price of a car with a highway miles-per-gallon value of 30.

- Using the predict method returns an estimated price of **\$13,771.30**.
- This value makes sense because it is not extremely high or low.

34.3 Interpreting Coefficients

We can examine the coefficient using the `coef_` attribute.

- The coefficient indicates how much the price changes when highway miles-per-gallon increases by 1 unit.
- In this case, the price decreases by approximately **\$821** for each unit increase.

34.4 Unrealistic Predictions and Model Assumptions

If we extend the highway miles-per-gallon range from 0 to 100, the model returns negative prices.

- This happens because values in that range are not realistic for cars.
- The linearity assumption may not hold when we exceed the real-world data range.

34.5 Generating Sequences with NumPy

We can generate a sequence of values to test predictions.

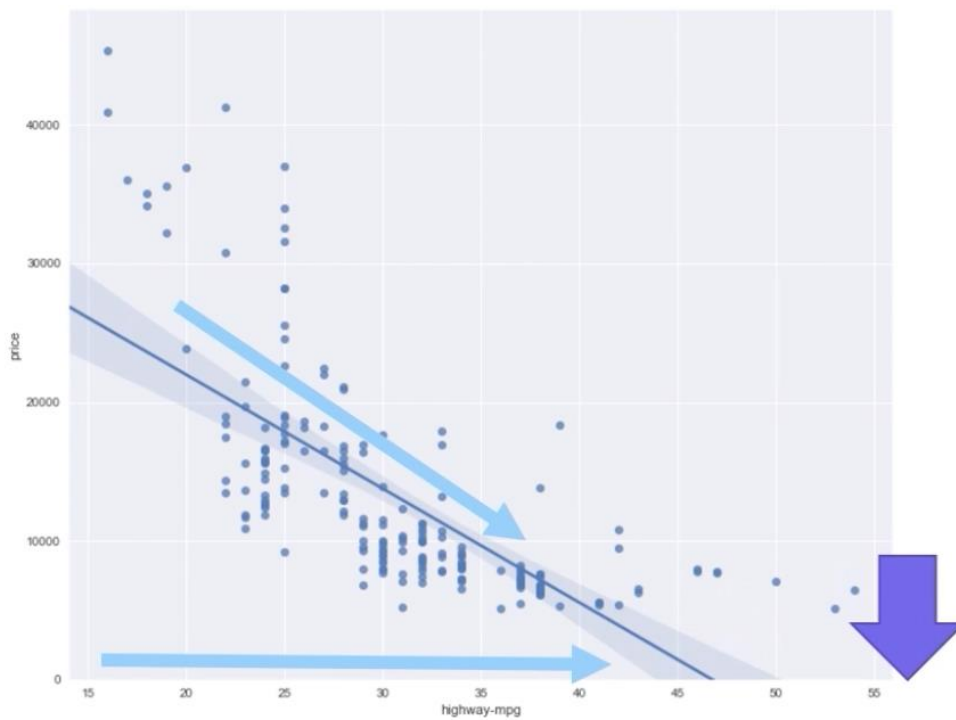
```
import numpy as np
x = np.arange(1, 101, 1)
yhat = lm.predict(new_input)
```

- 1 is the starting point.
- 101 is one more than the endpoint.
- 1 is the step size.
- This produces a NumPy array that can be used for making predictions.

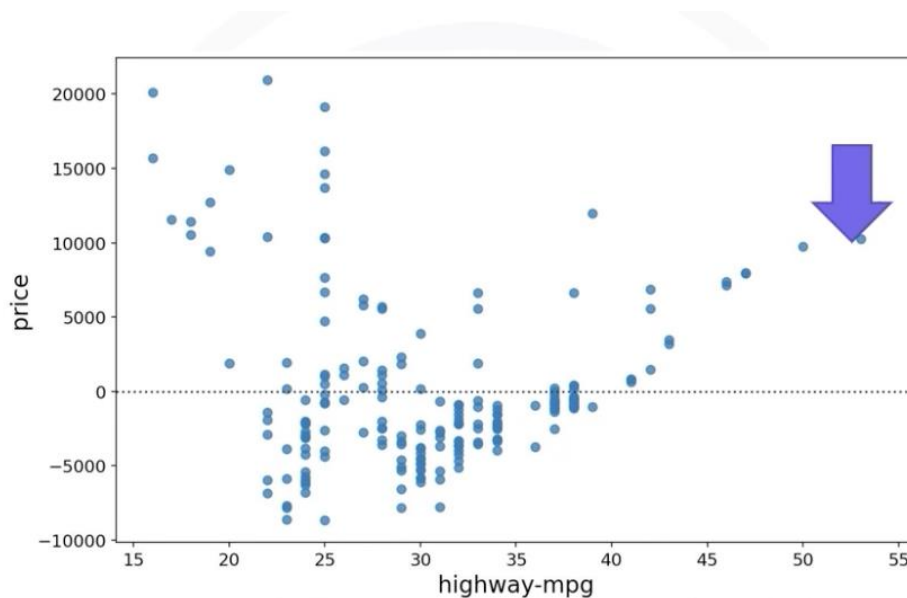
```
array([ 37601.57247984,  36779.83910151,  35958.10572319,  35136.37234487,
        34314.63896655,  33492.90558823,  32671.1722099 ,  31849.43883158,
        31027.70545326,  30205.97207494,  29384.23869662,  28562.50531829,
        27740.77193997,  26919.03856165,  26097.30518333,  25275.57180501,
        24453.83842668,  23632.10504836,  22810.37167004,  21988.63829172,
        21166.9049134 ,  20345.17153508,  19523.43815675,  18701.70477843,
        17879.97140011,  17058.23802179,  16236.50464347,  15414.77126514,
        14593.03788682,  13771.3045085 ,  12949.57113018,  12127.83775186,
        11306.10437353,  10484.37099521,  9662.63761689,  8840.90423857,
        8019.17086025,  7197.43748192,  6375.7041036 ,  5553.97072528,
        4732.23734696,  3910.50396864,  3088.77059031,  2267.03721199,
        1445.20000000,  622.50000000,  -199.50000000,  -821.50000000,
        -1841.62967962, -2663.36305794, -3485.09643626, -4306.82981458,
        -5128.5631929 , -5950.29657123, -6772.02994955, -7593.76332787,
        -8415.49670619, -9237.23008451, -10058.96346284, -10880.69684116,
        -11702.43021948, -12524.1635978 , -13345.89697612, -14167.63035445,
        -14989.36373277, -15811.09711109, -16632.83048941, -17454.56386773,
        -18276.29724606, -19098.03062438, -19919.7640027 , -20741.49738102,
        -21563.23075934, -22384.96413767, -23206.69751599, -24028.43089431,
        -24850.16427263, -25671.89765095, -26493.63102927, -27315.3644076 ,
        -28137.09778592, -28958.83116424, -29780.56454256, -30602.29792088,
        -31424.03129921, -32245.76467753, -33067.49805585, -33889.23143417,
        -34710.96481249, -35532.69819082, -36354.43156914, -37176.16494746,
        -37997.89832578, -38819.6317041 , -39641.36508243, -40463.09846075,
        -41284.83183907, -42106.56521739, -42928.29859571])
```

34.6 Visual Evaluation

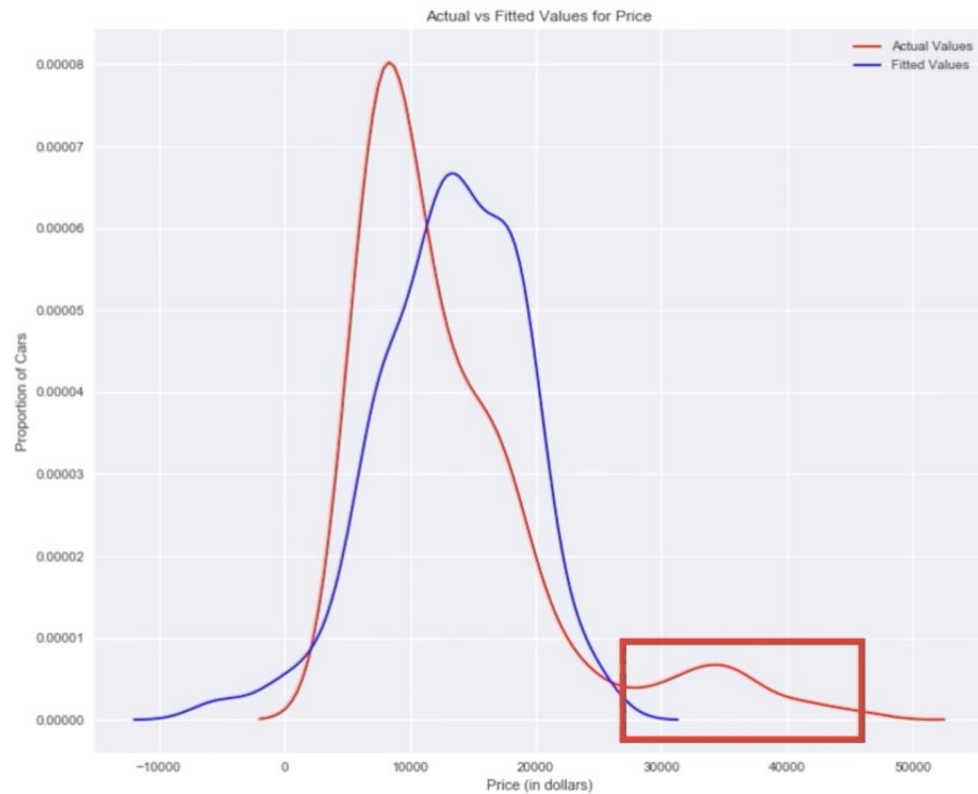
- A regression plot helps us understand the trend between variables.



- A residual plot helps us identify non-linear patterns.



- A distribution plot is helpful when working with multiple linear regression.



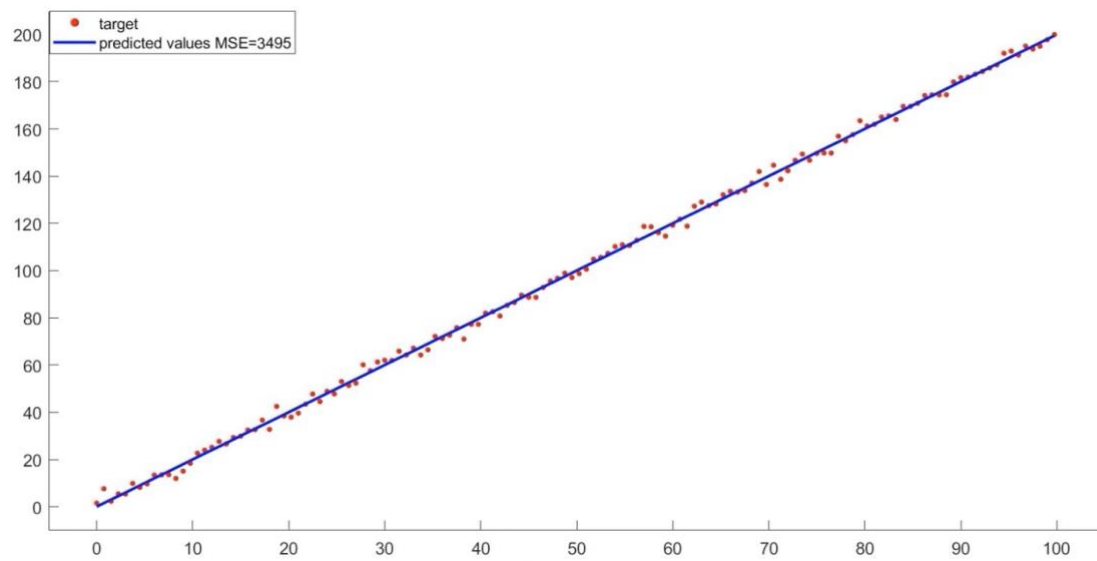
34.7 Mean Squared Error (MSE)

MSE is used to measure how far the predicted values are from the actual values.

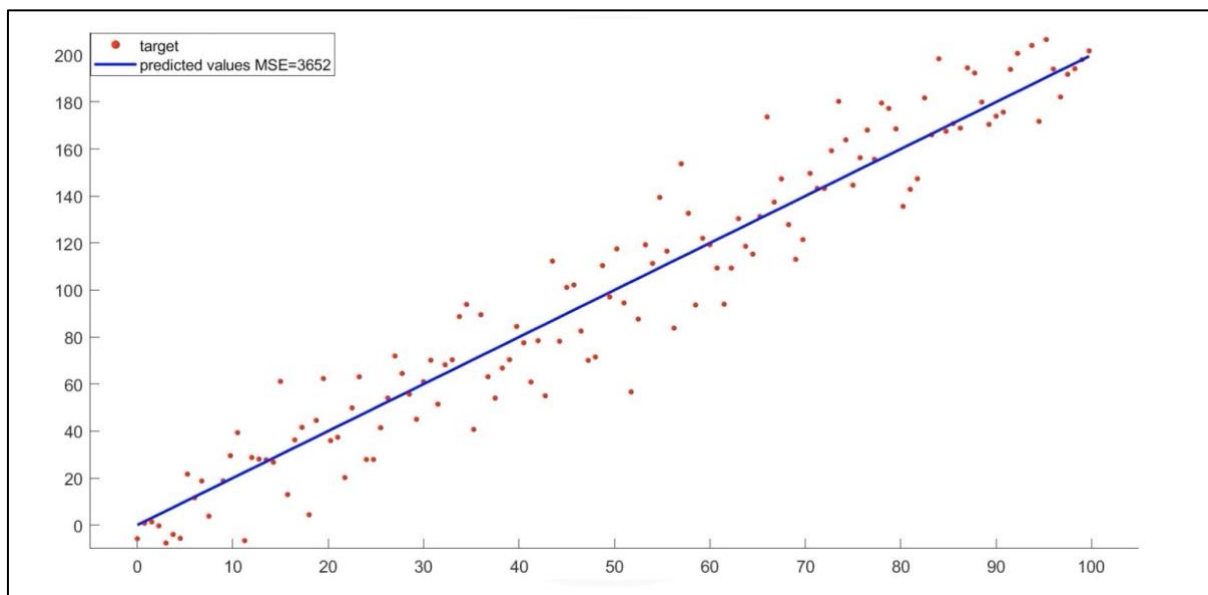
- A smaller MSE means the predicted values are closer to the actual values.
- Larger MSE means predictions are less accurate.

Examples:

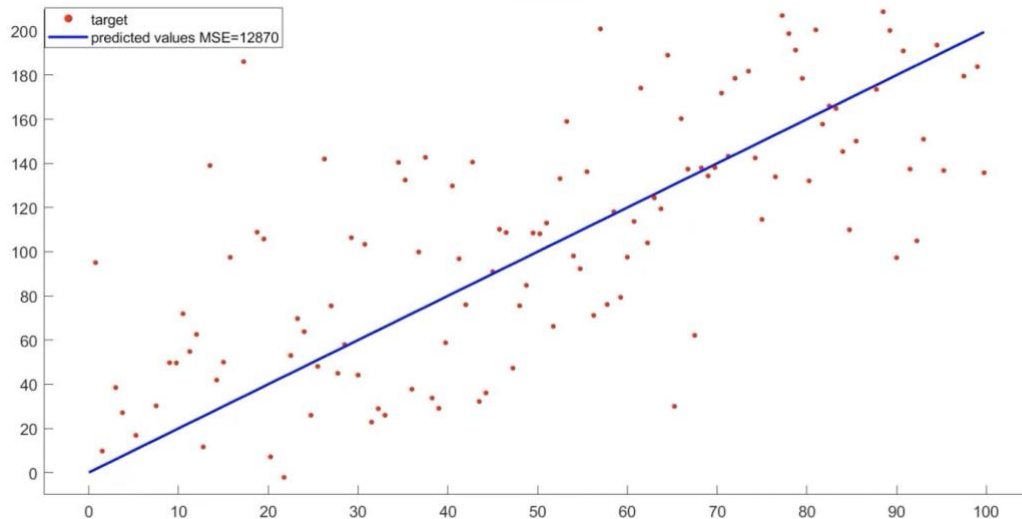
- $MSE = 3,495$ (good fit)



- MSE = 3,652 (slightly worse)



- MSE = 12,870 (poor fit)



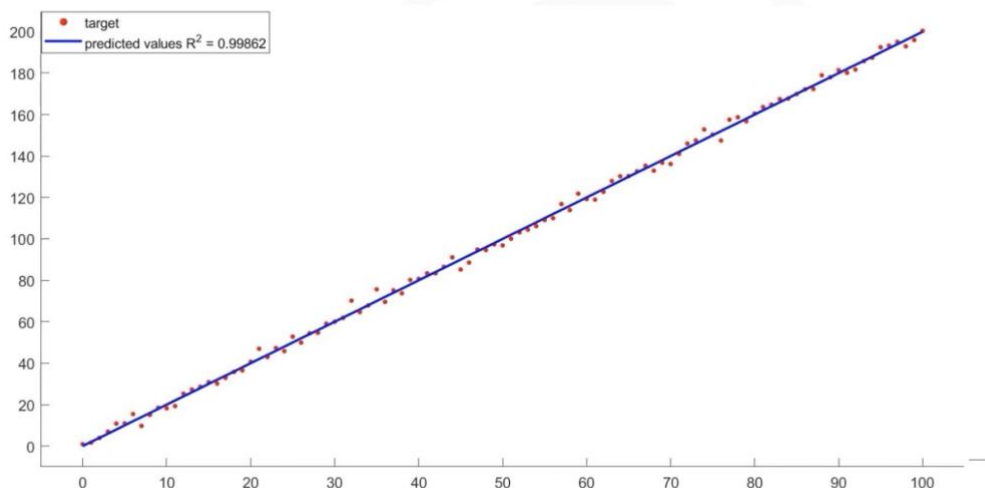
34.8 R-squared (Coefficient of Determination)

R-squared tells us how well the data fits the regression line.

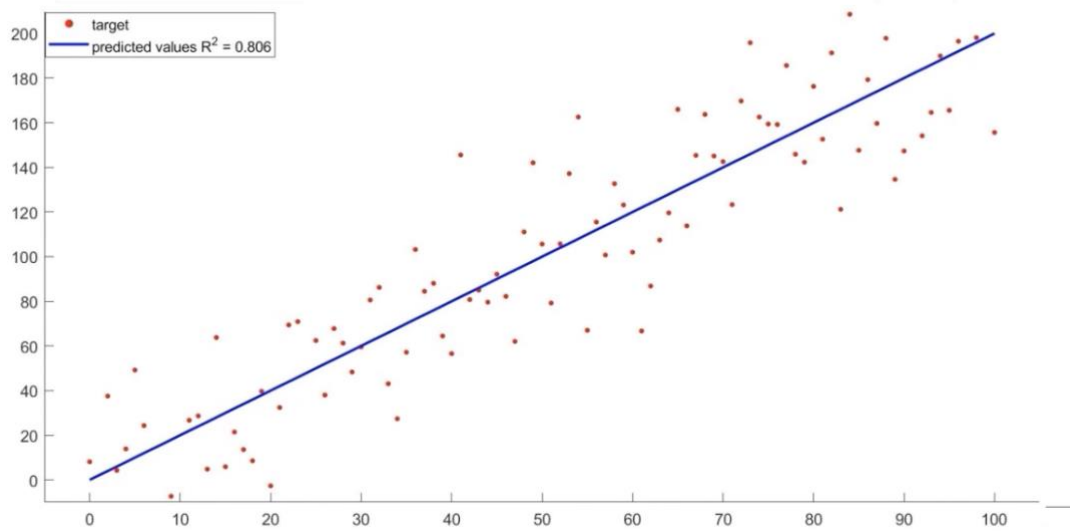
- **R-squared close to 1:** Strong relationship and good model fit.
- **R-squared close to 0:** Weak relationship; model does not explain much of the variation.

Example fits:

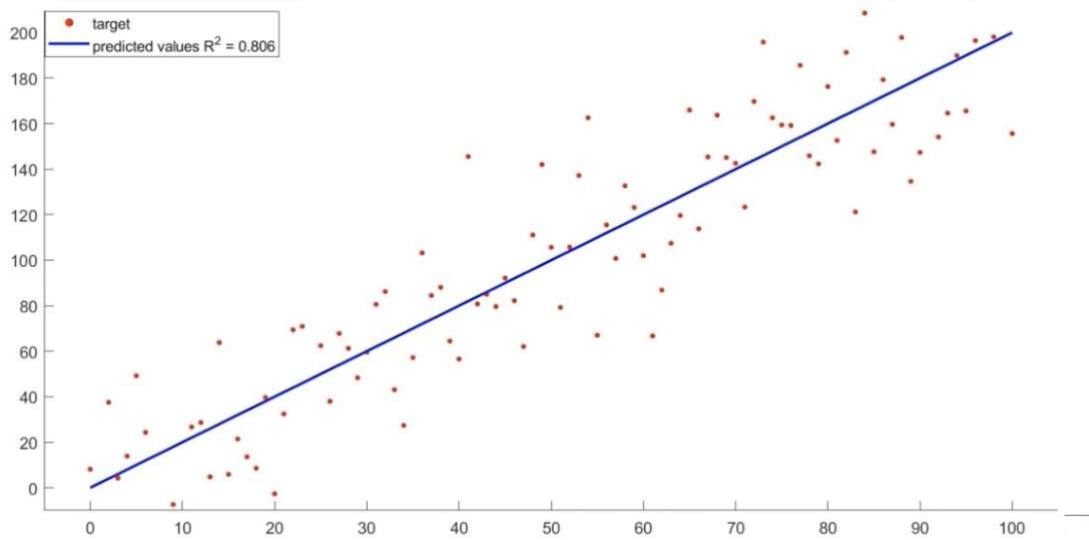
- R-squared = 0.9986 → Very good fit



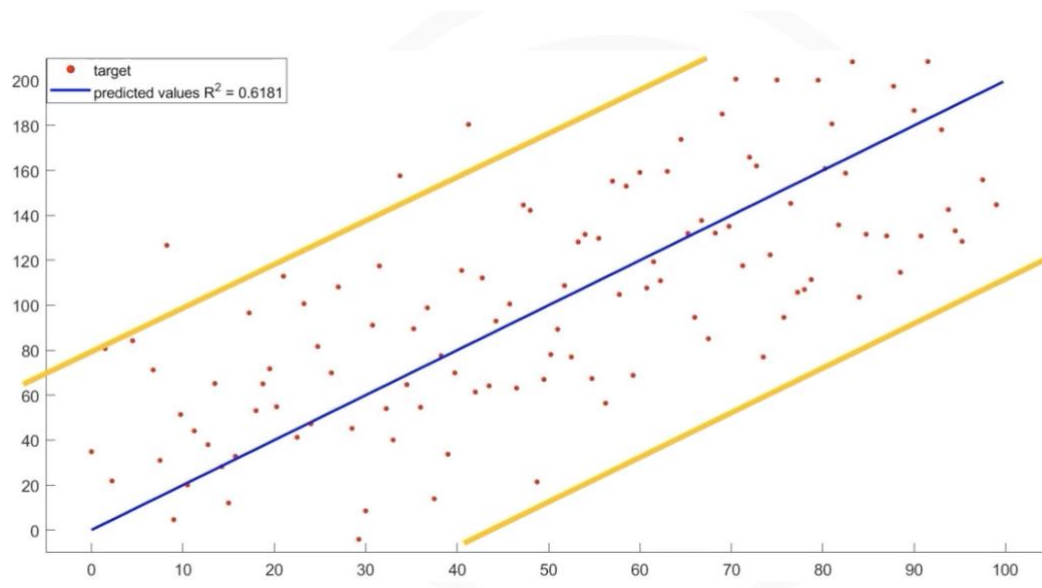
- R-squared = 0.9226 → Strong linear relationship



- $R\text{-squared} = 0.806 \rightarrow$ Weaker but visible linear trend



- $R\text{-squared} = 0.61 \rightarrow$ Linear pattern exists but is harder to see



Acceptable values vary by field, though some suggest **R-squared ≥ 0.10** .

34.9 Comparing Models

- Multiple Linear Regression (MLR) generally has a **lower MSE** compared to Simple Linear Regression (SLR) because it includes more predictors.
- Polynomial regression also tends to have **lower MSE** than simple linear models.
- However, lower MSE does not always mean the model is better.

34.10 Conclusion

We evaluate models using visualization and numerical methods such as MSE and R-squared. Depending on the complexity of the data, different models may perform better, and care must be taken when interpreting model performance.

35 Lesson Summary

35.1 Overview

We have completed this lesson. At this point, we understand key concepts related to simple and multiple linear regression, data transformation, model evaluation, and the use of pipelines to streamline modeling processes.

35.2 Key Concepts

35.2.1 Linear Regression

- Linear regression uses one independent variable to predict a dependent variable.
- Multiple linear regression involves one continuous target variable and two or more predictor variables.

35.2.2 Simple Linear Regression (SLR)

- Helps us understand the relationship between one independent variable (x) and one dependent variable (y).

35.2.3 Visualization Techniques

35.2.3.1 Residplot

- **regplot** and **residplot** (Seaborn) help analyze direction, strength, and linearity of relationships.
- When evaluating residual plots:
 - Residuals should have a mean of zero.
 - They should appear evenly distributed around the x-axis.
 - They should show consistent variance.

35.2.3.2 Distribution Plots

- Useful when working with multiple features.
- Help compare predicted and actual values.
- Provide insights into the performance of the model across various ranges.

35.2.4 Polynomial Regression

- The polynomial order impacts how well the model fits the data.
- We use Python's `polyfit` function to construct polynomial regression models.

35.2.5 Feature Transformation

- We can use **scikit-learn preprocessing** to transform features.
- Tools such as `PolynomialFeatures` and `StandardScaler` are used to normalize and enhance model performance.

35.2.6 Pipelines

- Pipelines automate sequential modeling steps such as normalization, polynomial transformation, and prediction.
- They simplify the process of preparing data and training models.

35.3 Model Evaluation

35.3.1 Mean Squared Error (MSE)

- Measures the average squared difference between actual and predicted values.

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_actual, y_predicted)
```

35.3.2 R-squared (Coefficient of Determination)

- Indicates how much variation in the dependent variable is explained by the model.

- R-squared close to **1** → good fit.
- R-squared close to **0** → weak fit.
- Negative R-squared may indicate **overfitting**.

35.3.3 Choosing a Good Model

- A good model has:
 - **High R-squared**
 - **Low MSE**
- Acceptable R-squared values vary depending on the use case.

35.3.4 Using Residual Plots for Evaluation

- Residuals should be randomly scattered around zero.
- If patterns or curves appear, it may indicate non-linearity or insufficient data.

35.4 Summary

- We evaluate models using both visual tools (regression and residual plots) and numerical measures (MSE, R-squared).
- Distribution plots are ideal when multiple features are involved.
- Always validate model behavior using both data transformation and evaluation techniques to ensure strong predictive performance.

Module 5

36 Model Evaluation and Cross Validation

36.1 Model Evaluation

Model evaluation tells us how our model performs in the real world.

In the previous module, we discussed **in-sample evaluation**. In-sample evaluation tells us how well our model fits the data already given to train it. However, it does not provide an estimate of how well the trained model can predict new data.

The solution is to split our data into two parts:

36.1.1 Training Data

The **training data** (in-sample data) is used to train the model.

36.1.2 Testing Data

The **testing data** (out-of-sample data) is used to approximate how the model performs in the real world.

This separation of data into training and testing sets is an important part of model evaluation.

36.2 Training and Testing Sets

Usually, the larger portion of the dataset is used for training, and a smaller portion is used for testing.

A common split is:

- **70% for training**
- **30% for testing**

The training set is used to:

- build the model
- discover predictive relationships

The testing set is used to:

- Evaluate model performance
- Estimate real-world performance

After completing the testing phase, we should use **all available data to train the final model**.

36.3 Train Test Split Function

A popular function in the Scikit-learn package for splitting datasets is `train_test_split()`.

```
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(  
    x_data,  
    y_data,  
    test_size=0.3,  
    random_state=1  
)
```

36.3.1 Parameters

- `x_data` → predictor variables
- `y_data` → target variable
- `test_size=0.3` → 30% data for testing
- `random_state` → random seed for reproducibility

36.3.2 Output

- `x_train` → training input data
- `y_train` → training target data
- `x_test` → testing input data
- `y_test` → testing target data

36.4 Generalization Error

Generalization error is a measure of how well our model predicts previously unseen data.

The error obtained using testing data is an approximation of this error.

In simple words, it tells us:

how the model is expected to perform in the real world

36.5 Accuracy and Precision in Data Splitting

Using a large amount of data for training gives us a more accurate estimate of performance, but the precision may be low.

For example:

- **90% training**
- **10% testing**

This may provide a good approximation of generalization error, but repeated experiments can produce different results.

This means:

- **accuracy is high**
- **precision is low**

If we use fewer data points for training and more for testing:

- accuracy becomes lower
- precision becomes higher

This means the results are more consistent but may be farther from the true generalization performance.

36.6 Cross Validation

To overcome this limitation, we use **cross-validation**.

Cross-validation is one of the most common out-of-sample evaluation techniques.

In this method, the dataset is split into **k equal groups**, called **folds**.

For example:

- 4 folds

Each group is called a fold.

Some folds are used for training, and the remaining fold is used for testing.

For example:

- 3 folds for training
- 1 fold for testing

This process is repeated until every fold has been used for both training and testing.

At the end, we use the **average result** as the estimate of out-of-sample error.

36.7 Cross Validation Score

The simplest way to apply cross-validation is by using `cross_val_score()`.

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
import numpy as np
```

```
lr = LinearRegression()
```

```
scores = cross_val_score(lr, x_data, y_data, cv=3)
```

```
average_score = np.mean(scores)
```

36.7.1 Parameters

- `lr` → model object
- `x_data` → predictor variable data
- `y_data` → target variable data
- `cv=3` → split data into 3 folds

The function returns an **array of scores**, one for each fold used as testing data.

Example:

```
[0.72, 0.75, 0.70]
```

Average score:

```
average_score = np.mean(scores)
```

This average helps estimate the **out-of-sample R^2 value**.

R^2

36.8 Cross-Validation Process

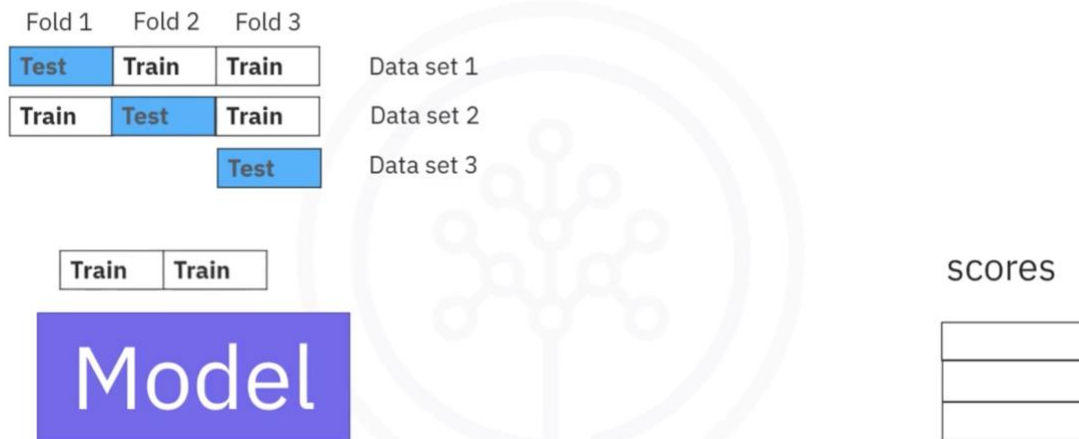
The process works as follows:

- Split the data into 3 folds
- Use 2 folds for training
- Use 1 fold for testing
- calculate score
- Repeat with different fold combinations
- store each score

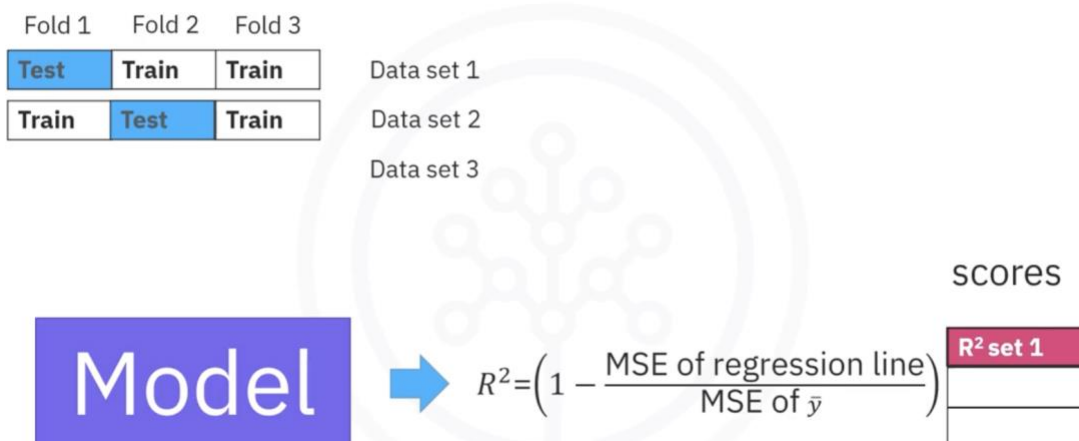
- Average all scores

This gives a better estimate of model performance.

Evaluating cross-validation score



Evaluating cross-validation score



36.9 Cross-Validation Prediction

If we want the actual predicted values before calculating the R^2 score, we use `cross_val_predict()`.

```
from sklearn.model_selection import cross_val_predict
```

```
predictions = cross_val_predict(lr, x_data, y_data, cv=3)
```

The input parameters are the same as `cross_val_score()`.

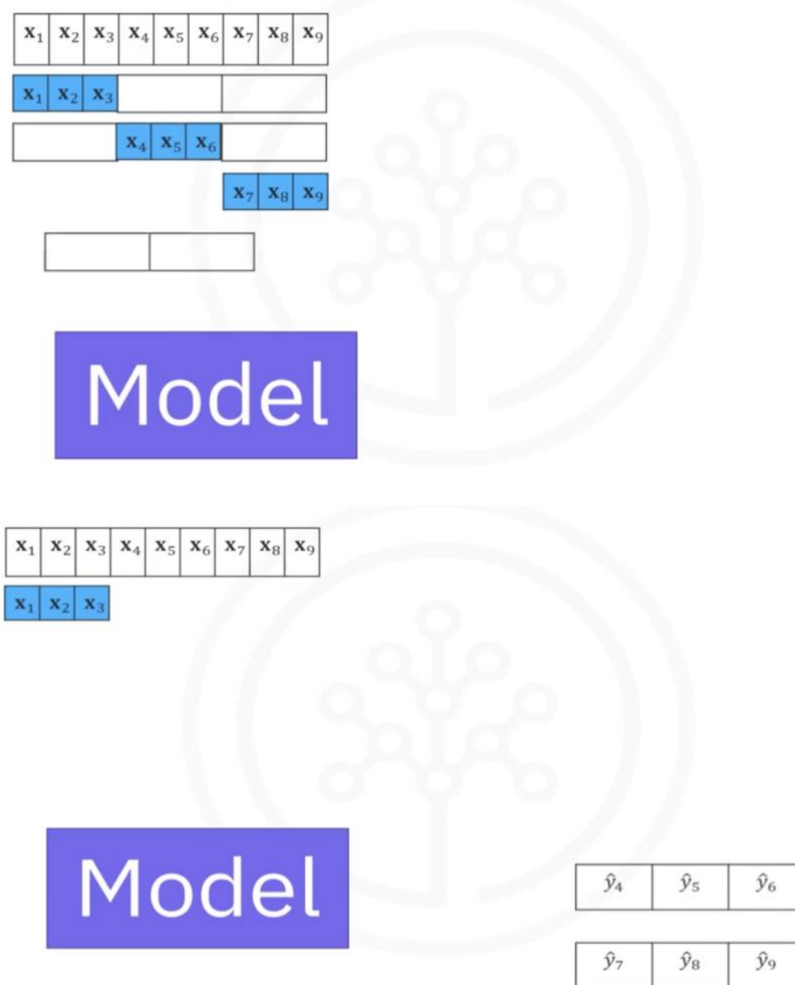
The output is a **prediction array**.

36.10 Cross-Validation Prediction Process

The process is:

- split data into 3 folds
- use 2 folds for training
- use 1 fold for testing
- store predictions
- repeat for all folds
- combine all predicted outputs into one array

These predictions are then stored in a single array for further analysis.



37 Polynomial Order Selection, Underfitting, and Overfitting

37.1 Data Analysis with Python

In this section, we discuss **how to choose the best polynomial order** and understand the problems that arise when the wrong order is selected.

37.2 Polynomial Regression and Model Selection

In polynomial regression, the goal is to find the polynomial order that provides the **best estimate of the function**.

The training points are usually assumed to come from:

- an actual polynomial function
- plus some random noise

The purpose of model selection is to determine the order of the polynomial that best estimates the target function.

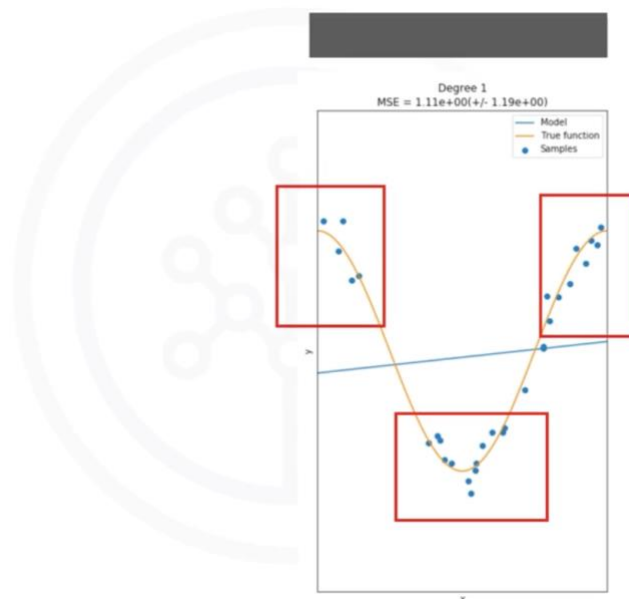
37.3 Underfitting

If we fit the data using a **linear function**, the model is too simple and cannot capture the complexity of the data.

This leads to many prediction errors.

This problem is called **underfitting**.

$$y = b_0 + b_1 x$$



 Skills Network

Definition:

Underfitting occurs when the model is **too simple to fit the data properly**.

Even if we slightly increase the order of the polynomial, the model may still remain too simple and continue to underfit.

37.4 Good Fit

When the polynomial order is increased to an appropriate level, the model begins to fit the data well.

For example, an **8th-order polynomial** may:

- fit the training points well
- estimate the curve accurately
- handle inflection points effectively

This is considered a good fit because it balances flexibility and generalization.

37.5 Overfitting

If the polynomial order is increased too much, such as to a **16th-order polynomial**, the model becomes too flexible.

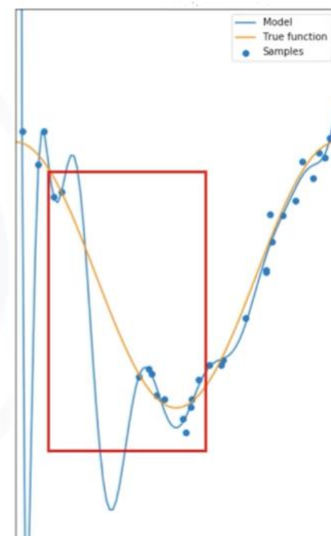
It may track every training point very closely, including the noise.

As a result, it performs poorly when estimating the actual function, especially in regions where training data is limited.

This causes oscillations and unstable predictions.

This problem is called **overfitting**.

$$\hat{y} = b_0 + b_1 x + b_2 x^2 + b_3 x^3 + b_4 x^4 + b_5 x^5 + b_6 x^6 + b_7 x^7 + b_8 x^8 + \dots + b_9 x^9 + b_{10} x^{10} + b_{11} x^{11} + b_{12} x^{12} + b_{13} x^{13} + b_{14} x^{14} + b_{15} x^{15} + b_{16} x^{16}$$



Definition:

Overfitting occurs when the model is **too flexible** and starts **fitting the noise** instead of the **true function**.

37.6 Mean Squared Error and Polynomial Order

To choose the best polynomial order, we analyze the **Mean Squared Error (MSE)** for both training and testing data.

$$\mathrm{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

37.6.1 Error Behavior

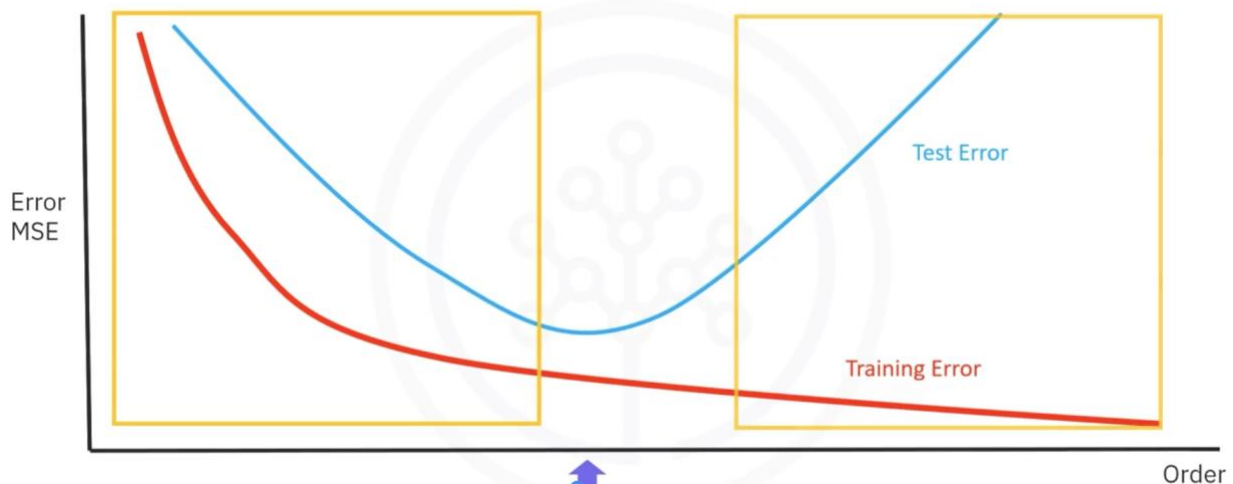
- **Training error** usually decreases as the polynomial order increases
- **Testing error** decreases up to the optimal order
- After the best order, testing error starts increasing

The best polynomial order is the one that **minimizes the testing error**.

For example:

- left side of optimal point → **underfitting**
- right side of optimal point → **overfitting**

Model Selection



37.7 Irreducible Error

Even after selecting the best polynomial order, some error still remains.

One major reason is **random noise in the data**.

This error cannot be eliminated because it is unpredictable.

This is called **irreducible error**.

37.8 Model Assumption Error

Another source of error occurs when our polynomial assumption is incorrect.

For example, if the actual data follows a **sine wave**, a polynomial model may not fit it properly.

In such cases:

- The selected model type is wrong
- The data pattern is different

- prediction accuracy decreases

37.9 Real Data Example: Horsepower vs Price

When applying polynomial regression to real data, such as **horsepower and car price**, different polynomial orders produce different results.

37.9.1 Mean Model

Using only the mean of the data gives poor performance.

37.9.2 Linear Model

A linear function performs better than the mean model.

37.9.3 Second and Third Order

Both second and third-order polynomial models improve the fit.

37.9.4 Fourth Order

A fourth-order polynomial may introduce unrealistic behavior.

For example:

- around **200 horsepower**
- The predicted price suddenly decreases

This suggests possible overfitting or incorrect model behavior.

37.10 Using R^2 for Model Selection

To validate the best polynomial order, we use the **R^2 score**.

R^2

The closer the R^2 value is to **1**, the better the model.

In this example:

- **3rd order polynomial** gives the best R^2
- **4th order polynomial** causes a sharp drop in R^2

This confirms that the third order model is better.

37.11 Code for Finding Best Polynomial Order

```
Rsqu_test = []
```

```
orders = [1, 2, 3, 4]

for n in orders:
    pr = PolynomialFeatures(degree=n)

    x_train_pr = pr.fit_transform(x_train[['horsepower']])
    x_test_pr = pr.fit_transform(x_test[['horsepower']])

    lr.fit(x_train_pr, y_train)

    Rsqu_test.append(lr.score(x_test_pr, y_test))
```

37.12 Explanation of the Code

The code performs the following steps:

- creates an empty list to store R^2 values
- creates a list of polynomial orders
- loops through each order
- transforms training and testing data
- trains the regression model
- calculates R^2 on testing data
- stores the score in the list

This helps compare different polynomial orders and select the best one.

37.13 Key Summary

- **Underfitting** → model too simple
- **Overfitting** → model too complex
- **Best fit** → lowest test error
- **MSE** helps analyze the error trend
- **R^2** helps validate model quality
- Choose the order where the **test error is minimum, and R^2 is maximum**

38 Introduction to Ridge Regression

Ridge Regression is a technique used to **control overfitting in regression models**, especially when working with:

- **multiple independent features**
- **polynomial feature extrapolation**
- **highly correlated (collinear) features**

In such cases, it is common to have **colinear combinations of features**, which means some input variables are strongly related to one another.

This problem is known as **multicollinearity**.

When multicollinearity is left unchecked, the model may **fit the training data too closely**, which leads to **overfitting**.

Overfitting means that the model performs very well on training data but performs poorly on unseen or real-world data.

To control this issue, the feature set is **regularized using hyperparameters**.

38.1 What is Ridge Regression?

Ridge Regression is the process of **regularizing the feature set using the hyperparameter alpha (α)**.

$$\text{Loss} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^p \beta_j^2$$

The alpha value controls the amount of regularization applied to the model.

- **small alpha** → less regularization
- **large alpha** → stronger regularization

By increasing alpha, Ridge Regression **penalizes large coefficient values**, which helps:

- reduce standard errors
- reduce model complexity
- avoid overfitting
- improve generalization performance

39 Ridge Regression

39.1 Introduction

In this section, we discuss **Ridge Regression**, a technique used to **prevent overfitting** in machine learning models.

Although the explanation here focuses mainly on **polynomial regression for visualization**, overfitting is also a major issue when working with **multiple independent variables (features)**.

39.2 Overfitting in Polynomial Regression

Consider a **fourth-order polynomial function** as the true function.

The data points are generated from this function, and we may try to fit them using a **10th-order polynomial**.

In an ideal case, the estimated function closely approximates the true function.

However, in real-world data, **outliers** often exist.

For example, if one data point does not follow the actual pattern, fitting a **10th-order polynomial** may produce an incorrect estimated function.

This leads to **overfitting**, where the model fits noise and outliers instead of the true pattern.

A major sign of overfitting is that the **polynomial coefficients become very large in magnitude**, especially for higher-order terms.

39.3 How Ridge Regression Solves Overfitting

Ridge Regression controls the magnitude of polynomial coefficients by introducing a parameter called **Alpha (α)**.

Alpha is selected **before training the model**.

As the value of Alpha increases:

- Coefficients become smaller
- Higher-order polynomial terms are controlled
- Overfitting reduces

This effect is especially noticeable in **higher-order polynomial features**.

Ridge Regression

$$\hat{y} = 1 + 2x - 3x^2 - 2x^3 - 12x^4 - 40x^5 + 80x^6 + 71x^7 - 141x^8 - 38x^9 + 75x^{10}$$

Alpha	x	x^2	x^3	x^4	x^5	x^6	x^7	x^8	x^9	x^{10}
0	2	-3	-2	-12	-40	80	71	-141	-38	75
0.001	2	-3	-7	5	4	-6	4	-4	4	6
0.01	1	-2	-5	-0.04	0.15	-1	1	-0.5	0.3	1
1	0.5	-1	-1	-0.614	0.70	-0.38	-0.56	-0.21	-0.5	-0.1
10	0	-0.5	-0.3	-0.37	-0.30	-0.30	-0.22	-0.22	-0.22	-0.17

39.4 Effect of Alpha Values

Alpha must be selected carefully.

39.4.1 If Alpha is Too Small

- Overfitting remains
- Coefficients remain large

For example:

- $\alpha = 0 \rightarrow$ clear overfitting
- $\alpha = 0.001 \rightarrow$ overfitting starts reducing
- $\alpha = 0.01 \rightarrow$ model tracks the actual function well

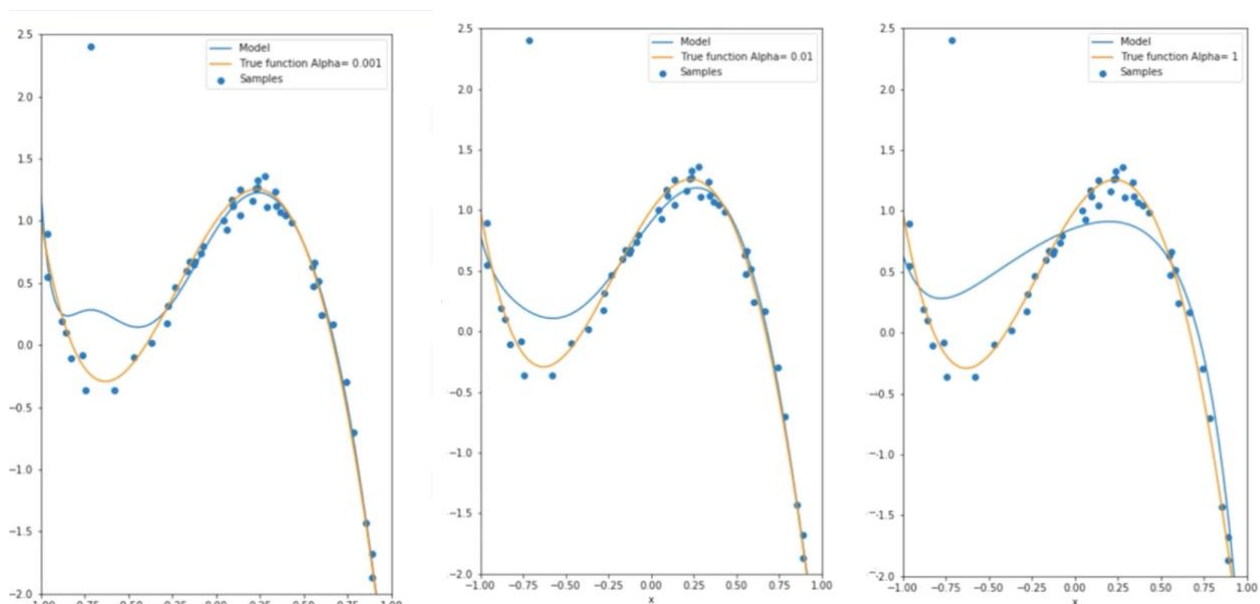
39.4.2 If Alpha is Too Large

- Coefficients approach zero
- Model loses flexibility
- Underfitting occurs

For example:

- $\alpha = 1 \rightarrow$ signs of underfitting begin
- $\alpha = 10 \rightarrow$ extreme underfitting

At very high Alpha values, the model may fail to even follow the data points properly.



39.5 Ridge Regression in Scikit-learn

To use Ridge Regression in Python:

```
from sklearn.linear_model import Ridge
```

```
ridge_model = Ridge(alpha=0.01)
```

```
ridge_model.fit(X_train, y_train)
```

```
predictions = ridge_model.predict(X_test)
```

39.6 Selecting the Best Alpha

To choose the best Alpha value, we use **validation data**.

Validation data is similar to test data, but it is specifically used for **hyperparameter selection**.

The process is:

- Start with a small Alpha
- Train the model
- Predict using validation data
- Calculate the evaluation metric
- Store the result
- Repeat for different Alpha values

The Alpha value that gives the **best performance score** is selected.

Common metrics include:

- **R² Score**
- **Mean Squared Error (MSE)**

39.7 Alpha Selection Using R²

A common approach is to calculate R² for multiple Alpha values and select the one that maximizes the score.

R²

The selected Alpha is the one that gives the **highest validation R² score**.

39.8 Behavior of R² with Alpha

When working with multiple features and polynomial terms:

- As Alpha increases, **validation R² may improve**
- It often converges to a stable value
- Example convergence: **R² ≈ 0.75**

Beyond a certain Alpha, increasing it further has very little impact.

In such cases, we select the Alpha value at or near the maximum stable score.

39.9 Training vs Validation Performance

As Alpha increases:

- **Training performance may decrease**
- **Validation performance may improve**

This happens because Ridge Regression reduces overfitting.

Although the model may fit training data less perfectly, it performs better on **unseen real-world data**.

This is the main purpose of Ridge Regression.

40 Grid Search and Hyperparameter Tuning

40.1 Data Analysis with Python

Grid Search is a technique used to **automatically test multiple hyperparameter values** and select the best-performing combination using **cross-validation**.

40.2 Introduction to Grid Search

Grid Search allows us to scan through **multiple free parameters with only a few lines of code**.

Parameters such as **alpha** are not part of the normal fitting or training process. These values are called **hyperparameters**.

Scikit-learn provides a way to automatically iterate through these hyperparameters using **cross-validation**.

This method is called **Grid Search**.

Grid Search takes:

- a **model or object to train**
- different **hyperparameter values**
- an **evaluation metric** such as:
 - **Mean Squared Error (MSE)**
 - **R² score**

It then evaluates all possible values and helps us choose the **best combination**.

40.3 What are Hyperparameters?

Hyperparameters are **configurable values set before training the model**.

Examples include:

- alpha
- normalize
- number of folds in cross-validation

These are different from model parameters learned during training.

40.4 How Grid Search Works

The process works as follows:

- Start with one hyperparameter value
- Train the model
- Evaluate performance
- Repeat for different values

- Compare scores
- Select the best value

Each model produces an error score.

The best hyperparameter is the one that:

- **minimizes error**
- **maximizes R^2 score**

40.5 Dataset Split for Grid Search

To select the best hyperparameter, the dataset is divided into:

- **Training Set**
- **Validation Set**
- **Test Set**

40.5.1 Training Set

Used to train the model

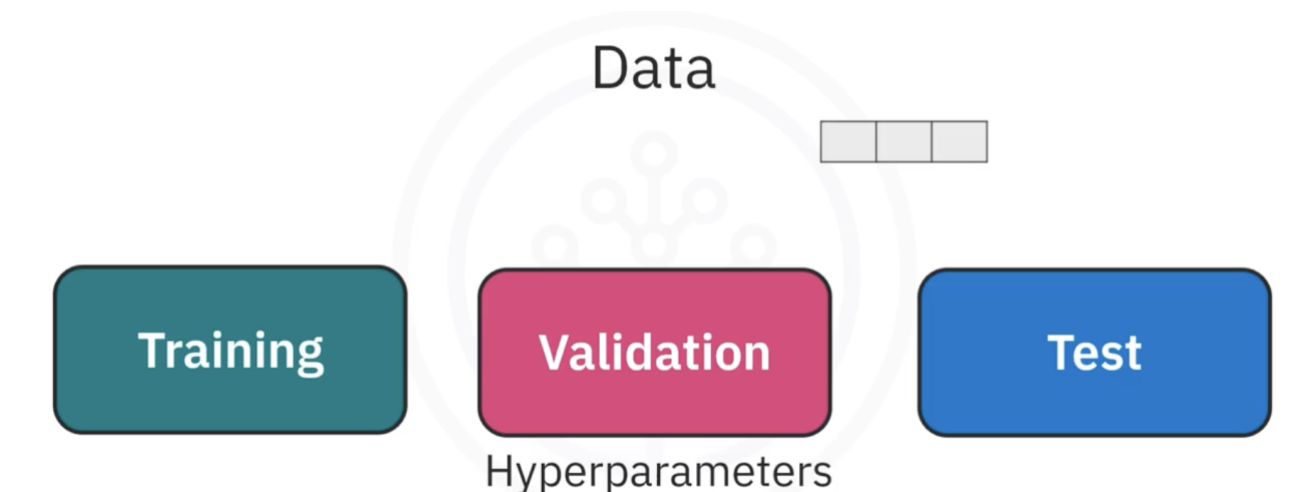
40.5.2 Validation Set

Used to compare different hyperparameter values

40.5.3 Test Set

Used for final performance evaluation

The best hyperparameter is selected based on validation performance.



40.6 Parameter Grid

The parameter values are stored in a **Python dictionary inside a list**.

Example:

```
parameters = [{'alpha': [0.001, 0.01, 0.1, 1, 10, 100]}]
```

Here:

- **key** → parameter name
- **value** → list of possible values

This acts like a **table or grid of values**.

40.7 GridSearchCV in Scikit-learn

First, import the required libraries.

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV
```

Create the model:

```
RR = Ridge()
```

Create the grid search object:

```
Grid1 = GridSearchCV(RR, parameters, cv=4)
```

Where:

- **RR** → Ridge Regression model
- **parameters** → parameter grid
- **cv=4** → number of folds

40.8 Fit the Grid Search Model

```
Grid1.fit(x_data, y_data)
```

This trains the model on all possible parameter combinations.

40.9 Best Hyperparameter Values

We can find the best parameter values using:

```
best_model = Grid1.best_estimator_
```

This returns the model with the best-performing hyperparameters.

40.10 Access Validation Results

To get detailed validation scores:

`Grid1.cv_results_`

This stores the scores for all parameter combinations.

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

parameters1= [{ 'alpha': [0.001,0.1,1, 10, 100, 1000,10000,100000,100000]}]

RR=Ridge()

Grid1 = GridSearchCV(RR, parameters1,cv=4)

Grid1.fit(x_data[['horsepower', 'curb-weight', 'engine-size',
'highway-mpg']],y_data)

Grid1.best_estimator_

scores = Grid1.cv_results_
scores['mean_test_score']
```

40.11 Multiple Hyperparameters

One major advantage of Grid Search is that it can test **multiple parameters at the same time**.

Example:

```
parameters = [
    {
        'alpha': [0.001, 0.01, 0.1, 1, 10, 100],
        'normalize': [True, False]
    }
]
```

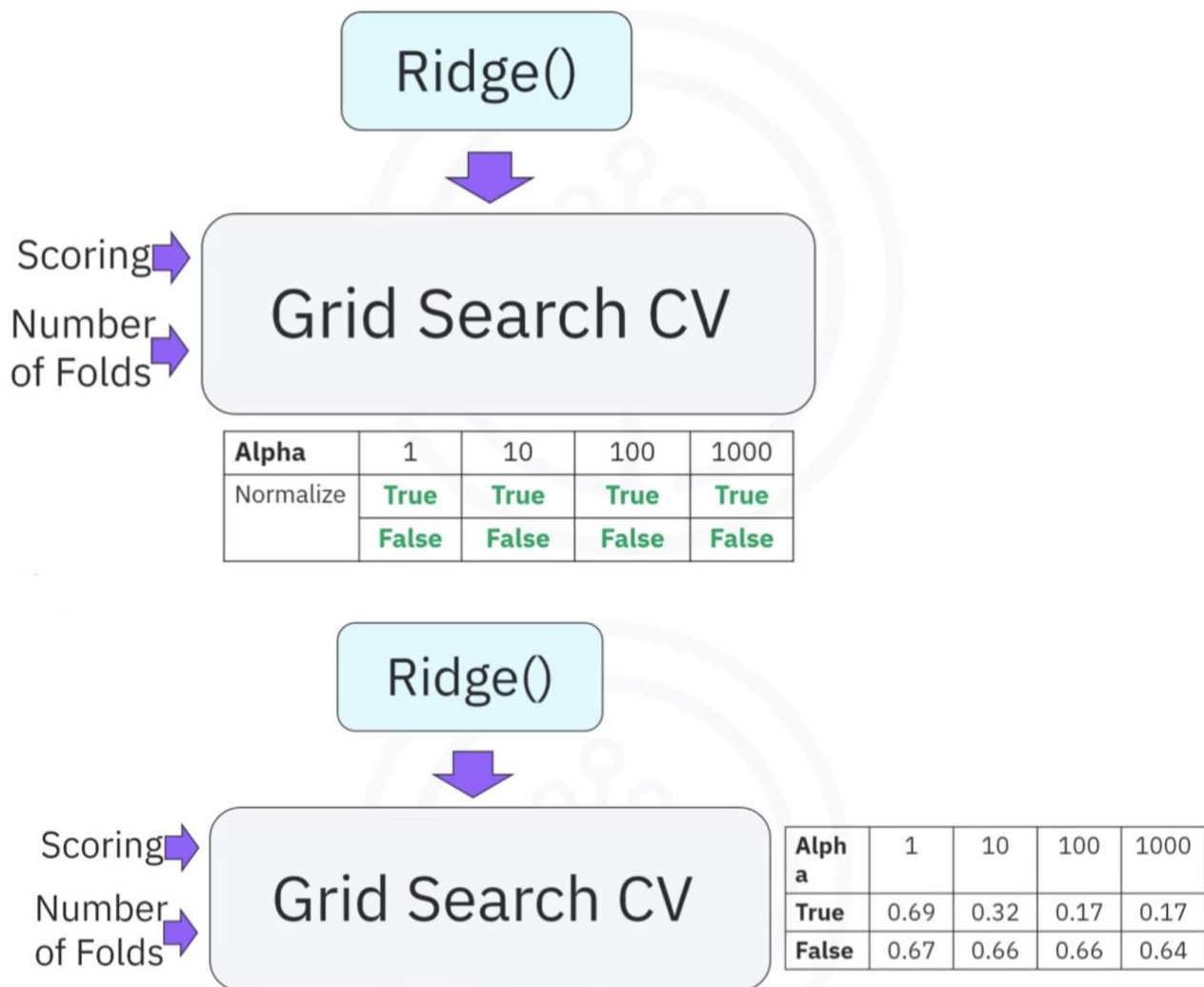
This creates a full **grid of combinations**.

For example:

- alpha = 0.01, normalize = True
- alpha = 0.01, normalize = False
- alpha = 0.1, normalize = True
- alpha = 0.1, normalize = False

and so on.

Grid Search tests **all combinations automatically**.



40.12 Score Results

The scores for all combinations are stored in:

`Grid1.cv_results_`

This includes:

- parameter values
- mean validation scores
- rank of each combination

40.13 Key Summary

- **Grid Search** → automatic hyperparameter tuning
- **Hyperparameters** → values set before training
- `GridSearchCV()` → performs tuning with cross-validation

- `best_estimator_` → best model
- `cv_results_` → all score results
- supports **multiple parameter combinations**

41 Lesson Summary

41.1 Train-Test Split

We know how to split data using the `train_test_split()` method into **training** and **testing sets**.

- The **training set** is used to train the model and discover predictive relationships.
- The **testing set** is used to evaluate the model's performance on unseen data.

This helps us understand how well the model is likely to perform in real-world situations.

41.2 Generalization Error

- We understand that **generalization error** measures how well the model predicts previously unseen data.
- This gives an estimate of the model's real-world predictive capability.

41.3 Cross Validation

We know how to use **cross-validation** by splitting the dataset into **folds**.

- Some folds are used as the **training set**
- The remaining fold(s) are used as the **testing set**

This process is repeated until each partition has been used for both training and testing.

At the end, the results are averaged to estimate the **out-of-sample error**.

41.4 Polynomial Order Selection

We understand how to choose the **best polynomial order** for fitting data.

This involves analyzing problems such as:

- **Underfitting**
- **Overfitting**

We select the best polynomial order by **minimizing the test error**.

A graph comparing **Mean Squared Error (MSE)** with the **order of fitted polynomials** helps in making this decision.

41.5 Ridge Regression

We now know that **ridge regression** should be used when there is a **strong relationship among independent variables**.

It is also important because it **prevents overfitting**.

Ridge regression controls the magnitude of polynomial coefficients by introducing a **hyperparameter alpha (α)**.

41.6 Selecting the Best Alpha Value

To determine the best value of alpha:

- Divide data into **training** and **validation sets**
- Start with a small value of alpha
- Train the model
- Make predictions using validation data
- Calculate **R^2**
- Store the result
- Repeat the process for larger alpha values

The best alpha is the one that **maximizes R^2** .

41.7 Grid Search

We understand that **Grid Search** helps scan through multiple hyperparameters using the Scikit-learn library.

It uses **cross-validation** to test different parameter combinations.

Based on the results, we select the **optimal hyperparameter values**.

41.8 GridSearchCV()

The GridSearchCV() method takes a **dictionary** as input.

- The **key** is the name of the hyperparameter
- The **values** are the hyperparameter values to be tested

This method automatically iterates through all possible values and helps identify the best combination.